Data Structures: A Pseudocode Approach with C (Gilberg and Forouzan)

Pages selected from the course reference book to match the DSA lecture curriculum.
Book page + 18 = PDF page. Text-searchable copy.

Contents (book page range : topic) in printed order:

pp. 5-17 Ch1 Pseudocode, atomic/composite data, ADT, array and linked implementations
pp. 28-38 Ch1 Algorithm efficiency, Big-O
pp. 45-57 Ch2 Recursion (factorial, design, GCD, Fibonacci)
pp. 79-125 Ch3 Stacks (linked, ADT, reverse, parsing, postfix)
pp. 134-138 Ch3 How recursion works
pp. 147-175 Ch4 Queues (operations, linked, ADT, application)
pp. 193-212 Ch5 General linear lists (linked list operations)
pp. 265-282 Ch6 Trees, binary trees, traversals, expression trees
pp. 299-309 Ch7 Binary search trees (search, insert, delete)
pp. 533-543 Ch12 Sorting concepts, selection sort
pp. 558-569 Ch12 Bubble sort and quick sort
pp. 597-611 Ch13 Sequential and binary search
pp. 675-680 App E Mathematical series and recursive relations
pp. 681-696 App F Array implementations of stacks and queues

Chapter 1
Basic Concepts

This text assumes that the student has a solid foundation in structured programming principles and has written programs of moderate complexity. Although the text uses C for all of its implementation examples, the design and logic of the data structure algorithms are based on pseudocode. This approach creates a language-independent environment for the algorithms.

In this chapter we establish a background for the tools used in the rest of the text, most specifically pseudocode, the abstract data type, algorithm efficiency analysis, and the concepts necessary to create generic code.

1.1 Pseudocode

Although several tools are used to define algorithms, one of the most common is **pseudocode**. Pseudocode is an English-like representation of the algorithm logic. It is part English, part structured code. The English part provides a relaxed syntax that describes *what* must be done without showing unnecessary details such as error messages. The code part consists of an extended version of the basic algorithmic constructs—sequence, selection, and iteration.

> One of the most common tools for defining algorithms is pseudocode, which is part English, part structured code.

In this text we use pseudocode to represent both data structures and code. Data items do not need to be declared. The first time we use a data name in an algorithm, it is automatically declared. Its type is determined by context. The following statement declares a numeric data item named `count` and sets its value to zero.

```
set count to 0
```

The structure of the data, on the other hand, must be declared. We use a simple syntactical statement that begins with a structure name and concludes with the keyword *end* and the name of the structure. Within the structure we list the structural elements by indenting the data items as shown below.

```
node
       data
       link
end node
```

This data definition describes a node in a self-referential list that consists of a nested structure (`data`) and a pointer to the next node (link). An element's type is implied by its name and usage in the algorithm.

As mentioned, pseudocode is used to describe an algorithm. To facilitate a discussion of the algorithm statements, we number them using the hierarchical system shown in Algorithm 1-1. The following sections describe the components of an algorithm. Colored comments provide documentation or clarification when required.

ALGORITHM 1-1 Example of Pseudocode

```
Algorithm sample (pageNumber)
This algorithm reads a file and prints a report.
   Pre    pageNumber passed by reference
   Post   Report Printed
          pageNumber contains number of pages in report
   Return Number of lines printed
 1 loop (not end of file)
   1   read file
   2   if (full page)
       1   increment page number
       2   write page heading
   3   end if
   4   write report line
   5   increment line count
 2 end loop
 3 return line count
end sample
```

Algorithm Header

Each algorithm begins with a header that names it, lists its parameters, and describes any preconditions and postconditions. This information is important because it serves to document the algorithm. Therefore, the header information must be complete enough to communicate to the programmer everything he or she must know to write the algorithm. In Algorithm 1-1 there is only one parameter, the page number.

Purpose, Conditions, and Return

The purpose is a short statement about what the algorithm does. It needs to describe only the general algorithm processing. It should not attempt to describe all of the processing. For example, in Algorithm 1-1 the purpose does not need to state that the file will be opened or how the report will be printed. Similarly, in the search example the purpose does not need to state which of the possible array searches will be used.

The precondition lists any precursor requirements for the parameters. For example, in Algorithm 1-1 the algorithm that calls `sample` must pass the page number by reference. Sometimes there are no preconditions, in which case we still list the precondition with a statement that nothing is required, as shown below.

```
Pre    nothing
```

If there are several input parameters, the precondition should be shown for each. For example, a simple array search algorithm has the following header:

```
Algorithm search  (list, argument, location)
Search array for specific item and return index location.
   Pre    list contains data array to be searched
          argument contains data to be located in list
   Post   location contains matching index
          -or- undetermined if not found
   Return true if found, false if not found
```

In `search` the precondition specifies that the two input parameters, list and argument, must be initialized. If a binary search were being used, the precondition would also state that the array data must be sorted.

The postcondition identifies any action taken and the status of any output parameters. In Algorithm 1-1 the postcondition contains two parts. First, it states that the report has been printed. Second, the reference parameter, `pageNumber`, contains the updated number of pages in the report. In the search algorithm shown above, there is only one postcondition, which may be one of two different values.

If a value is returned, it is identified by a **return condition**. Often there is none, and no return condition is needed. In Algorithm 1-1 we return the number of lines printed. The search algorithm returns true if the argument was found, false if it was not found.

Statement Numbers

Statements are numbered using an abbreviated decimal notation in which only the last of the number sequence is shown on each statement. The expanded number of the statement in Algorithm 1-1 that reads the file is 1.1.

The statement that writes the page heading is 1.2.2. This technique allows us to identify an individual statement while providing statements that are easily read.

Variables

To ensure that the meaning is understood, we use **intelligent data names**—that is, names that describe the meaning of the data. However, it is not necessary to define the variables used in an algorithm, especially when the name indicates the context of the data.

The selection of the name for an algorithm or variable goes a long way toward making the algorithm and its coded implementation more readable. In general, you should follow these rules:

1. Do not use single-character names.
2. Do not use generic names in application programs. Examples of generic names are `count`, `sum`, `total`, `row`, `column`, and `file`. In a program of any size there are several counts, sums, and totals. Rather, add an intelligent qualifier to the generic name so that the reader knows exactly to which piece of data the name refers. For example, `studentCount` and `numberOfStudents` are both better than `count`.
3. Abbreviations are not excluded as intelligent data names. For example, `stuCnt` is a good abbreviation for `student count`, and `numOfStu` is a good abbreviation for `number of students`. Note, however, that `noStu` would not be a good abbreviation for `number of students` because it is too easily read as *no students*.

Statement Constructs

When he first proposed the structured programming model, Edsger Dijkstra stated that any algorithm could be written using only three programming **constructs**: sequence, selection, and loop. Our pseudocode contains only these three basic constructs. The implementation of these constructs relies on the richness of the implementation language. For example, the loop can be implemented as a *while, do…while,* or *for* statement in the C language.

Sequence

A **sequence** is one or more statements that do not alter the execution path *within an algorithm*. Although it is obvious that statements such as assign and add are sequence statements, it is not so obvious that a call to other algorithms is also considered a sequence statement. The reason calls are considered sequential statements lies in the structured programming concept that each algorithm has only one entry and one exit. Furthermore, when an algorithm completes, it returns to the statement immediately after the call that invoked it. Therefore, we can consider an algorithm call a sequence statement. In Algorithm 1-1 statements 1.2.1 and 1.2.2 are sequence statements.

Selection

A **selection statement** evaluates a condition and executes zero or more alternatives. The results of the evaluation determine which alternates are taken.

The typical selection statement is the two-way selection as implemented in an *if* statement. Whereas most languages provide for multiway selections, such as the *switch* in C, we provide none in the pseudocode. The parts of the selection are identified by indentation, as shown in the short pseudocode statement below.

```
1 if (condition)
  1    action1
2 else
  1    action2
3 end if
```

Statement 1.2 in Algorithm 1-1 is an example of a selection statement. The end of the selection is indicated by the *end if* in statement 1.3.

Loop

A **loop** statement iterates a block of code. The loop that we use in our pseudocode closely resembles the *while* loop. It is a pretest loop; that is, the condition is evaluated before the body of the loop is executed. If the condition is true, the body is executed. If the condition is false, the loop terminates.

In Algorithm 1-1 statement 1 is an example of a loop. The end of the loop is indicated by *end loop* in statement 2.

Algorithm Analysis

For selected algorithms, we follow the algorithm with an analysis section that explains some of its salient points. Not every line of code is explained. Rather, the analysis examines only those points that either need to be emphasized or that may require some clarification. The algorithm analysis also often introduces style or efficiency considerations.

Pseudocode Example

As another example of pseudocode, consider the logic required to calculate the deviation from a mean. In this problem we must first read a series of numbers and calculate their average. Then we subtract the mean from each number and print the number and its deviation. At the end of the calculation, we also print the totals and the average.

The obvious solution is to place the data in an array as they are read. Algorithm 1-2 contains the code for this simple problem as it would be implemented in a callable algorithm.

ALGORITHM 1-2 Print Deviation from Mean for Series

```
Algorithm deviation
   Pre    nothing
   Post   average and numbers with their deviation printed
 1 loop (not end of file)
   1   read number into array
   2   add number to total
   3   increment count
 2 end loop
 3 set average to total / count
 4 print average
 5 loop (not end of array)
   1   set devFromAve to array element - average
   2   print array element and devFromAve
 6 end loop
end deviation
```

Algorithm 1-2 Analysis There are two points worth mentioning in Algorithm 1-2. First, there are no parameters. Second, as previously explained, we do not declare variables. A variable's type and purpose should be easily determined by its name and usage.

1.2 The Abstract Data Type

In the history of programming concepts, we started with nonstructured, linear programs, known as **spaghetti code**, in which the logic flow wound through the program like spaghetti on a plate. Next came the concept of **modular programming**, in which programs were organized in functions, each of which still used a linear coding technique. In the 1970s, the basic principles of **structured programming** were formulated by computer scientists such as Edsger Dijkstra and Niklaus Wirth. They are still valid today.

Atomic and Composite Data

Atomic data are data that consist of a single piece of information; that is, they cannot be divided into other meaningful pieces of data. For example, the integer 4562 may be considered a single integer value. Of course, we can decompose it into digits, but the decomposed digits do not have the same characteristics of the original integer; they are four single-digit integers ranging from 0 to 9. In some languages atomic data are known as scalar data because of their numeric properties.

The opposite of atomic data is **composite data**. Composite data can be broken out into subfields that have meaning. As an example of a composite data item, consider your telephone number. A telephone number actually has three different parts. First, there is the area code. Then, what you consider to be your phone number is actually two different data items, a prefix consisting of a three-digit exchange and the number within the exchange,

consisting of four digits. In the past, these prefixes were names such as DAvenport and CYpress.

Data Type

A **data type** consists of two parts: a set of data and the operations that can be performed on the data. Thus we see that the integer type consists of values (whole numbers in some defined range) and operations (add, subtract, multiply, divide, and any other operations appropriate for the data).

Data Type
1. A set of values 2. A set of operations on values

Table 1-1 shows three data types found in all systems.

Type	Values	Operations
integer	-∞, … , -2, -1, 0, 1, 2,…,∞	*, +, -, %, /, ++, --, …
floating point	-∞, … , 0.0, … , ∞	*, +, -, /, …
character	\0, …, 'A', 'B', … , 'a', 'b', … , ~	<, >, …

TABLE 1-1 Three Data Types

Data Structure

A **data structure** is an aggregation of atomic and composite data into a set with defined relationships. In this definition *structure* means a set of rules that holds the data together. In other words, if we take a combination of data and fit them into a structure such that we can define its relating rules, we have made a data structure. Data structures can be nested. We can have a data structure that consists of other data structures. For example, we can define the two structures array and record as shown in Table 1-2.

Array	Record
Homogeneous sequence of data or data types known as elements	Heterogeneous combination of data into a single structure with an identified key
Position association among the elements	No association

TABLE 1-2 Data Structure Examples

Most of the programming languages support several data structures. In addition, modern programming languages allow programmers to create new data structures for an application.

Data Structure

1. A combination of elements in which each is either a data type or another data structure
2. A set of associations or relationships (structure) involving the combined elements

Abstract Data Type

Generally speaking, programmers' capabilities are determined by the tools in their tool kits. These tools are acquired by education and experience. A knowledge of data structures is one of those tools.

When we first started programming, there were no abstract data types. If we wanted to read a file, we wrote the code to read the physical file device. It did not take long to realize that we were writing the same code over and over again. So we created what is known today as an **abstract data type (ADT)**. We wrote the code to read a file and placed it in a library for all programmers to use.

This concept is found in modern languages today. The code to read the keyboard is an ADT. It has a data structure, a character, and a set of operations that can be used to read that data structure. Using the ADT we can not only read characters but we can also convert them into different data structures such as integers and strings.

With an ADT users are not concerned with *how* the task is done but rather with *what* it can do. In other words, the ADT consists of a set of definitions that allow programmers to use the functions while hiding the implementation. This generalization of operations with unspecified implementations is known as abstraction. We abstract the essence of the process and leave the implementation details hidden.

The concept of abstraction means:

1. We know *what* a data type can do.
2. *How* it is done is hidden.

Consider the concept of a list. At least four data structures can support a list. We can use a matrix, a linear list, a tree, or a graph. If we place our list in an ADT, users should not be aware of the structure we use. As long as they can insert and retrieve data, it should make no difference how we store the data. Figure 1-1 shows four logical structures that might be used to hold a list.

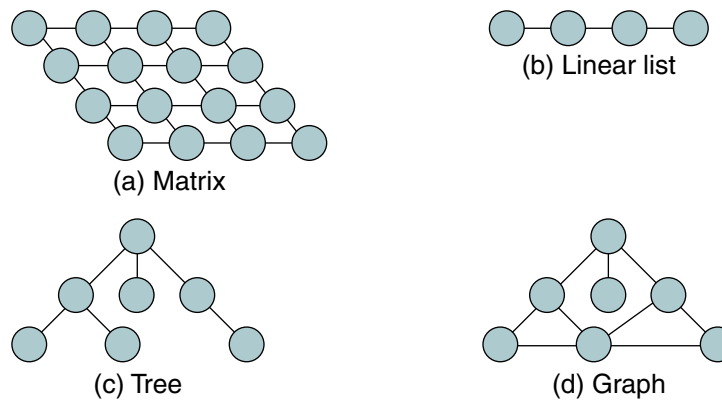


FIGURE 1-1 Some Data Structures

As another example, consider the system analyst who needs to simulate the waiting line of a bank to determine how many tellers are needed to serve customers efficiently. This analysis requires the simulation of a queue. However, queues are not generally available in programming languages. Even if a queue type were available, our analyst would still need some basic queue operations, such as enqueuing (insertion) and dequeuing (deleting), for the simulation.

There are two potential solutions to this problem: (1) we can write a program that simulates the queue our analyst needs (in this case, our solution is good only for the one application at hand) or (2) we can write a queue ADT that can be used to solve any queue problem. If we choose the latter course, our analyst still needs to write a program to simulate the banking application, but doing so is much easier and faster because he or she can concentrate on the application rather than the queue.

We are now ready to define ADT. An abstract data type is a data declaration packaged together with the operations that are meaningful for the data type. In other words, we **encapsulate** the data and the operations on the data, and then we hide them from the user.

Abstract Data Type
1. Declaration of data
2. Declaration of operations
3. Encapsulation of data and operations

We cannot overemphasize the importance of hiding the implementation. The user should not have to know the data structure to use the ADT. Referring to our queue example, the application program should have no knowledge of the data structure. All references to and manipulation of the data in the queue must be handled through defined interfaces to the

structure. Allowing the application program to directly reference the data structure is a common fault in many implementations. This keeps the ADT from being fully portable to other applications.

1.3 Model for an Abstract Data Type

The ADT model is shown in Figure 1-2. The colored area with an irregular outline represents the ADT. Inside the ADT are two different aspects of the model: data structures and functions (public and private). Both are entirely contained in the model and are not within the application program scope. However, the data structures are available to all of the ADT's functions as needed, and a function may call on other functions to accomplish its task. In other words, the data structures and the functions are within scope of each other.

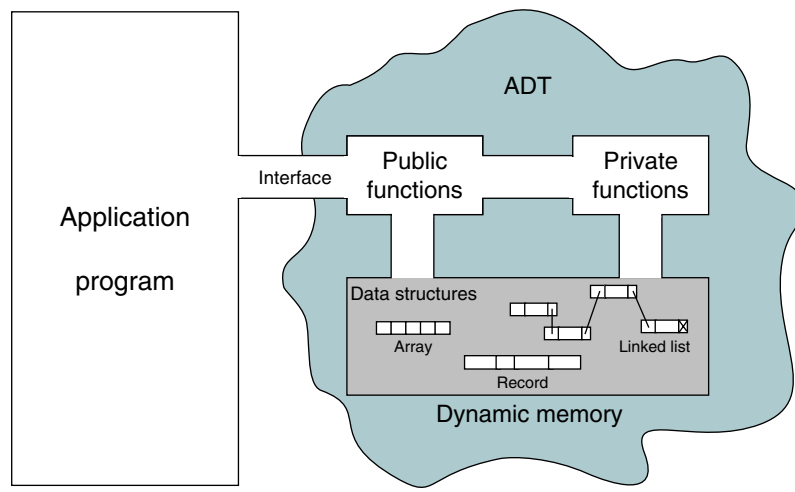


FIGURE 1-2 Abstract Data Type Model

ADT Operations

Data are entered, accessed, modified, and deleted through the external interface drawn as a passageway partially in and partially out of the ADT. Only the public functions are accessible through this interface. For each ADT operation there is an algorithm that performs its specific task. Only the operation name and its parameters are available to the application, and they provide the only interface to the ADT.

ADT Data Structure

When a list is controlled entirely by the program, it is often implemented using simple structures similar to those used in your programming class. Because the abstract data type must hide the implementation from the user, however, all data about the structure must be maintained inside the ADT. Just encapsulating the structure in an ADT is not sufficient. It is also necessary for multiple versions of the structure to be able to coexist. Consequently, we must hide the implementation from the user while being able to store different data.

In this text, we develop ADTs for stacks, queues, lists, binary search trees, AVL trees, B-trees, heaps, and graphs. If you would like a preview, look at the stack ADT in Chapter 3.

1.4 ADT Implementations

There are two basic structures we can use to implement an ADT list: arrays and linked lists.

Array Implementations

In an array, the sequentiality of a list is maintained by the order structure of elements in the array (indexes). Although searching an array for an individual element can be very efficient, addition and deletion of elements are complex and inefficient processes. For this reason arrays are seldom used, especially when the list changes frequently. In addition, array implementations of non-linear lists can become excessively large, especially when there are several successors for each element. Appendix F provides array implementations for two ADTs.

Linked List Implementations

A **linked list** is an ordered collection of data in which each element contains the location of the next element or elements. In a linked list, each element contains two parts: **data** and one or more **links**. The data part holds the application data—the data to be processed. Links are used to chain the data together. They contain pointers that identify the next element or elements in the list.

We can use a linked list to create linear and non-linear structures. In linear linked lists, each element has only zero or one successor. In non-linear linked lists, each element can have zero, one, or more successors.

The major advantage of the linked list over the array is that data are easily inserted and deleted. It is not necessary to shift elements of a linked list to make room for a new element or to delete an element. On the other hand, because the elements are no longer physically sequenced, we are limited to sequential searches:[1] we cannot use a binary search.[2]

1. Sequential and binary searches are discussed in Chapter 13.
2. When we examine trees, you will see several data structures that allow for easy updates and efficient searches.

Figure 1-3(a) shows a linked list implementation of a linear list. The link in each element, except the last, points to its unique successor; the link in the last element contains a null pointer, indicating the end of the list. Figure 1-3(b) shows a linked list implementation of a non-linear list. An element in a non-linear list can have two or more links. Here each element contains two links, each to one successor. Figure 1-3(c) contains an example of an **empty list**, linear or non-linear. We define an empty list as a null list pointer.

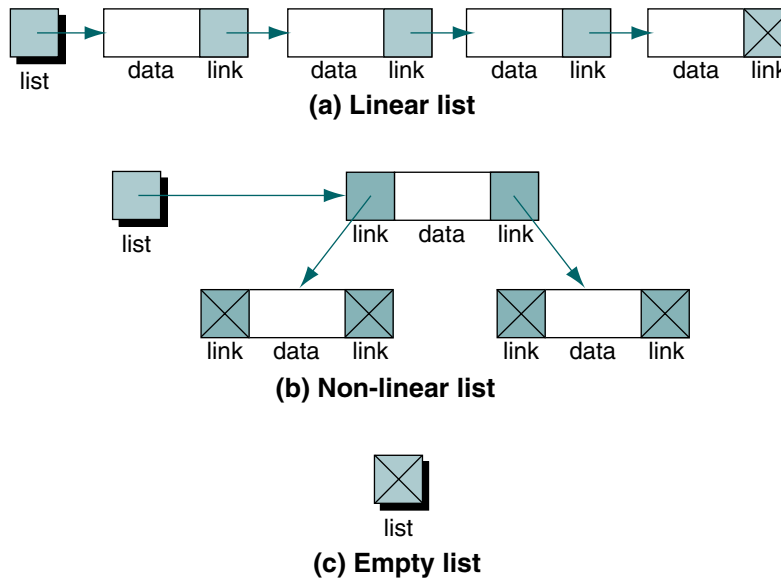


FIGURE 1-3 Linked Lists

In this section, we discuss only the basic concepts for linked lists. We expand on these concepts in future chapters.

Nodes

In linked list implementation, the elements in a list are called nodes. A **node** is a structure that has two parts: the data and one or more links. Figure 1-4 shows two different nodes: one for a linear list and the other for a non-linear list.

The nodes in a linked list are called **self-referential** structures. In a self-referential structure, each instance of the structure contains one or more pointers to other instances of the same structural type. In Figure 1-4, the colored boxes with arrows are the pointers that make the linked list a self-referential structure.

The data part in a node can be a single field, multiple fields, or a structure that contains several fields, but it always acts as a single field. Figure 1-5 shows three designs for a node of a linear list. The upper-left node contains a

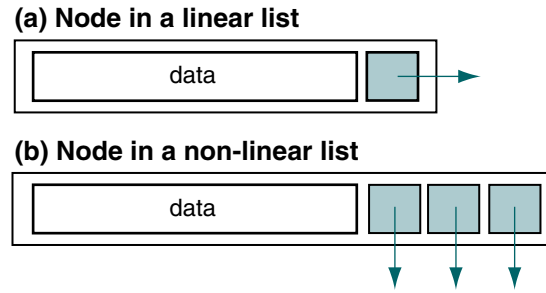


FIGURE 1-4 Nodes

single field, a number, and a link. The upper-right node is more typical. It contains three data fields: a name, an id, and grade points (`grdPts`)—and a link. The third example is the one we recommend. The fields are defined in their own structure, which is then put into the definition of a node structure.

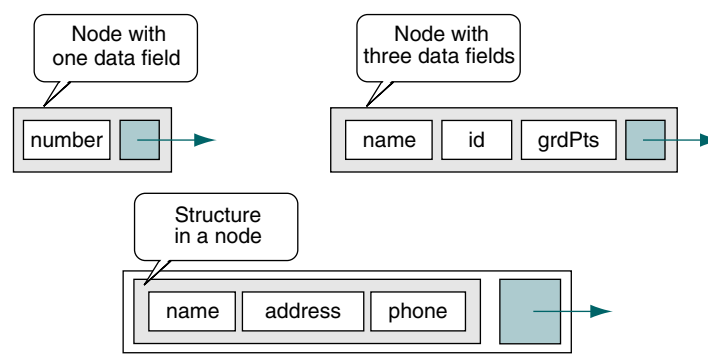


FIGURE 1-5 Linked List Node Structures

Pointers to Linked Lists

A linked list must always have a head pointer. Depending on how we use the list, we may have several other pointers as well. For example, if we are going to search a linked list, we will need an additional pointer to the location where we found the data we were looking for. Furthermore, in many structures, programming is more efficient if there is a pointer to the last node in the list as well as a head pointer.

1.5 Generic Code for ADTs

In data structures we need to create generic code for abstract data types. **Generic code** allows us to write one set of code and apply it to any data type. For

PROGRAM 1-7 Compare Two Floating-Point Values *(continued)*

```
30              returns -1 if ptr1 <  ptr2
31  */
32  int compare (void* ptr1, void* ptr2)
33  {
34    if (*(float*)ptr1 >=  *(float*)ptr2)
35       return 1;
36    else
37       return -1;
38  }  // compare
```

```
Results:
Larger value is: 81.7
```

1.6 Algorithm Efficiency

There is seldom a single algorithm for any problem. When comparing two different algorithms that solve the same problem, you often find that one algorithm is an order of magnitude more efficient than the other. In this case, it only makes sense that you be able to recognize and choose the more efficient algorithm.

Although computer scientists have studied algorithms and algorithm efficiency extensively, the field has not been given an official name. Brassard and Bratley coined the term **algorithmics**, which they define as "the systematic study of the fundamental techniques used to design and analyze efficient algorithms."[3] We use the term in this book.

If a function is linear—that is, if it contains no loops or recursions—its efficiency is a function of the number of instructions it contains. In this case, its efficiency depends on the speed of the computer and is generally not a factor in the overall efficiency of a program. On the other hand, functions that use loops or recursion vary widely in efficiency. The study of algorithm efficiency therefore focuses on loops. Our analysis concentrates on loops because recursion can always be converted to a loop.

As we study specific examples, we generally discuss the algorithm's efficiency as a function of the number of elements to be processed. The general format is

$$f(n) = \text{efficiency}$$

The basic concepts are discussed in this section.

3. Gilles Brassard and Paul Bratley, *Algorithmics Theory and Practice* (Englewood Cliffs, N.J.: Prentice Hall, 1988), xiii.

Linear Loops

Let us start with a simple loop. We want to know how many times the body of the loop is repeated in the following code:[4]

```
for (i = 0; i < 1000; i++)
     application code
```

Assuming `i` is an integer, the answer is 1000 times. The number of iterations is directly proportional to the loop factor, 1000. The higher the factor, the higher the number of loops. Because the efficiency is directly proportional to the number of iterations, it is

$$f(n) = n$$

However, the answer is not always as straightforward as it is in the above example. For instance, consider the following loop. How many times is the body repeated in this loop? Here the answer is 500 times. Why?

```
for (i = 0; i < 1000; i += 2)
     application code
```

In this example the number of iterations is half the loop factor. Once again, however, the higher the factor, the higher the number of loops. The efficiency of this loop is proportional to half the factor, which makes it

$$f(n) = n / 2$$

If you were to plot either of these loop examples, you would get a straight line. For that reason they are known as **linear loops**.

Logarithmic Loops

In a linear loop, the loop update either adds or subtracts. In a **logarithmic loop**, the controlling variable is multiplied or divided in each iteration. How many times is the body of the loops repeated in the following program segments?

Multiply Loops

```
for (i = 0; i < 1000; i *= 2)
     application code
```

Divide Loops

```
for (i = 0; i < 1000; i /= 2)
     application code
```

To help you understand this problem, Table 1-3 analyzes the values of *i* for each iteration. As you can see, the number of iterations is 10 in both cases. The reason is that in each iteration the value of `i` doubles for the multiply loop and is cut in half for the divide loop. Thus, the number of iterations

4. For algorithm efficiency analysis, we use C code so that we can clearly see the looping constructs.

Multiply		Divide	
Iteration	**Value of `i`**	**Iteration**	**Value of `i`**
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

TABLE 1-3 Analysis of Multiply and Divide Loops

is a function of the multiplier or divisor, in this case 2. That is, the loop continues while the condition shown below is true.

```
multiply   2^Iterations < 1000
divide     1000 / 2^Iterations >= 1
```

Generalizing the analysis, we can say that the iterations in loops that multiply or divide are determined by the following formula:

$$f(n) = \log n$$

Nested Loops

Loops that contain loops are known as **nested loops**. When we analyze nested loops, we must determine how many iterations each loop completes. The total is then the product of the number of iterations in the inner loop and the number of iterations in the outer loop.

```
Iterations = outer loop iterations x inner loop iterations
```

We now look at three nested loops: linear logarithmic, quadratic, and dependent quadratic.

Linear Logarithmic

The inner loop in the following code is a loop that multiplies. To see the multiply loop, look at the update expression in the inner loop.

```
for (i = 0; i < 10; i++)
     for (j = 0; j < 10; j *= 2)
          application code
```

The number of iterations in the inner loop is therefore $\lceil \log 10 \rceil$. However, because the inner loop is controlled by an outer loop, the above formula must be multiplied by the number of times the outer loop executes, which is 10. This gives us

$$10 \log 10$$

which is generalized as

$$f(n) = n \log n$$

Quadratic

In a **quadratic loop**, the number of times the inner loop executes is the same as the outer loop. Consider the following example.

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
         application code
```

The outer loop (*for i*) is executed 10 times. For each of its iterations, the inner loop (*for j*) is also executed 10 times. The answer, therefore, is 100, which is 10×10, the square of the loops. This formula generalizes to

$$f(n) = n^2$$

Dependent Quadratic

In a **dependent quadratic loop**, the number of iterations of the inner loop depends on the outer loop. Consider the nested loop shown in the following example.

```
for (i = 0; i < 10; i++)
    for (j = 0; j < i; j++)
         application code
```

The outer loop is the same as the previous loop. However, the inner loop depends on the outer loop for one of its factors. It is executed only once the first iteration, twice the second iteration, three times the third iteration, and so forth. The number of iterations in the body of the inner loop is calculated as shown below.

```
1 + 2 + 3 + … + 9 + 10 = 55
```

If we compute the average of this loop, it is 5.5 (55/10), which is the same as the number of iterations (10) plus 1 divided by 2. Mathematically, this calculation is generalized to

$$\frac{(n+1)}{2}$$

Multiplying the inner loop by the number of times the outer loop is executed gives us the following formula for a dependent quadratic loop:

$$f(n) = n\left(\frac{n+1}{2}\right)$$

Big-O Notation

With the speed of computers today, we are not concerned with an exact measurement of an algorithm's efficiency as much as we are with its general order of magnitude. If the analysis of two algorithms shows that one executes 15 iterations while the other executes 25 iterations, they are both so fast that we can't see the difference. On the other hand, if one iterates 15 times and the other 1500 times, we should be concerned.

We have shown that the number of statements executed in the function for n elements of data is a function of the number of elements, expressed as $f(n)$. Although the equation derived for a function may be complex, a dominant factor in the equation usually determines the order of magnitude of the result. Therefore, we don't need to determine the complete measure of efficiency, only the factor that determines the magnitude. This factor is the big-O, as in "on the order of," and is expressed as O(n)—that is, on the order of n.

This simplification of efficiency is known as big-O analysis. For example, if an algorithm is quadratic, we would say its efficiency is

$$O(n^2)$$

or on the order of n squared.

The **big-O notation** can be derived from $f(n)$ using the following steps:

1. In each term, set the coefficient of the term to 1.

2. Keep the largest term in the function and discard the others. Terms are ranked from lowest to highest as shown below.

$$\log n \quad n \quad n\log n \quad n^2 \quad n^3 \ldots n^k \quad 2^n \quad n!$$

For example, to calculate the big-O notation for

$$f(n) = n\frac{(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

we first remove all coefficients. This gives us

$$n^2 + n$$

which after removing the smaller factors gives us

$$n^2$$

which in big-O notation is stated as

$$O(f(n)) = O(n^2)$$

To consider another example, let's look at the polynomial expression

$$f(n) = a_j n^k + a_{j-1} n^{k-1} + \ldots + a_2 n^2 + a_1 n + a_0$$

We first eliminate all of the coefficients as shown below.

$$f(n) = n^k + n^{k-1} + \ldots + n^2 + n + 1$$

The largest term in this expression is the first one, so we can say that the order of a polynomial expression is

$$O(f(n)) = O(n^k)$$

Standard Measures of Efficiency

Computer scientists have defined seven categories of algorithm efficiency. We list them in Table 1-4 in order of decreasing efficiency and show the first five of them graphically in Figure 1-14.

Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10{,}000^2$	minutes
Polynomial	$O(n^k)$	$10{,}000^k$	hours
Exponential	$O(c^n)$	$2^{10{,}000}$	intractable
Factorial	$O(n!)$	10,000!	intractable

TABLE 1-4 Measures of Efficiency for n = 10,000

Any measure of efficiency presumes that a sufficiently large sample is being considered. If you are dealing with only 10 elements and the time required is a fraction of a second, there is no meaningful difference between two algorithms. On the other hand, as the number of elements being processed grows, the difference between algorithms can be staggering.

Returning for a moment to the question of why we should be concerned about efficiency, consider the situation in which you can solve a problem in three ways: one is linear, another is linear logarithmic, and a third is quadratic. The magnitude of their efficiency for a problem containing 10,000

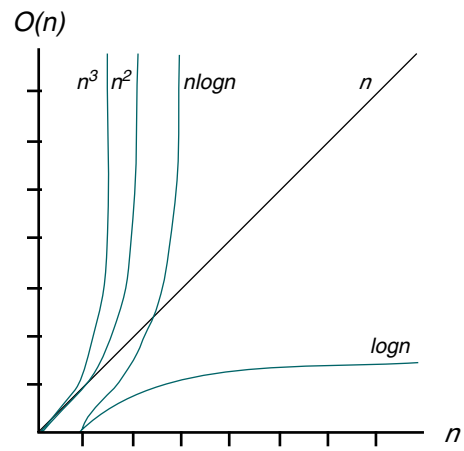


FIGURE 1-14 Plot of Effeciency Measures

elements shows that the linear solution requires a fraction of a second whereas the quadratic solution requires minutes (see Table 1-4).

Looking at the problem from the other end, if we are using a computer that executes a million instructions per second and the loop contains 10 instructions, we spend 0.00001 second for each iteration of the loop. Table 1-4 also contains an estimate of the time needed to solve the problem given different efficiencies.

Big-O Analysis Examples

To demonstrate the concepts we have been discussing, we examine two more algorithms: add and multiply two matrices.

Add Square Matrices

To add two square matrices, we add their corresponding elements; that is, we add the first element of the first matrix to the first element of the second matrix, the second element of the first matrix to the second element of the second matrix, and so forth. Of course, the two matrices must be the same size. This concept is shown in Figure 1-15.

4	2	1	+	6	1	7	=	10	3	8
0	–3	4		3	2	–1		3	–1	3
5	6	2		4	6	2		9	12	4

FIGURE 1-15 Add Matrices

The pseudocode to add two matrices is shown in Algorithm 1-3.

ALGORITHM 1-3 Add Two Matrices

```
Algorithm addMatrix (matrix1, matrix2, size, matrix3)
Add matrix1 to matrix2 and place results in matrix3
   Pre  matrix1 and matrix2 have data
        size is number of columns or rows in matrix
   Post matrices added--result in matrix3
1  loop (not end of row)
   1   loop (not end of column)
       1   add matrix1 and matrix2 cells
       2   store sum in matrix3
   2   end loop
2  end loop
end addMatrix
```

Algorithm 1-3 Analysis

In this algorithm, we see that for each element in a row, we add all of the elements in a column. This is the classic quadratic loop. The efficiency of the algorithm is therefore $O(\text{size}^2)$ or $O(n^2)$.

Multiply Square Matrices

When two square matrices are multiplied, we must *multiply* each element in a row of the first matrix by its corresponding element in a column of the second matrix. The value in the resulting matrix is then the *sum* of the products. For example, given the matrix in our addition example above, the first element in the resulting matrix—that is, the element at `[0, 0]`—is the sum of the products obtained by multiplying each element in the first *row* (row 0) by its corresponding element in the first *column* (column 0). The value of the element at index location `[0, 1]` is the sum of the products of each element in the first row (again row 0) multiplied by its corresponding element in the second column (column 1). The value of the element at index location `[1, 2]` is the sum of the products of each element in the second *row* multiplied by the corresponding elements in the third *column*. Once again the square matrices must be the same size. Figure 1-16 graphically shows how two matrices are multiplied.

Generalizing this concept, we see that

```
matrix3 [row, col] =
            matrix1[row, 0] x matrix2[0, col]
      +     matrix1[row, 1] x matrix2[1, col]
      +     matrix1[row, 2] x matrix2[2, col]
      ...
      +     matrix1[row, s-1] x matrix2[s-1, col]
where s = size of matrix
```

The pseudocode used for multiplying matrices is provided in Algorithm 1-4.

ALGORITHM 1-4 Multiply Two Matrices

```
Algorithm multiMatrix (matrix1, matrix2, size, matrix3)
Multiply matrix1 by matrix2 and place product in matrix3
   Pre  matrix1 and matrix2 have data
        size is number of columns and rows in matrix
   Post matrices multiplied--result in matrix3
1  loop (not end of row)
   1   loop (not end of column)
       1   loop (size of row times)
           1   calculate sum of
                   (all row cells) * (all column cells)
           2   store sum in matrix3
```

continued

ALGORITHM 1-4 Multiply Two Matrices *(continued)*

```
   2   end loop
 2  end loop
 3  return
end multiMatrix
```

Algorithm 1-4 Analysis

In this algorithm we see three nested loops. Because each loop starts at the first element, we have a cubic loop. Loops with three nested loops have a big-O efficiency of $O(size^3)$ or $O(n^3)$.

It is also possible to multiply two matrices if the number of rows in the first matrix is the same as the number of columns in the second. We leave the solution to this problem as an exercise (Exercise 21).

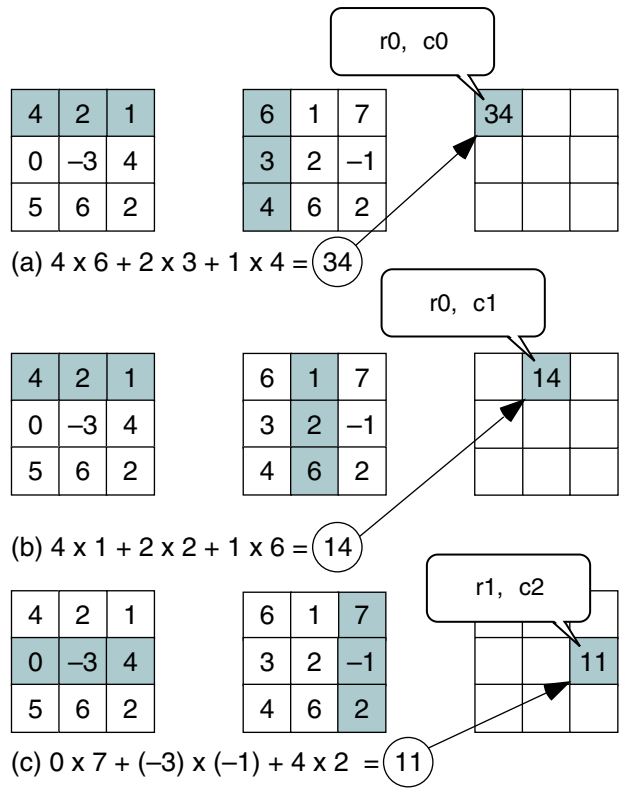


FIGURE 1-16 Multiply Matrices

1.7 Key Terms

abstract data type (ADT)
algorithmics
atomic data
big-O notation
composite data
construct
data
data structure
data type
dependent quadratic loop
empty list
encapsulation
generic code
intelligent data names
linear loop
link
linked list
logarithmic loop
loop
modular programming
nested loop
node
pointer to *void*
pointer to function
pseudocode
quadratic loop
return condition
self-referential
selection statement
sequence
spaghetti code
structured programming

1.8 Summary

- One of the most common tools used to define algorithms is pseudocode.
- Pseudocode is an English-like representation of the code required for an algorithm. It is part English, part structured code.
- Atomic data are data that are single, nondecomposable entities.
- Atomic data types are defined by a set of values and a set of operations that act on the values.
- A data structure is an aggregation of atomic and composite data with a defined relationship.
- An abstract data type (ADT) is a data declaration packaged together with the operations that are meaningful for the data type.
- There are two basic structures used to implement an ADT list: arrays and linked lists.
- In an array, the sequentiality of a list is preserved by the ordered structure of elements. Although searching an array is very efficient, adding and deleting is not.
- Although adding and deleting in a linked list is efficient, searching is not because we must use a sequential search.
- In a linked list, each element contains the location of the next element or elements.

Chapter 2
Recursion

In general, there are two approaches to writing repetitive algorithms. One uses iteration; the other uses recursion. Recursion is a repetitive process in which an algorithm calls itself. Note, however, that some older languages do not support recursion.

In this chapter we study recursion. We begin by studying a classic recursive case—factorial. Once we explain how recursion works, we develop some principles for developing recursive algorithms and then use them to develop another recursive case study, Fibonacci numbers. We conclude the theory of recursion with a discussion of a classic recursive algorithm, the Towers of Hanoi. In the final section, we develop C implementations for Fibonacci numbers, prefix to postfix conversion, and the Towers of Hanoi.

2.1 Factorial—A Case Study

To begin with a simple example, let's consider the calculation of **factorial**. The factorial of a positive number is the product of the integral values from 1 to the number. This definition is shown in Figure 2-1.

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1) \times (n-2) \times \ldots \times 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

FIGURE 2-1 Iterative Factorial Algorithm Definition

Note that this definition is iterative. A repetitive algorithm is defined iteratively whenever the definition involves only the algorithm parameter(s) and not the algorithm itself. We can calculate the value of factorial(4) using Figure 2-1, as follows:

```
factorial(4) = 4 × 3 × 2 × 1 = 24
```

Recursion Defined

A repetitive algorithm uses **recursion** whenever the algorithm appears within the definition itself. For example, the factorial algorithm can be defined recursively as shown in Figure 2-2.

$$\text{Factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (\text{Factorial}(n-1)) & \text{if } n > 0 \end{cases}$$

FIGURE 2-2 Recursive Factorial Algorithm Definition

The decomposition of factorial(3) using Figure 2-2 is shown in Figure 2-3. If you study Figure 2-3 carefully, you will note that the recursive solution for a problem involves a two-way journey: first we decompose the problem from the top to the bottom, then we solve it from the bottom to the top.

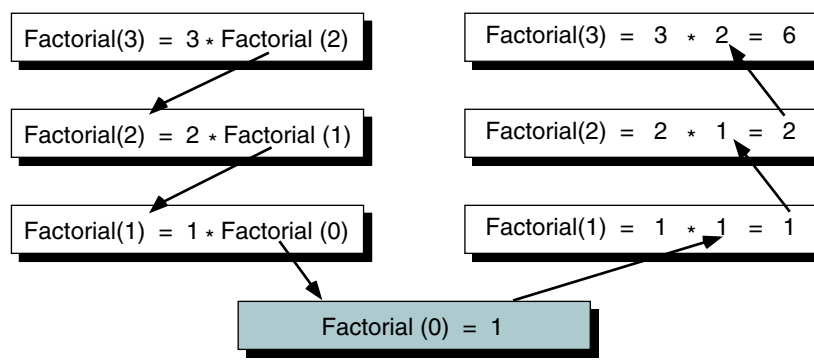


FIGURE 2-3 Factorial (3) Recursively

Judging by this example, the recursive calculation appears to be much longer and more difficult. So why would we want to use the recursive method? Although the recursive calculation looks more difficult when using

paper and pencil, it is often a much easier and more elegant solution when we use computers. Also, it offers a conceptual simplicity to the creator and the reader.

Recursion is a repetitive process in which an algorithm calls itself.

Iterative Solution

Let's write an algorithm to solve the factorial problem iteratively. This solution usually involves using a loop such as the one shown in Algorithm 2-1.

ALGORITHM 2-1 Iterative Factorial Algorithm

```
Algorithm iterativeFactorial (n)
Calculates the factorial of a number using a loop.
   Pre  n  is the number to be raised factorially
   Post n! is returned
 1 set i to 1
 2 set factN to 1
 3 loop (i <= n)
   1  set factN to factN * i
   2  increment i
 4 end loop
 5 return factN
end iterativeFactorial
```

Recursive Solution

Now let's write the same algorithm recursively. The recursive solution does not need a loop; recursion is itself repetition. In the recursive version, we let the factorial algorithm call itself, each time with a different set of parameters. The algorithm for recursive factorial is shown in Algorithm 2-2.

ALGORITHM 2-2 Recursive Factorial

```
Algorithm recursiveFactorial (n)
Calculates factorial of a number using recursion.
   Pre    n  is the number being raised factorially
   Post   n! is returned
 1 if (n equals 0)
   1  return 1
 2 else
   1  return (n * recursiveFactorial (n - 1))
 3 end if
end recursiveFactorial
```

Algorithm 2-2 Analysis

If you compare the iterative and recursive versions of factorial, you should be immediately struck by how much simpler the code is in the recursive version. First, there is no loop. The recursive version consists of a simple selection statement that returns either the value 1 or the product of two values, one of which is a call to factorial itself.

Figure 2-4 traces the recursion and the parameters for each individual call.

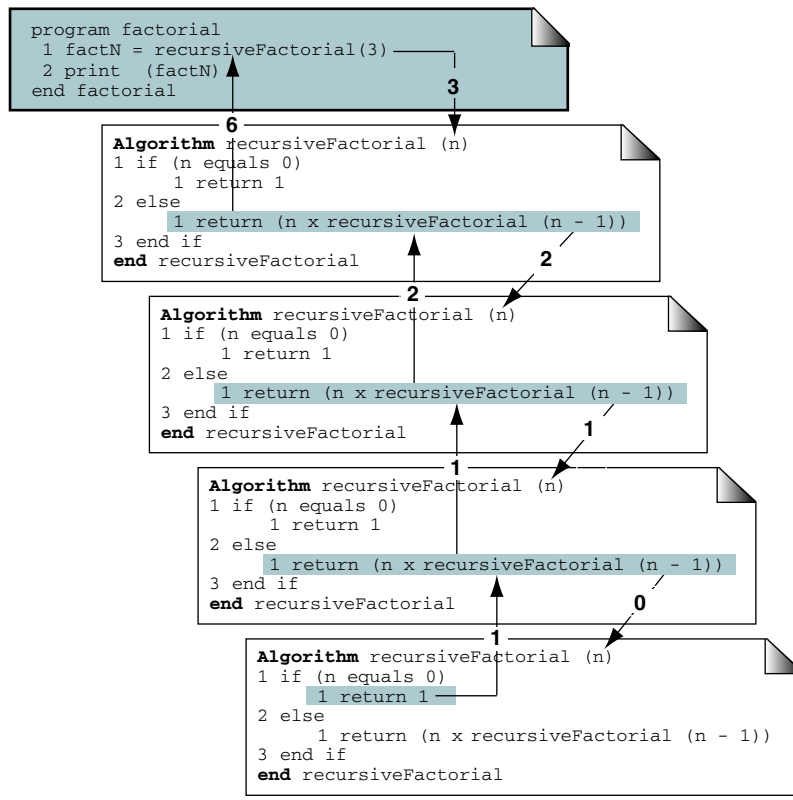


FIGURE 2-4 Calling a Recursive Algorithm

2.2 Designing Recursive Algorithms

Now that we have seen how recursion works, let's turn our attention to the steps for designing a recursive algorithm. We first look at the basic design methodology, then we discuss the limitations of recursion, and finally we design and implement another recursive algorithm.

The Design Methodology

If we were to examine all hypothetically possible recursive algorithms, we would see that they all have two elements: each call either *solves* one part of

the problem or it reduces *the size* of the problem. In Algorithm 2-2 statement 1.1 solves a small piece of the problem—factorial(0) is 1. Statement 2.1, on the other hand, reduces the size of the problem by recursively calling factorial with *n* - 1. Once the solution to factorial (n - 1) is known, statement 2.1 provides a part of the solution to the general problem by returning a value to the calling algorithm.

As we see in statement 2.1, the general part of the solution is the recursive call: statement 2.1 calls itself to solve the problem. We also see this in Figure 2-4. At each recursive call, the size of the problem is reduced, from the factorial of 3, to 2, to 1, and finally to factorial 0.

The statement that "solves" the problem is known as the **base case**. *Every recursive algorithm must have a base case.* The rest of the algorithm is known as the **general case**. In our factorial example, the base case is factorial (0); the general case is n × factorial (n - 1). The general case contains the logic needed to reduce the size of the problem.

Every recursive call must either solve a part of the problem or reduce the size of the problem.

In the factorial problem, once the base case has been reached, the solution begins. We now know one part of the answer and can return that part to the next, more general statement. Thus, in Algorithm 2-2, we know that factorial(0) is 1, and we return that value. This allows us to solve the next general case

```
factorial(1) ⇨ 1 × factorial(0) ⇨ 1 × 1 ⇨ 1
```

We can now return the value of `factorial(1)` to the more general case, factorial(2), which we know to be

```
factorial(2) ⇨ 2 × factorial(1) ⇨ 2 × 1 ⇨ 2
```

As we solve each general case in turn, we are able to solve the next-higher general case until we finally solve the most general case, the original problem.

Returning to the purpose of this section, we are now ready to state the rules for designing a recursive algorithm:

1. First, determine the base case.
2. Then determine the general case.
3. Combine the base case and the general cases into an algorithm.

In combining the base and the general case into an algorithm, we must pay careful attention to the logic. Each call must reduce the size of the problem and move it toward the base case. The base case, when reached, must terminate without a call to the recursive algorithm; that is, it must execute a return.

Limitations of Recursion

We have introduced only a brief explanation of recursion in this section. Recursion works best when the algorithm uses a data structure that naturally supports recursion. For example, in Chapter 6 we will study trees. Trees are a naturally recursive structure and recursion works well with them.

In other cases the algorithm is naturally suited to recursion. For example, the binary search algorithm (see Chapter 13) lends itself to a natural recursive algorithm, as does the Towers of Hanoi algorithm, which we discuss later in this chapter. On the other hand, not all looping algorithms can or should be implemented with recursion, as we discuss below.

Recursive solutions may involve extensive overhead (both time and memory) because they use calls. Each call takes time to execute. A recursive algorithm therefore generally runs more slowly than its nonrecursive implementation.[1]

You should not use recursion if the answer to any of the following questions is no:

1. Is the algorithm or data structure naturally suited to recursion?
2. Is the recursive solution shorter and more understandable?
3. Does the recursive solution run within acceptable time and space limits?

For reasons we will explain in Chapter 3, each time we make a call we use up some of our memory allocation. If the recursion is deep—that is, if there are many recursive calls—we may run out of memory.

Because of the time and memory overhead, algorithms such as factorial are better developed iteratively if large numbers are involved. As a general rule, recursive algorithms should be used only when their efficiency is logarithmic.

Design Implementation—Reverse Keyboard Input

Having studied the design methodology for recursive algorithms and their limitations, we are now ready to put the concepts into practice. Assume that we are reading data from the keyboard and need to print the data in reverse. The easiest way to print the list in reverse is to write a recursive algorithm.

It should be obvious that to print the list in reverse, we must first read all of the data. The base case, therefore, is that we have read the last piece of data. Similarly, the general case is to read the next piece of data. The question is, when do we print? If we print before we read all of the data, we print the list in sequence. If we print the list after we read the last piece of data—that is, if we print it as we back out of the recursion—we print it in reverse sequence. The code is shown in Algorithm 2-3.

1. Most of today's compilers optimize code when possible. When the recursion occurs at the end of a function (known as tail recursion), an optimized compiler turns the recursive code into a simple loop, thus eliminating the function call inefficiency.

ALGORITHM 2-3 Print Reverse

```
Algorithm printReverse (data)
Print keyboard data in reverse.
   Pre  nothing
   Post data printed in reverse
 1 if (end of input)
   1   return
 2 end if
 3 read data
 4 printReverse (data)
Have reached end of input: print nodes
 5 print data
 6 return
end printReverse
```

Algorithm 2-3 Analysis

As you study Algorithm 2-3, remember that statement 5 cannot be executed until we reach the end of the input. It is not executed immediately after statement 4 because statement 4 is a recursive call. We get to statement 5 only after we return from statement 1.1 or after we return from the end statement at the end of the algorithm. Figure 2-5 traces the execution of Algorithm 2-3.

This algorithm also demonstrates the use of local variables in recursion. Although there is only one variable, **data**, in the pseudocode, when we implement the algorithm in a recursive language, the computer creates a separate set of local variables for each call. These variables are kept in memory until we return from the call that created them, at which time they are recycled.

Now that we've designed the algorithm, we need to analyze it to determine whether it is really a good solution; that is, is the recursive algorithm a good candidate for recursion? To analyze this algorithm, we turn to the three questions we developed in the preceeding "Limitations of Recursion" section.

1. Is the algorithm or data structure naturally suited to recursion? A list, such as data read from the keyboard, is not a naturally recursive structure. Furthermore, the algorithm is not naturally suited to recursion because it is not a logarithmic algorithm.
2. Is the recursive solution shorter and more understandable? The answer to this question is yes.
3. Does the recursive solution run within acceptable time and space limits? The number of iterations in the traversal of a list can become quite large because the algorithm has a linear efficiency—that is, it is $O(n)$.

We thus see that the answer to two of the three questions is no. Therefore, although we can successfully write the algorithm recursively, we should not. It is not a good candidate for recursion and is better implemented as an iterative loop.

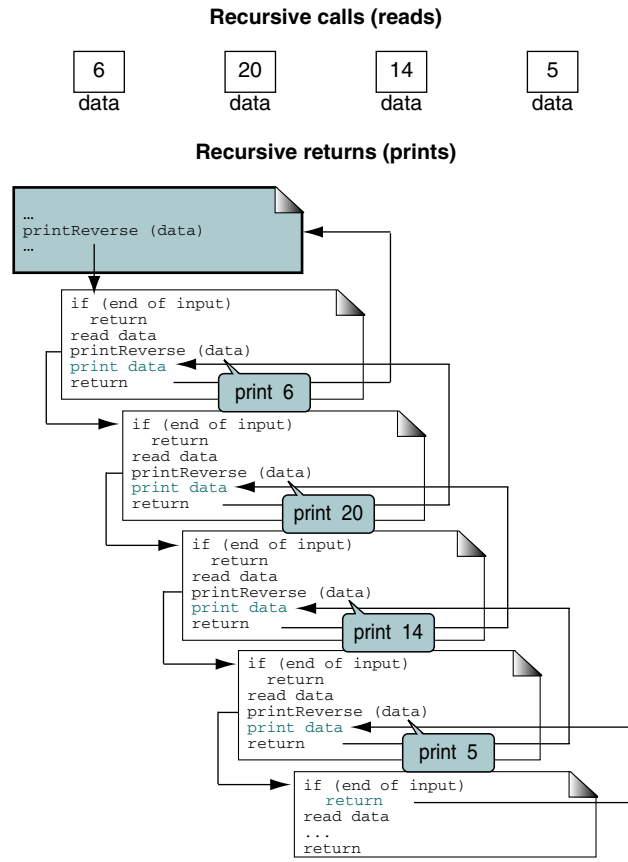


FIGURE 2-5 Print Keyboard Input in Reverse

2.3 Recursive Examples

In this section we discuss four recursive examples: greatest common divisor, Fibonacci numbers, prefix to postfix conversion, and the Towers of Hanoi. For each example, we start with a design and then implement it in C.

Greatest Common Divisor

A common mathematics function is to determine the **greatest common divisor (GCD)** for two numbers. For example, the greatest common divisor for 10 and 25 is 5.

GCD Design

We use the Euclidean algorithm to determine the greatest common divisor between two nonnegative integers. Given two integers, a and b, the greatest common divisor is recursively found using the formula in Figure 2-6.

$$\gcd(a, b) = \begin{cases} a & \text{if } b = 0 \\ b & \text{if } a = 0 \\ \gcd(b, a \bmod b) & \text{otherwise} \end{cases}$$

FIGURE 2-6 Greatest Common Divisor Recursive Definition

The pseudocode design for the Euclidean algorithm is shown in Algorithm 2-4.

ALGORITHM 2-4 Euclidean Algorithm for Greatest Common Divisor

```
Algorithm gcd (a, b)
Calculates greatest common divisor using the Euclidean algo-
rithm.
   Pre  a and b are positive integers greater than 0
   Post greatest common divisor returned
1  if (b equals 0)
   1   return a
2  end if
3  if (a equals 0)
   2   return b
4  end if
5  return gcd (b, a mod b)
end gcd
```

GCD C Implementation

Our implementation of the GCD algorithm uses a driver that asks the user for two numbers. After editing the numbers, it calls a recursive implementation that returns the greatest common divisor.

PROGRAM 2-1 GCD Driver

```
1  /* This program determines the greatest common divisor
2     of two numbers.
3        Written by:
4        Date:
5  */
6  #include <stdio.h>
```

continued

PROGRAM 2-1 GCD Driver *(continued)*

```
 7  #include <ctype.h>
 8
 9  // Prototype Statements
10  int gcd (int a, int b);
11
12  int main (void)
13  {
14  // Local Declarations
15     int  gcdResult;
16
17  // Statements
18     printf("Test GCD Algorithm\n");
19
20     gcdResult = gcd (10, 25);
21     printf("GCD of 10 & 25 is %d", gcdResult);
22     printf("\nEnd of Test\n");
23     return 0;
24  }  // main
25  /* ================ gcd ================
26     Calculates greatest common divisor using the
27     Euclidean algorithm.
28        Pre  a and b are positive integers greater than 0
29        Post greatest common divisor returned
30  */
31  int gcd (int a, int b)
32  {
33     // Statements
34     if (b == 0)
35        return a;
36     if (a == 0)
37        return b;
38     return gcd (b, a % b);
39  }  // gcd
```

```
Results:
Test GCD Algorithm
GCD of 10 & 25 is 5
End of Test
```

Fibonacci Numbers

Fibonacci numbers are named after Leonardo Fibonacci, an Italian mathematician who lived in the early thirteenth century. In this series each number is the sum of the previous two numbers. The first few numbers in the Fibonacci series are

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
```

Design

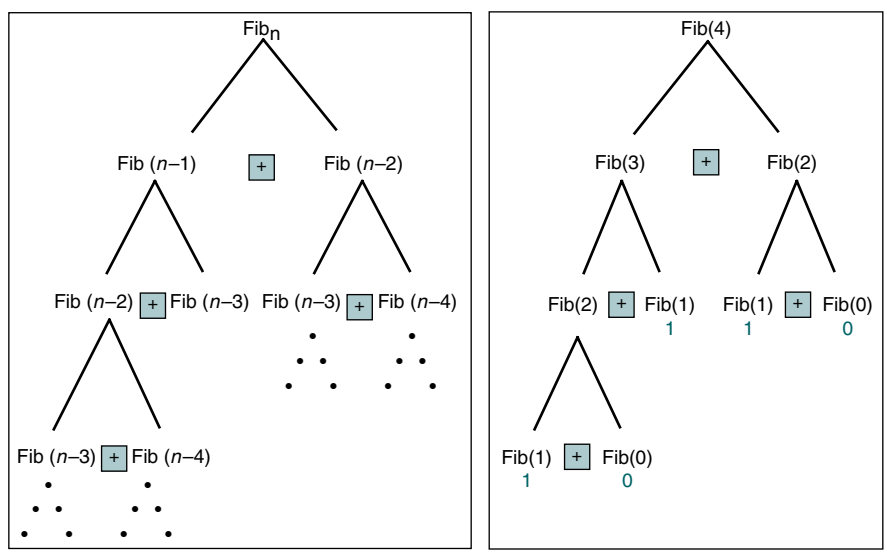
To start a Fibonacci series, we need to know the first two numbers. As you can see from the previous example, they are 0 and 1. Because we are discussing recursion, you should recognize these two numbers as the base cases.

We can generalize the definition of the Fibonacci series as shown in Figure 2-7.

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{otherwise} \end{cases}$$

FIGURE 2-7 Fibonacci Numbers Recursive Definition

The generalization of *Fibonacci(4)* is shown in Figure 2-8. Figure 2-8(a) shows the components of *Fibonacci(4)* using a general notation. Figure 2-8(b) shows the components as they would be called to generate the numbers in the series.



(a) Fib(*n*) (b) Fib(4)

FIGURE 2-8 Fibonacci Numbers

Fibonacci C Implementation

In our Fibonacci program, we begin by asking the user how many numbers are needed. We then use a *for* loop that calls the Fibonacci function the prescribed number of times, printing the next number in the series in each loop.

In the C implementation of a Fibonacci series (Program 2-2), it is interesting to note that the bulk of the code in the algorithm is the mainline code to read the number of numbers to be generated and to print the results. There are actually only three statements in the recursive algorithm.

PROGRAM 2-2 Recursive Fibonacci Series

```
/* This program prints out a Fibonacci series.
      Written by:
      Date:
*/
#include <stdio.h>

// Prototype Statements
   long fib (long num);

int main (void)
{
// Local Declarations
   int seriesSize = 10;

// Statements
   printf("Print a Fibonacci series.\n");

   for (int looper = 0; looper < seriesSize; looper++)
       {
        if (looper % 5)
           printf(", %8ld", fib(looper));
        else
           printf("\n%8ld", fib(looper));
       } // for
   printf("\n");
   return 0;
}  // main

/* ================ fib =================
   Calculates the nth Fibonacci number
      Pre  num identifies Fibonacci number
      Post returns nth Fibonacci number
*/
long fib (long num)
{
// Statements
   if (num == 0 || num == 1)
```

continued

PROGRAM 2-2 Recursive Fibonacci Series *(continued)*

```
38          // Base Case
39          return num;
40       return (fib (num - 1) + fib (num - 2));
41    }  // fib
```

```
Results:
Print a Fibonacci series.

       0,        1,        1,        2,        3
       5,        8,       13,       21,       34
```

Program 2-2 Analysis If you have difficulty following the code, refer to Figure 2-8 and trace the first four Fibonacci numbers. The first 10 numbers are displayed at the end of the algorithm. Unless you have a very fast computer, we recommend that you don't try to calculate more than 30 to 35 numbers.

How many calls does it take to determine *Fibonacci(5)*? The answer is 15. As you can see from Table 2-1, the number of calls goes up quickly as we increase the size of the Fibonacci number we are calculating.

fib(*n*)	Calls	fib(n)	Calls
1	1	11	287
2	3	12	465
3	5	13	753
4	9	14	1219
5	15	15	1973
6	25	20	21,891
7	41	25	242,785
8	67	30	2,692,573
9	109	35	29,860,703
10	177	40	331,160,281

TABLE 2-1 Fibonacci Calls

Table 2-1 leads us to the obvious conclusion that a recursive solution to calculate Fibonacci numbers is not efficient for large numbers.

Prefix to Postfix Conversion

An arithmetic expression can be represented in three different formats: infix, postfix, and prefix. In an **infix** notation, the operator comes between the two operands, the basic format of the algebraic notation we learned in grammar

Chapter 3
Stacks

A **stack** is a linear list in which all additions and deletions are restricted[1] to one end, called the **top.** If you insert a data series into a stack and then remove it, the order of the data is reversed. Data input as {5, 10, 15, 20} is removed as {20, 15, 10, 5}. This reversing attribute is why stacks are known as the **last in–first out (LIFO)** data structure.

We use many different types of stacks in our daily lives. We often talk of a stack of coins or a stack of dishes. Any situation in which you can only add or remove an object at the top is a stack. If you want to remove any object other than the one at the top, you must first remove all objects above it. A graphic representation of a stack is shown in Figure 3-1.

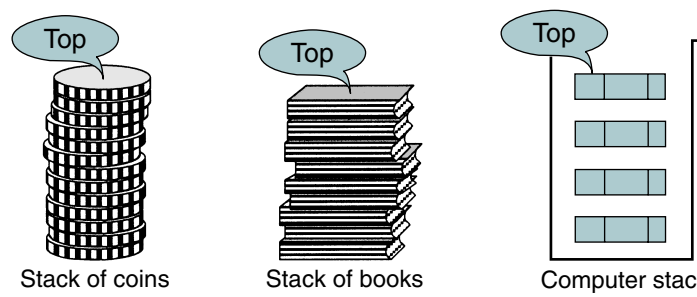


FIGURE 3-1 Stack

1. The stack is one of three data structures known collectively as *restrictive data structures* because the operations are restricted to the ends of the structure. The other two are the queue, which we study in Chapter 4, and the deque or double-ended queue. The deque, which is not covered in this text, allows insertions and deletions at both ends.

Although nothing prevents us from designing a data structure that allows us to perform other operations, such as moving the item at the top of the stack to the bottom, the result would not be a stack.

A stack is a last in–first out (LIFO) data structure in which all insertions and deletions are restricted to one end, called the top.

3.1 Basic Stack Operations

The three basic stack operations are push, pop, and stack top. Push is used to insert data into the stack. Pop removes data from a stack and returns the data to the calling module. Stack top returns the data at the top of the stack without deleting the data from the stack.

Push

Push adds an item at the top of the stack. After the push, the new item becomes the top. The only potential problem with this simple operation is that we must ensure that there is room for the new item. If there is not enough room, the stack is in an **overflow** state and the item cannot be added. Figure 3-2 shows the push stack operation.

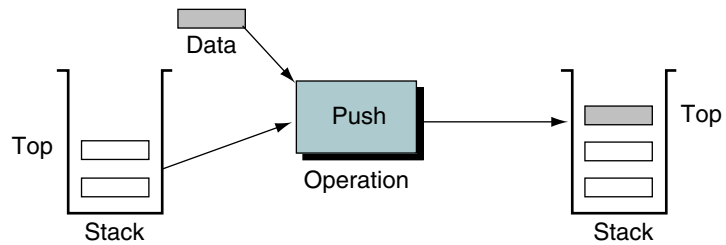


FIGURE 3-2 Push Stack Operation

Pop

When we **pop** a stack, we remove the item at the top of the stack and return it to the user. Because we have removed the top item, the next older item in the stack becomes the top. When the last item in the stack is deleted, the stack must be set to its empty state. If pop is called when the stack is empty, it is in an **underflow** state. The pop stack operation is shown in Figure 3-3.

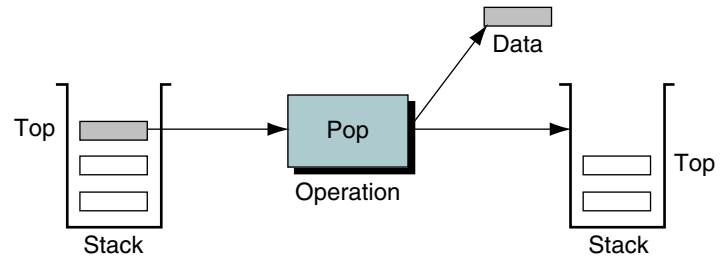


FIGURE 3-3 Pop Stack Operation

Stack Top

The third stack operation is **stack top**. Stack top copies the item at the top of the stack; that is, it returns the data in the top element to the user but does not delete it. You might think of this operation as reading the stack top. Stack top can also result in underflow if the stack is empty. The stack top operation is shown in Figure 3-4.

The three basic stack operations are push, pop, and stack top.

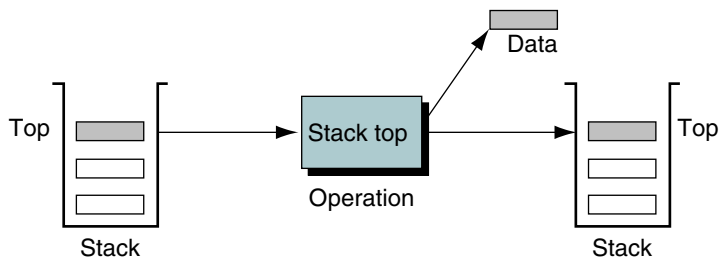


FIGURE 3-4 Stack Top Operation

Figure 3-5 traces these three operations in an example. We start with an empty stack and push *green* and *blue* into the stack. At this point the stack contains two entries. We then pop *blue* from the top of the stack, leaving *green* as the only entry. After pushing *red*, the stack again contains two entries. At this point we retrieve the top entry, *red*, using stack top. Note that stack top does not remove *red* from the stack; it is still the top element. We then pop *red*, leaving *green* as the only entry. When *green* is popped, the stack is again empty. Note also how this example demonstrates the last in–first out operation of a stack. Although *green* was pushed first, it is the last to be popped.

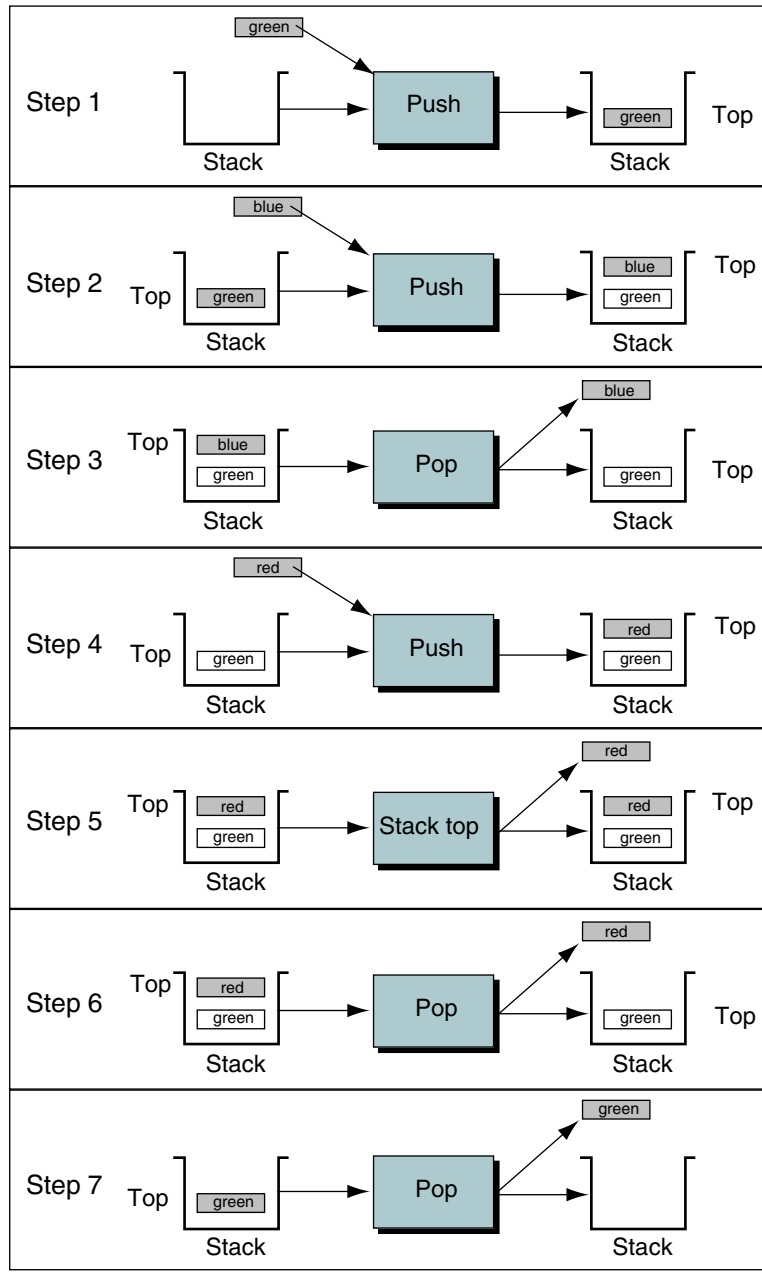


FIGURE 3-5 Stack Example

3.2 Stack Linked List Implementation

Several data structures can be used to implement a stack. In this section we implement the stack as a linked list. We introduced the concept of a linked list in Chapter 1, "Linked List Implementations."

Data Structure

To implement the linked list stack, we need two different structures, a head and a data node. The head structure contains **metadata**—that is, data about data—and a pointer to the top of the stack. The data structure contains data and a link pointer to the next node in the stack. The conceptual and physical implementations of the stack are shown in Figure 3-6.

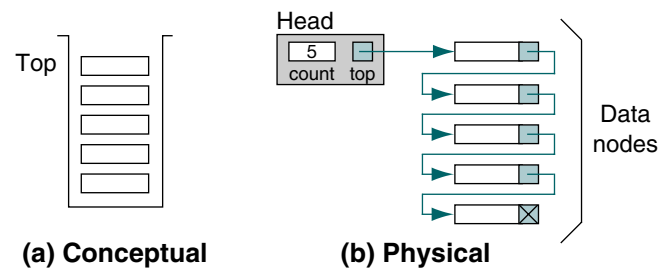


FIGURE 3-6 Conceptual and Physical Stack Implementations

Stack Head

Generally, the **head** for a stack requires only two attributes: a top pointer and a count of the number of elements in the stack. These two elements are placed in a structure. Other stack attributes can be placed here also. For example, it is possible to record the time the stack was created and the total number of items that have ever been placed in the stack. These two metadata items allow the user to determine the average number of items processed through the stack in a given period. Of course, we would do this only if such a statistic were required for some reason. A basic head structure is shown in Figure 3-7.

Stack Data Node

The rest of the data structure is a typical linked list **data node**. Although the application determines the data that are stored in the stack, the stack data node looks like any linked list node. In addition to the data, it contains a link pointer to other data nodes, making it a **self-referential data structure**. In a self-referential structure, each instance of the structure contains a pointer to another instance of the same structure. The stack data node is also shown in Figure 3-7.

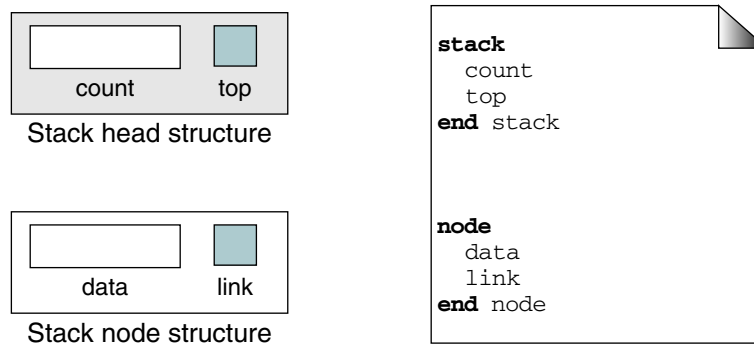


FIGURE 3-7 Stack Data Structure

Stack Algorithms

The eight stack operations defined in this section should be sufficient to solve any basic stack problem. If an application requires additional stack operations, they can be easily added. For each operation we give its name, a brief description, and then develop its algorithm.

Although the implementation of a stack depends somewhat on the implementation language, it is usually implemented with a stack head structure in C. We use the design shown in Figure 3-8, which demonstrates the four most common stack operations: create stack, push stack, pop stack, and destroy stack. Operations such as `stacktop` are not shown in the figure because they do not change the stack structure.

Create Stack

Create stack allocates memory for the stack structure and initializes its metadata. The pseudocode is shown in Algorithm 3-1.

ALGORITHM 3-1 Create Stack

```
Algorithm createStack
Creates and initializes metadata structure.
   Pre    Nothing
   Post   Structure created and initialized
   Return stack head
 1 allocate memory for stack head
 2 set count to 0
 3 set top to null
 4 return stack head
end createStack
```

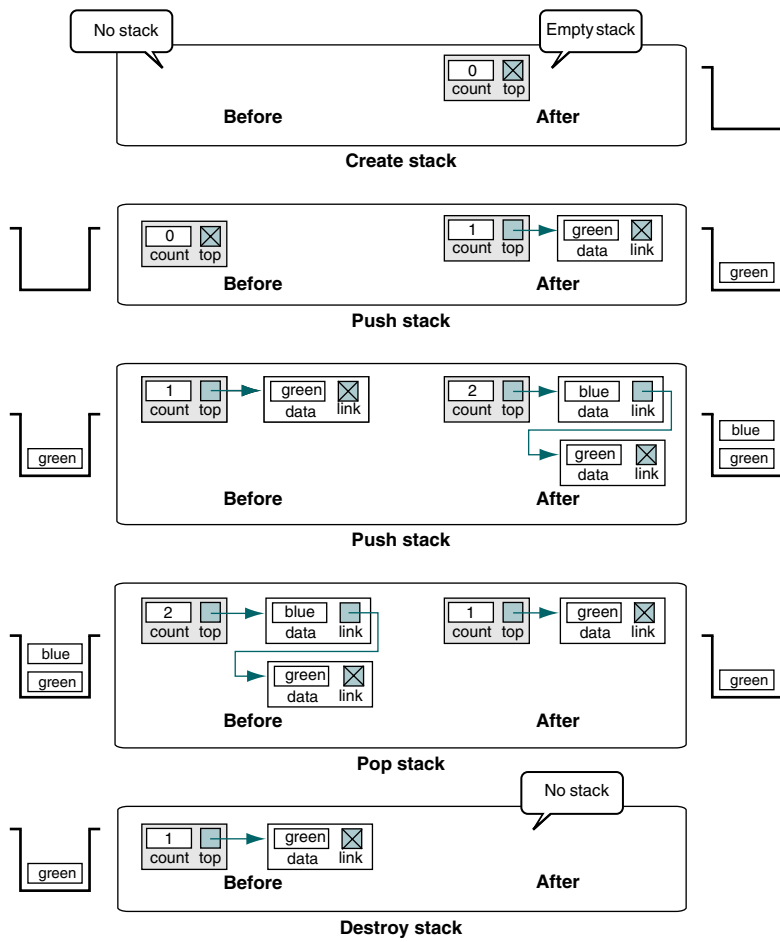


FIGURE 3-8 Stack Operations

Push Stack

Push stack inserts an element into the stack. The first thing we need to do when we push data into a stack is find memory for the node. We must therefore allocate a node from dynamic memory. Once the memory is allocated, we simply assign the data to the stack node and then set the link pointer to point to the node currently indicated as the stack top. We also need to update the stack top pointer and add 1 to the stack count field. Figure 3-9 traces a push stack operation in which a new pointer (`pNew`) is used to identify the data to be inserted into the stack.

To develop the insertion algorithm using a linked list, we need to analyze three different stack conditions: (1) insertion into an empty stack, (2) insertion into a stack with data, and (3) insertion into a stack when the available memory is exhausted. The first two of these situations are shown in Figure 3-8. The third is an error condition.

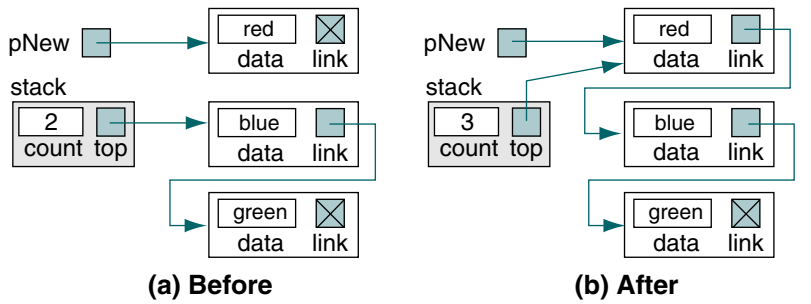


FIGURE 3-9 Push Stack Example

When we insert into a stack that contains data, the new node's link pointer is set to point to the node currently at the top, and the stack's top pointer is set to point to the new node. When we insert into an empty stack, the new node's link pointer is set to null and the stack's top pointer is set to point to the new node. However, because the stack's top pointer is null, we can use it to set the new node's link pointer to null. Thus the logic for inserting into a stack with data and the logic for inserting into an empty stack are identical.

ALGORITHM 3-2 Push Stack Design

```
Algorithm pushStack (stack, data)
Insert (push) one item into the stack.
   Pre  stack passed by reference
        data contain data to be pushed into stack
   Post data have been pushed in stack
 1  allocate new node
 2  store data in new node
 3  make current top node the second node
 4  make new node the top
 5  increment stack count
end pushStack
```

Pop Stack

Pop stack sends the data in the node at the top of the stack back to the calling algorithm. It then adjusts the pointers to logically delete the node. After the node has been logically deleted, it is physically deleted by recycling the memory, that is, returning it to dynamic memory. After the count is adjusted by subtracting 1, the algorithm returns the status to the caller: if the pop was successful, it returns true; if the stack is empty when pop is called, it returns false. The operations for pop stack are traced in Figure 3-10.

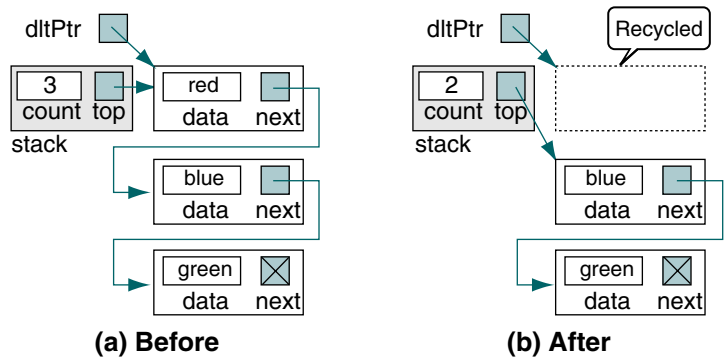


FIGURE 3-10 Pop Stack Example

The pop stack code is shown in Algorithm 3-3.

ALGORITHM 3-3 Pop Stack

```
Algorithm popStack (stack, dataOut)
This algorithm pops the item on the top of the stack and
returns it to the user.
   Pre    stack passed by reference
          dataOut is reference variable to receive data
   Post   Data have been returned to calling algorithm
   Return true if successful; false if underflow
 1 if (stack empty)
   1  set success to false
 2 else
   1  set dataOut to data in top node
   2  make second node the top node
   3  decrement stack count
   4  set success to true
 3 end if
 4 return success
end popStack
```

Algorithm 3-3 Analysis

It is interesting to follow the logic when the last node is being deleted. In this case, the result is an empty stack. No special logic is required; however, the empty stack is created automatically because the last node has a null link pointer, which when moved to top indicates that the stack is empty. A count of zero, which automatically occurs when we decrement the count, is also an indication of an empty stack.

Stack Top

The stack top algorithm (Algorithm 3-4) sends the data at the top of the stack back to the calling module without deleting the top node. This is

included in case the application needs to know what will be deleted with the next pop stack.

ALGORITHM 3-4 Stack Top Pseudocode

```
Algorithm stackTop  (stack, dataOut)
This algorithm retrieves the data from the top of the stack
without changing the stack.
   Pre    stack is metadata structure to a valid stack
          dataOut is reference variable to receive data
   Post   Data have been returned to calling algorithm
   Return true if data returned, false if underflow
 1 if (stack empty)
   1  set success to false
 2 else
   1  set dataOut to data in top node
   2  set success to true
 3 end if
 4 return success
end stackTop
```

Algorithm 3-4 Analysis

The logic for the stack top is virtually identical to that for the pop stack except for the delete logic. As with the pop, there is only one potential problem with the algorithm: the stack may be empty. If the algorithm is successful, it returns true; if the stack is empty, it returns false.

Empty Stack

Empty stack is provided to implement the structured programming concept of data hiding. If the entire program has access to the stack head structure, it is not needed. However, if the stack is implemented as a separately compiled program to be linked with other programs, the calling program may not have access to the stack head node. In these cases it is necessary to provide a way to determine whether the stack is empty. The pseudocode for empty stack is shown in Algorithm 3-5.

ALGORITHM 3-5 Empty Stack

```
Algorithm emptyStack (stack)
Determines if stack is empty and returns a Boolean.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack status
   Return true if stack empty, false if stack contains data
 1 if (stack count is 0)
   1  return true
 2 else
   1  return false
 3 end if
end emptyStack
```

Full Stack

Full stack is another structured programming implementation of data hiding. Depending on the language, it may also be one of the most difficult algorithms to implement. ANSI C, for instance, provides no direct way to implement it. The pseudocode for full stack is shown in Algorithm 3-6.

ALGORITHM 3-6 Full Stack

```
Algorithm fullStack  (stack)
Determines if stack is full and returns a Boolean.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack status
   Return true if stack full, false if memory available
 1 if (memory not available)
   1   return true
 2 else
   1   return false
 3 end if
end fullStack
```

Stack Count

Stack count returns the number of elements currently in the stack. It is another implementation of the data-hiding principle of structured programming. The pseudocode is shown in Algorithm 3-7.

ALGORITHM 3-7 Stack Count

```
Algorithm stackCount (stack)
Returns the number of elements currently in stack.
   Pre    stack is metadata structure to a valid stack
   Post   returns stack count
   Return integer count of number of elements in stack
 1 return (stack count)
end stackCount
```

Destroy Stack

Destroy stack deletes all data in a stack. Figure 3-8 graphically shows the results of destroy stack, and Algorithm 3-8 contains the pseudocode.

ALGORITHM 3-8 Destroy Stack

```
Algorithm destroyStack (stack)
This algorithm releases all nodes back to the dynamic memory.
   Pre    stack passed by reference
   Post   stack empty and all nodes deleted
 1 if (stack not empty)
```

continued

ALGORITHM 3-8 Destroy Stack *(continued)*

```
   1  loop (stack not empty)
      1  delete top node
   2  end loop
 2 end if
 3 delete stack head
end destroyStack
```

Algorithm 3-8 Analysis It is only necessary to destroy a stack when you no longer need it and the program is not complete. If the program is complete, the stack is automatically cleared when the program terminates.

3.3 C Language Implementations

As previously stated, there are two approaches to implementing stacks. We can write unique C programs, which are easy to write but not reusable, or we can create an abstract data type (ADT). In this section we develop simple, but not very reusable, programs. In the next section, we develop the stack ADT.

To demonstrate the push and pop stack algorithms, we write a program that inserts data into a stack. To keep it simple, the data is random uppercase characters. After the characters have been inserted, they are popped and printed. When the stack is empty, the program terminates. While we are not using a stack header, we are using the basic push and pop algorithms discussed in Section 3.2, "Stack Linked List Implementation." The design for this program is shown in Figure 3-11.

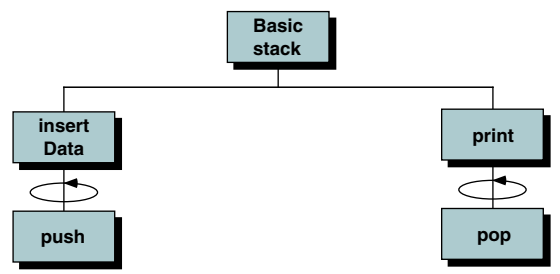


FIGURE 3-11 Design for Basic Stack Program

The node declaration, prototype statements, and test driver are contained in Program 3-1.

PROGRAM 3-1 Simple Stack Application Program

```
/* This program is a test driver to demonstrate the
   basic operation of the stack push and pop functions.
      Written by:
      Date:
*/
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Structure Declarations
typedef struct node
   {
    char         data;
    struct node* link;
   } STACK_NODE;

// Prototype Declarations
void insertData (STACK_NODE** pStackTop);
void print      (STACK_NODE** pStackTop);

bool push       (STACK_NODE** pList, char  dataIn);
bool pop        (STACK_NODE** pList, char* dataOut);

int main (void)
{
// Local Definitions
STACK_NODE* pStackTop;

// Statements
   printf("Beginning Simple Stack Program\n\n");

   pStackTop = NULL;
   insertData  (&pStackTop);
   print       (&pStackTop);

   printf("\n\nEnd Simple Stack Program\n");
   return 0;
}  // main
```

```
Results:
   Beginning Simple Stack Program

   Creating characters: QMZRHLAJOE
   Stack contained:     EOJALHRZMQ

   End Simple Stack Program
```

Program 3-1 Analysis To verify that the program works correctly, we print the characters as they are generated. This allows us to verify that the stack output was in fact correct. Note that this simple program verifies the LIFO operation of a stack.

Insert Data

The insert data function loops while creating characters and inserting them into the stack. To create random characters, we use the random number generator and scale the return value to the uppercase alphabetic range. This code is developed in Program 3-2.

PROGRAM 3-2 Insert Data

```
/* ================== insertData ==================
   This program creates random character data and
   inserts them into a linked list stack.
      Pre  pStackTop is a pointer to first node
      Post Stack has been created
*/
void insertData (STACK_NODE** pStackTop)
{
// Local Definitions
   char  charIn;
   bool  success;

// Statements
   printf("Creating characters: ");
   for (int nodeCount = 0; nodeCount < 10; nodeCount++)
       {
        // Generate uppercase character
        charIn  = rand() % 26 + 'A';
        printf("%c", charIn);
        success = push(pStackTop, charIn);
        if (!success)
           {
            printf("Error 100: Out of Memory\n");
            exit (100);
           } // if
       } // for
   printf("\n");
   return;
}  // insertData
```

Program 3-2 Analysis

The insert logic is straightforward. The generation of the uppercase characters requires a little explanation. Because there are 26 alphabetic characters, we scale the random number to the range 0 to 25. To map the resulting random number to the uppercase characters, we then add the value of the character `'A'` to the random number.

Also note that even in this small example, we test for overflow. Although the probability of an overflow occurring is small, we always test for it. Remember Murphy's Law.[2]

2. Murphy's Law: If something can go wrong, it will.

Push Stack

The insert data function calls the push function. With the exception of testing for overflow technique and the error-handling code, and maintaining the header metadata, it is a direct implementation of Algorithm 3-2. The code is developed in Program 3-3.

PROGRAM 3-3 Push Stack

```
/* ================== push ====================
   Inserts node into linked list stack.
      Pre     pStackTop is pointer to valid stack
      Post    charIn inserted
      Return  true  if successful
              false if underflow
*/
bool push (STACK_NODE** pStackTop, char charIn)
{
// Local Definitions
   STACK_NODE* pNew;
   bool        success;

// Statements
   pNew = (STACK_NODE*)malloc(sizeof (STACK_NODE));
   if (!pNew)
       success = false;
   else
      {
       pNew->data =  charIn;
       pNew->link = *pStackTop;
       *pStackTop =  pNew;
       success = true;
      } // else
   return success;
}  // push
```

Print Stack

Once the stack has been built, we print it to verify the output. The print function calls the pop function until the stack is empty. The print code is found in Program 3-4.

PROGRAM 3-4 Print Stack

```
/* ==================== print ====================
   This function prints a singly linked stack.
      Pre     pStackTop is pointer to valid stack
      Post    data in stack printed
*/
void print (STACK_NODE** pStackTop)
{
```

continued

PROGRAM 3-4 Print Stack *(continued)*

```
 8  // Local Definitions
 9     char printData;
10
11  // Statements
12     printf("Stack contained:     ");
13     while (pop(pStackTop, &printData))
14         printf("%c", printData);
15     return;
16  }  // print
```

Pop Character

The pop character function copies the character at the top of the stack to the output location specified in the second parameter. With the exception of the code to maintain the header metadata, it is an exact implementation of Algorithm 3-3. The implementation is shown in Program 3-5.

PROGRAM 3-5 Pop Stack

```
 1  /* ================== pop ====================
 2     Delete node from linked list stack.
 3        Pre  pStackTop is pointer to valid stack
 4        Post charOut contains deleted data
 5        Return  true  if successful
 6                false if underflow
 7  */
 8  bool pop (STACK_NODE** pStackTop, char* charOut)
 9  {
10  // Local Definitions
11     STACK_NODE* pDlt;
12     bool        success;
13
14  // Statements
15     if (*pStackTop)
16        {
17         success    = true;
18         *charOut   = (*pStackTop)->data;
19         pDlt       = *pStackTop;
20         *pStackTop = (*pStackTop)->link;
21         free (pDlt);
22        } // else
23     else
24        success = false;
25     return success;
26  }  // pop
```

3.4 Stack ADT

Each of the algorithms in the previous section used several stack operations. While we could write them each time we needed them, this would certainly be an inefficient use of our time. What we need, therefore, is a stack abstract data type that we can put in a library and call whenever we need it. We develop a stack abstract data type in this section.[3]

Data Structure

The stack ADT implementation in C is straightforward. Rather than store data in each node, we store a pointer to the data. It is the application program's responsibility to allocate memory for the data and pass the address to the stack ADT. Within the ADT, the stack node looks like any linked list node except that it contains a pointer to the data rather than the actual data. Because the data pointer type is unknown, it is stored as a pointer to *void*.

The head node and the data nodes are encapsulated in the ADT. The calling function's only view of the stack is a pointer to the stack structure in the ADT, which is declared as a type definition. We name this stack type STACK. This design is very similar to C's FILE structure.

To create a stack, the programmer defines a pointer to a stack as shown in the following example and then calls the create stack function. The address of the stack structure is returned by create stack and assigned to the pointer in the calling function.

```
STACK* stack;
…
stack = createStack ();
```

The stack ADT structure is found in Figure 3-12.

ADT Implementation

It takes more than a data structure to make an abstract data type: there must also be operations that support the stack. We develop the C functions in the sections that follow.

Stack Structure

The stack abstract data type structure is shown in Program 3-6. The node structure consists only of a data pointer and a link node pointer. The stack head structure also contains only two elements, a pointer to the top of the stack and a count of the number of entries currently in the stack.

3. The array implementation for a stack can be found in Appendix F.

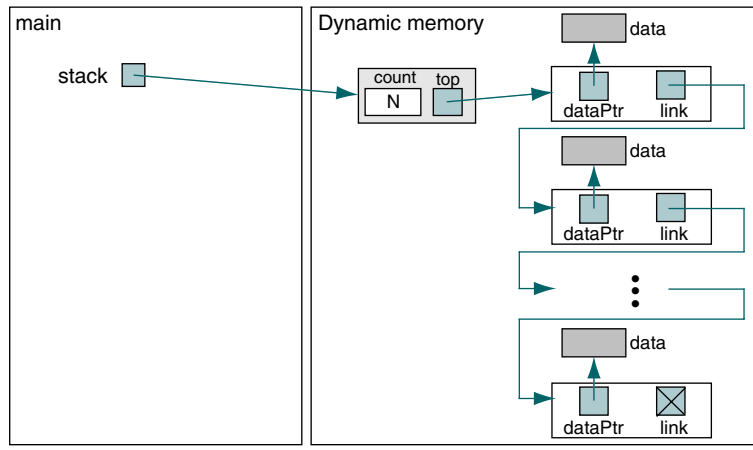


FIGURE 3-12 Stack ADT Structural Concepts

The type definition for the stack, as shown in Program 3-6, is included in a header file so that any function that needs to define a stack can do so easily.

PROGRAM 3-6 Stack ADT Definitions

```
1  // Stack ADT Type Defintions
2     typedef struct node
3        {
4         void*        dataPtr;
5         struct node* link;
6        } STACK_NODE;
7
8     typedef struct
9        {
10        int          count;
11        STACK_NODE* top;
12       } STACK;
```

Create Stack

Create stack allocates a stack head node, initializes the top pointer to null, and zeros the count field. The address of the node in the dynamic memory is then returned to the caller. The call to create a stack must assign the return pointer value to a stack pointer as shown below.

```
stack = createStack ( );
```

Figure 3-8 graphically shows the results of creating a stack. The code for the create stack function is shown in Program 3-7.

PROGRAM 3-7 ADT Create Stack

```
1  /* ============== createStack ==============
2     This algorithm creates an empty stack.
3        Pre  Nothing
4        Post Returns pointer to a null stack
5                -or- NULL if overflow
6  */
7  STACK* createStack (void)
8  {
9  // Local Definitions
10    STACK* stack;
11
12 // Statements
13    stack = (STACK*) malloc( sizeof (STACK));
14    if (stack)
15       {
16        stack->count = 0;
17        stack->top   = NULL;
18       } // if
19    return stack;
20 }  // createStack
```

Push Stack

The first thing that we need to do when we push data into a stack is to find a place for the data. This requires that we allocate memory from the heap using *malloc*. Once the memory is allocated, we simply assign the data pointer to the node and then set the link to point to the node currently indicated as the stack top. We also need to add one to the stack count field. Figure 3-8 shows several pushes into the stack. Program 3-8 is an implementation of push stack.

PROGRAM 3-8 Push Stack

```
1  /* ================ pushStack ================
2     This function pushes an item onto the stack.
3        Pre     stack is a pointer to the stack
4                dataPtr pointer to data to be inserted
5        Post    Data inserted into stack
6        Return  true  if successful
7                false if underflow
8  */
9  bool pushStack (STACK* stack, void* dataInPtr)
10 {
11 // Local Definitions
12    STACK_NODE* newPtr;
13
14 // Statements
15    newPtr = (STACK_NODE* ) malloc(sizeof( STACK_NODE));
16    if (!newPtr)
```

continued

PROGRAM 3-8 Push Stack *(continued)*

```
17          return false;
18
19       newPtr->dataPtr = dataInPtr;
20
21       newPtr->link    = stack->top;
22       stack->top      = newPtr;
23
24       (stack->count)++;
25       return true;
26    }  // pushStack
```

Program 3-8 Analysis The ADT implementation of push differs from a traditional implementation only in its generality. Because it is an abstract data type, we need to handle any type of data pointer; therefore, we use a *void* pointer in the stack node. Recall that a *void* pointer can be used to store any type of pointer, and, conversely, it can be assigned to any pointer type. About the only thing we can't do with a *void* pointer is dereference it. In the stack, however, there is no need to refer to the actual data, so we never need to dereference it. As you study the rest of the stack algorithms, note the use of *void* pointers.

The last point also addresses generic code. Because we are writing an ADT, we cannot assume that heap overflow is an error. That decision belongs to the application. All we can do is report status: either the push was successful or it wasn't. If the allocation is successful, which is the normal case, we report that the push was successful by returning true. If the heap is full, we report that the push failed by returning false. It is then the calling function's responsibility to detect and respond to an overflow.

Pop Stack

Pop stack returns the data in the node at the top of the stack. It then deletes and recycles the node. After the count is adjusted by subtracting 1, the function returns to the caller. Note the way underflow is reported. In statement 15 we set the data pointer to NULL. If the stack is empty, when we return the data pointer in statement 24 we return NULL. Refer to Figure 3-8 for a graphic example of a stack pop in a linked list environment. The code is developed in Program 3-9.

PROGRAM 3-9 ADT Pop Stack

```
 1  /* ================== popStack ==================
 2     This function pops item on the top of the stack.
 3        Pre  stack is pointer to a stack
 4        Post Returns pointer to user data if successful
 5                     NULL if underflow
 6  */
 7  void* popStack (STACK* stack)
 8  {
 9  // Local Definitions
10     void*       dataOutPtr;
```

continued

PROGRAM 3-9 ADT Pop Stack *(continued)*

```
11      STACK_NODE* temp;
12
13  // Statements
14      if (stack->count == 0)
15          dataOutPtr = NULL;
16      else
17         {
18          temp       = stack->top;
19          dataOutPtr = stack->top->dataPtr;
20          stack->top = stack->top->link;
21          free (temp);
22          (stack->count)--;
23         } // else
24      return dataOutPtr;
25  }  // popStack
```

Program 3-9 Analysis

The code for the pop is rather straightforward. Two local pointers are required, one for the data pointer to be returned to the caller, and one that is used to free the deleted node.

It is interesting to follow the logic when the last node is being deleted. In this case the result is an empty stack. No special logic is required: the empty stack is created automatically because the last node has a null pointer, which when assigned to top indicates that the stack is empty.

Because a null pointer is false and a pointer with an address is true, we don't need a separate success flag. Rather, we just return the pointer with the address allocated by the new function. If memory was allocated successfully, it contains an address, which is true. If the allocation failed, it contains a null pointer, which is false.

Study the code that updates the stack top carefully. Note that it takes two levels of indirection to access the link field of the top node. If these references are confusing, trace them using Figure 3-8.

Finally, when we delete a node from the stack, the pop stack releases its memory.

Stack Top

The stack top function returns the data at the top of the stack without deleting the top node. It allows the user to see what will be deleted when the stack is popped. The code is developed in Program 3-10.

PROGRAM 3-10 Retrieve Stack Top

```
1  /* ================== stackTop =================
2     Retrieves data from the top of stack without
3      changing the stack.
4        Pre  stack is a pointer to the stack
5        Post Returns data pointer if successful
6                     null pointer if stack empty
7  */
```

continued

PROGRAM 3-10 Retrieve Stack Top *(continued)*

```
 8  void* stackTop (STACK* stack)
 9  {
10  // Statements
11     if (stack->count == 0)
12         return NULL;
13     else
14         return stack->top->dataPtr;
15  }  // stackTop
```

Program 3-10 Analysis

The logic is virtually identical to the pop stack except for the delete and recycle. The code, however, is significantly different. As with the pop, there is only one potential problem with the retrieve function—the stack may be empty. If the stack is empty, we return a null pointer.

Empty Stack

Because the calling function has no access to the data structure, it cannot determine if there are data in the stack without actually trying to retrieve them. We therefore provide empty stack, a function that simply reports that the stack has data or that it is empty. (See Program 3-11.)

PROGRAM 3-11 Empty Stack

```
 1  /* ================= emptyStack ================
 2     This function determines if a stack is empty.
 3        Pre  stack is pointer to a stack
 4        Post returns 1 if empty; 0 if data in stack
 5  */
 6  bool emptyStack (STACK* stack)
 7  {
 8  // Statements
 9     return (stack->count == 0);
10  }  // emptyStack
```

Full Stack

Full stack is one of the most complex of the supporting functions. There is no straightforward way to tell if the next memory allocation is going to succeed or fail. All we can do is try it. But by trying to make an allocation, we use up part of the heap. Therefore, after allocating space for a node, we immediately free it so that it will be there when the program requests memory. This logic is shown in Program 3-12.

PROGRAM 3-12 Full Stack

```
 1  /* ================== fullStack =================
 2     This function determines if a stack is full.
```

continued

PROGRAM 3-12 Full Stack *(continued)*

```
 3    Full is defined as heap full.
 4       Pre    stack is pointer to a stack head node
 5       Return true if heap full
 6              false if heap has room
 7 */
 8 bool fullStack (STACK* stack)
 9 {
10 // Local Definitions
11 STACK_NODE* temp;
12
13 // Statements
14    if ((temp =
15       (STACK_NODE*)malloc (sizeof(*(stack->top)))))
16       {
17        free (temp);
18        return false;
19       } // if
20
21    // malloc failed
22    return true;
23 }  // fullStack
```

Stack Count

Stack count returns the number of items in the stack. Because this count is stored in the stack head node, we do not need to traverse the stack to determine how many items are currently in it. We simply return the head count.

PROGRAM 3-13 Stack Count

```
 1 /* ================== stackCount =================
 2    Returns number of elements in stack.
 3       Pre  stack is a pointer to the stack
 4       Post count returned
 5 */
 6 int stackCount (STACK* stack)
 7 {
 8 // Statements
 9    return stack->count;
10 }  // stackCount
```

Destroy Stack

Destroy stack deletes the nodes in a stack and returns a null pointer. It is the user's responsibility to set the stack pointer in the calling area to NULL by assigning the return value to the local stack pointer. Because the stack is implemented as a dynamic data structure in the heap, the memory is also released for reuse. Figure 3-8 graphically shows the results of destroying a

stack. Note that the two stack nodes and the stack head are all freed. The code for `destroyStack` is identical to its implementation in Program 3-14.

PROGRAM 3-14 Destroy Stack

```
/* ================== destroyStack =================
   This function releases all nodes to the heap.
      Pre  A stack
      Post returns null pointer
*/
STACK* destroyStack (STACK* stack)
{
// Local Definitions
   STACK_NODE* temp;

// Statements
   if (stack)
      {
       // Delete all nodes in stack
       while (stack->top != NULL)
          {
           // Delete data entry
           free (stack->top->dataPtr);

           temp = stack->top;
           stack->top = stack->top->link;
           free (temp);
          } // while

       // Stack now empty. Destroy stack head node.
       free (stack);
      } // if stack
   return NULL;
}  // destroyStack
```

Program 3-14 Analysis

Note that we guard against a call to destroy a stack that does not exist. Only if there is a stack do we execute the *while* loop. In the loop we walk through the stack, first recycling the user's data nodes and then recycling the stack nodes. When the last node has been freed, the stack top becomes NULL and the loop terminates. After all data nodes have been deleted, we also free the head node and return a null stack pointer.

3.5 Stack Applications

Stack applications can be classified into four broad categories: reversing data, parsing data, postponing data usage, and backtracking steps. For each of these applications we provide one or two examples. For reversing data we reverse a list. We also use reversing data to convert a decimal number to its binary equivalent. For parsing we show how to match the parentheses in a

source program. We use postponement to convert infix to postfix notation and also to evaluate a postfix expression. Finally, we use backtracking to choose between two or more paths. Each of these applications uses the stack ADT described in the previous section. You should make sure that you fully understand these algorithms before studying the following applications.

Four common stack applications are: reversing data, parsing, postponing, and backtracking.

Reversing Data

Reversing data requires that a given set of data be reordered so that the first and last elements are exchanged, with all of the positions between the first and last being relatively exchanged also. For example, {1 2 3 4} becomes {4 3 2 1}. We examine two different reversing applications: reverse a list and convert decimal to binary.

Reverse a List

One of the applications of a stack is to reverse a list of items. For example, Program 3-15 reads a list of integers and prints them in reverse.

PROGRAM 3-15 Reverse a Number Series

```
/* This program reverses a list of integers read
   from the keyboard by pushing them into a stack
   and retrieving them one by one.
      Written by:
      Date:
*/
#include <stdio.h>
#include <stdbool.h>
#include "stacksADT.h"

int main (void)
{
// Local Definitions
   bool done = false;
   int* dataPtr;

   STACK* stack;

// Statements
   // Create a stack and allocate memory for data
   stack = createStack ();

   // Fill stack
   while (!done)
      {
```

continued

PROGRAM 3-15 Reverse a Number Series *(continued)*

```
26          dataPtr = (int*) malloc (sizeof(int));
27          printf ("Enter a number: <EOF> to stop: ");
28          if ((scanf ("%d" , dataPtr)) == EOF
29                  || fullStack (stack))
30             done = true;
31          else
32             pushStack (stack, dataPtr);
33         } // while
34
35   // Now print numbers in reverse
36      printf ("\n\nThe list of numbers reversed:\n");
37      while (!emptyStack (stack))
38         {
39          dataPtr = (int*)popStack (stack);
40          printf ("%3d\n", *dataPtr);
41          free (dataPtr);
42         } // while
43
44   // Destroying Stack
45      destroyStack (stack);
46      return 0;
47   }  // main
```

```
Results:
Enter a number: <EOF> to stop: 3
Enter a number: <EOF> to stop: 5
Enter a number: <EOF> to stop: 7
Enter a number: <EOF> to stop: 16
Enter a number: <EOF> to stop: 91
Enter a number: <EOF> to stop:

The list of numbers reversed:
 91
 16
  7
  5
  3
```

Program 3-15 Analysis

This program is very simple. After creating a stack, it reads a series of numbers and pushes them into the stack. When the user keys end of file, the program then pops the stack and prints the numbers in reverse order.

The important point to note in this simple program is that we never referenced the stack structure directly. All stack references were through the stack ADT interface. This is an important concept in the structured programming principles of encapsulation and function reusability.

Figure 3-13 follows the program as it executes. As you study Program 3-15, use Figure 3-13 to trace its operations. Make sure that you have the same results at each different point in the figure.

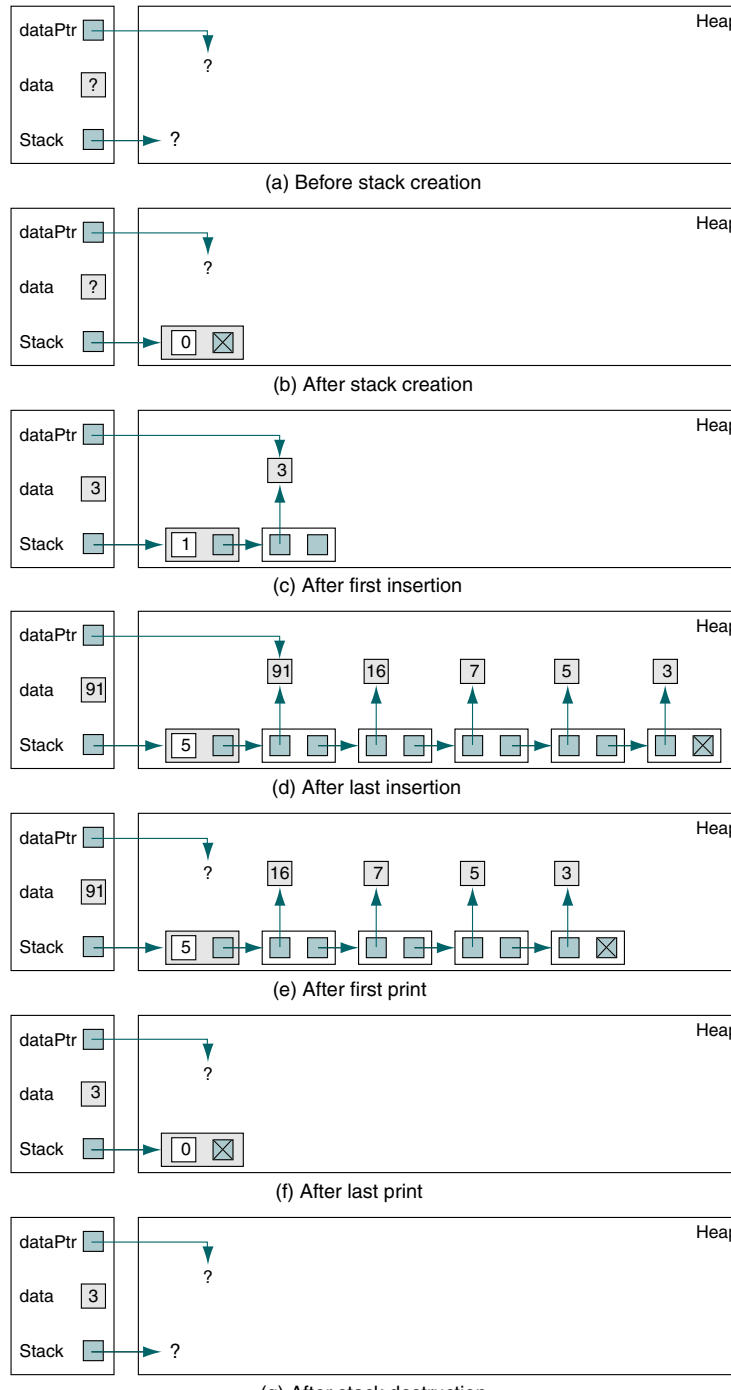


FIGURE 3-13 Program 3-15 Step-by-Step Execution

Convert Decimal to Binary

The idea of reversing a series can be used in solving classical problems such as transforming a decimal number to a binary number. The following simple code segment transforms a decimal number into a binary number:

```
1  read (number)
2  loop (number > 0)
   1  set digit to number modulo 2
   2  print (digit)
   3  set number to quotient of number / 2
3  end loop
```

This code has a problem, however. It creates the binary number backward. Thus, 19 becomes `11001` rather than `10011`.

We can solve this problem by using a stack. Instead of printing the binary digit as soon as it is produced, we push it into the stack. Then, after the number has been completely converted, we simply pop the stack and print the results one digit at a time in a line. This program is shown in Program 3-16.

PROGRAM 3-16 Convert Decimal to Binary

```
1  /* This program reads an integer from the keyboard
2     and prints its binary equivalent. It uses a stack
3     to reverse the order of 0s and 1s produced.
4        Written by:
5        Date:
6  */
7  #include <stdio.h>
8  #include "stacksADT.h"
9
10 int main (void)
11 {
12 // Local Definitions
13    unsigned int    num;
14             int*   digit;
15             STACK* stack;
16
17 // Statements
18    // Create Stack
19    stack = createStack ();
20
21    // prompt and read a number
22    printf ("Enter an integer:      ");
23    scanf ("%d", &num);
24
25    // create 0s and 1s and push them into the stack
26    while (num > 0)
```

continued

PROGRAM 3-16 Convert Decimal to Binary *(continued)*

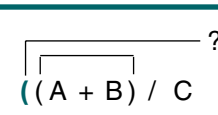
```
27          {
28           digit  = (int*) malloc (sizeof(int));
29           *digit = num % 2;
30           pushStack (stack, digit);
31           num = num /2;
32          } // while
33
34       // Binary number created. Now print it
35       printf ("The binary number is : ");
36       while (!emptyStack (stack))
37          {
38           digit = (int*)popStack (stack);
39           printf ("%1d", *digit);
40          } // while
41       printf ("\n");
42
43       // Destroying Stack
44       destroyStack (stack);
45       return 0;
46   }  // main
```

```
Results:
Enter an integer:      45
The binary number is : 101101
```

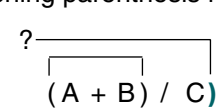
Parsing

Another application of stacks is parsing. **Parsing** is any logic that breaks data into independent pieces for further processing. For example, to translate a source program to machine language, a compiler must parse the program into individual parts such as keywords, names, and tokens.

One common programming problem is unmatched parentheses in an algebraic expression. When parentheses are unmatched, two types of errors can occur: the opening parenthesis can be missing or the closing parenthesis can be missing. These two errors are shown in Figure 3-14.



(a) Opening parenthesis not matched



(b) Closing parenthesis not matched

FIGURE 3-14 Unmatched Parentheses Examples

In Algorithm 3-9, we parse a source program to ensure that all of the parentheses are properly paired.

ALGORITHM 3-9 Parse Parentheses

```
Algorithm parseParens
This algorithm reads a source program and parses it to make
sure all opening-closing parentheses are paired.
 1  loop (more data)
    1   read (character)
    2   if (opening parenthesis)
        1   pushStack (stack, character)
    3   else
        1   if (closing parenthesis)
            1   if (emptyStack (stack))
                1   print (Error: Closing parenthesis not matched)
            2   else
                1   popStack(stack)
            3   end if
        2   end if
    4   end if
 2  end loop
 3  if (not emptyStack (stack))
    1   print (Error: Opening parenthesis not matched)
end parseParens
```

The implementation is shown in Program 3-17.

PROGRAM 3-17 Verify Parentheses Paired in Source Program

```
 1  /* This program reads a source program and parses it to
 2     make sure all opening-closing parentheses are paired.
 3        Written by:
 4        Date:
 5  */
 6  #include <stdio.h>
 7  #include "stacksADT.h"
 8
 9  // Error Messages
10  const char closMiss[] = "Close paren missing at line";
11  const char openMiss[] = "Open paren missing at line";
12
13  int main (void)
14  {
15  // Local Definitions
16     STACK* stack;
17     char   token;
18     char*  dataPtr;
```

continued

PROGRAM 3-17 Verify Parentheses Paired in Source Program *(continued)*

```
19      char   fileID[25];
20      FILE*  fpIn;
21      int    lineCount = 1;
22
23  // Statements
24      // Create Stack
25      stack = createStack ();
26      printf ("Enter file ID for file to be parsed: ");
27      scanf  ("%s", fileID);
28
29      fpIn = fopen (fileID, "r");
30      if (!fpIn)
31          printf("Error opening %s\n", fileID), exit(100);
32
33      // read characters from the source code and parse
34      while ((token = fgetc (fpIn)) != EOF )
35         {
36          if (token == '\n')
37             lineCount++;
38          if (token == '(' )
39             {
40              dataPtr = (char*) malloc (sizeof (char));
41              pushStack (stack, dataPtr);
42             } // if
43          else
44             {
45              if (token == ')')
46                 {
47                  if (emptyStack (stack))
48                     {
49                      printf ("%s %d\n",
50                              openMiss, lineCount);
51                      return 1;
52                     } // if true
53                  else
54                     popStack (stack);
55                 } // token ==
56             } // else
57         } // while
58
59      if (!emptyStack (stack))
60         {
61          printf ("%s %d\n", closMiss, lineCount);
62          return 1;
63         } // if
64
65      // Now destroy the stack
66      destroyStack (stack);
```

continued

PROGRAM 3-17 Verify Parentheses Paired in Source Program *(continued)*

```
67      printf ("Parsing is OK: %d Lines parsed.\n",
68                lineCount);
69      return 0;
70  }  // main
```

```
Results:
Run 1:
   Enter file ID for file to be parsed: no-errors.txt
   Parsing is OK: 65 Lines parsed.
Run 2:
   Enter file ID for file to be parsed: close-match.txt
   Close paren missing at line 46
Run 3:
   Enter file ID for file to be parsed: open-match.txt
   Open paren missing at line 23
```

Program 3-17 Analysis

Whenever we find an opening parenthesis in a program, we push it into the stack. When we find a closing parenthesis, we pop its matching opening parenthesis from the stack. Note that there are two different pairing errors in the program: (1) a closing parenthesis without a matching opening parenthesis (see statement 49) and (2) an opening parenthesis without a matching closing parenthesis (see statement 61).

To test the program, we ran it three times against three different files. While this testing demonstrates all three situations, it does not fully test the program. Additional test situations, such as missing first opening parenthesis, must also be run.

Finally, note how we placed the error messages at the beginning of the program. This is a common documentation technique that not only provides good documentation but makes the program easier to read.

Postponement

When we used a stack to reverse a list, the entire list was read before we began outputting the results. Often the logic of an application requires that the usage of data be deferred until some later point. A stack can be useful when the application requires that the use of data be postponed for a while. We develop two stack postponement applications in this section: infix to postfix transformation and postfix expression evaluation.

Infix to Postfix Transformation

One of the disadvantages of the infix notation[4] is that we need to use parentheses to control the evaluation of the operators. We thus have an evaluation method that includes parentheses and two operator priority classes. In the postfix and prefix notations, we do not need parentheses; each provides only one evaluation rule.

Although some high-level languages use infix notation, such expressions cannot be directly evaluated. Rather, they must be analyzed to determine the

4. For a discussion of arithmetic notations, see "Prefix to Postfix Conversion" in Chapter 2.

order in which the expressions are to be evaluated. A common evaluation technique is to convert the expressions to postfix notation before generating the code to evaluate them. We demonstrate this concept here. We first examine a manual method for converting infix to postfix expressions and then develop an algorithm that can be implemented on a computer.

Manual Transformation

The rules for manually converting infix to postfix expressions are as follows:

1. Fully parenthesize the expression using any explicit parentheses and the arithmetic precedence—multiply and divide before add and subtract.
2. Change all infix notations in each parenthesis to postfix notation, starting from the innermost expressions. Conversion to postfix notation is done by moving the operator to the location of the expression's closing parenthesis.
3. Remove all parentheses.

For example, for the following infix expression:

```
A + B * C
```

Step 1 results in

```
( A + ( B * C ) )
```

Step 2 moves the multiply operator after C

```
( A + ( B C * ) )
```

and then moves the addition operator to between the last two closing parentheses. This change is made because the closing parenthesis for the plus sign is the last parenthesis. We now have

```
( A (B C * ) +)
```

Finally, step 3 removes the parentheses.

```
A B C * +
```

Let's look at a more complex example. This example is not only longer but it already has one set of parentheses to override the default evaluation order.

```
( A + B ) * C  + D + E * F - G
```

Step 1 adds parentheses.

```
( ( ( ( ( A + B ) * C ) + D ) + ( E * F ) ) - G )
```

Step 2 then moves the operators.

```
( ( ( ( ( A B + ) C * ) D + ) ( E F * ) + ) G - )
```

Step 3 removes the parentheses.

```
A B + C * D + E F * + G -
```

Algorithmic Transformation

This manual operation would be difficult to implement in a computer. Let's look at another technique that is easily implemented with a stack.

We start with a very simple example, transforming a multiplication operation. The multiplication of two variables is shown below, first in infix notation and then in postfix notation.

```
A * B    converts to    A B *
```

We can obviously read the operands and output them in order. The problem becomes how to handle the multiply operator; we need to postpone putting it in the output until we have read the right operand, `B`. In this simple case, we push the operator into a stack and, after the whole infix expression has been read, pop the stack and put the operator in the postfix expression.

Now let's look at a more complex expression. Again, the infix expression is given on the left and the equivalent postfix expression on the right.

```
A * B + C    converts to    A B * C +
```

Here again we can read the infix operators and copy them to the postfix expression. If we were to simply put the operators into the stack as we did earlier and then pop them to the postfix expression after all of the operands had been read, we would get the wrong answer. Somehow we must pair the two operators with their correct operands. One possible rule might be to postpone an operator only until we get another operator. Then, before we push the second operator, we could pop the first one and place it in the output expression. This logic works in this case, but it won't for others. Consider the following example.

```
A + B * C    converts to    A B C * +
```

As we discussed previously, infix expressions use a precedence rule to determine how to group the operands and operators in an expression. We can use the same rule when we convert infix to postfix. When we need to push an operator into the stack, if its priority is higher than the operator at the top of the stack, we go ahead and push it into the stack. Conversely, if the operator at the top of the stack has a higher priority than the current operator, it is popped and placed in the output expression. Using this rule with the above expression, we would take the following actions:

1. Copy operand `A` to output expression.
2. Push operator + into stack.
3. Copy operand `B` to output expression.
4. Push operator `*` into stack. (Priority of `*` is higher than +.)
5. Copy operand `C` to output expression.

6. Pop operator * and copy to output expression.
7. Pop operator + and copy to output expression.

We need to cover one more rule to complete the logic. When a current operator with a lower or equal priority forces the top operator to be popped from the stack, we must check the new top operator. If it is also greater than the current operator, it is popped to the output expression. Consequently, we may pop several operators to the output expression before pushing the new operator into the stack.

Let's work on one more example before we formally develop the algorithm.

`A + B * C - D / E` converts to `A B C * + D E / -`

The transformation of this expression is shown in Figure 3-15. Because it uses all of the basic arithmetic operators, it is a complete test.

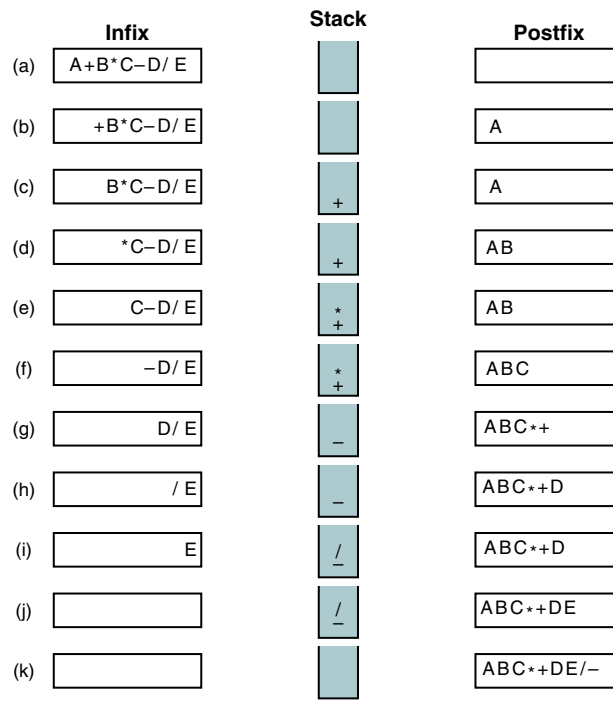


FIGURE 3-15 Infix Transformations

We begin by copying the first operand, `A`, to the postfix expression. See Figure 3-15(b). The add operator is then pushed into the stack and the second operand is copied to the postfix expression. See Figure 3-15(d). At this point we are ready to insert the multiply operator into the stack. As we see in

Figure 3-15(e), its priority is higher than that of the add operator at the top of the stack, so we simply push it into the stack. After copying the next operand, C, to the postfix expression, we need to push the minus operator into the stack. Because its priority is lower than that of the multiply operator, however, we must first pop the multiply and copy it to the postfix expression. The plus sign is now popped and appended to the postfix expression because the minus and plus have the same priority. The minus is then pushed into the stack. The result is shown in Figure 3-15(g). After copying the operand D to the postfix expression, we push the divide operator into the stack because it is of higher priority than the minus at the top of the stack in Figure 3-15(i). After copying E to the postfix expression, we are left with an empty infix expression and two operators in the stack. See Figure 3-15(j). All that is left at this point is to pop the stack and copy each operator to the postfix expression. The final expression is shown in Figure 3-15(k).

We are now ready to develop the algorithm. We assume only the operators shown below. They have been adapted from the standard algebraic notation.

```
Priority 2:    *  /
Priority 1:    +  -
Priority 0:    (
```

The design is shown in Algorithm 3-10.

ALGORITHM 3-10 Convert Infix to Postfix

```
Algorithm inToPostFix (formula)
Convert infix formula to postfix.
   Pre    formula is infix notation that has been edited
          to ensure that there are no syntactical errors
   Post   postfix formula has been formatted as a string
   Return postfix formula
 1 createStack (stack)
 2 loop (for each character in formula)
   1  if (character is open parenthesis)
      1  pushStack (stack, character)
   2  elseif (character is close parenthesis)
      1  popStack (stack, character)
      2  loop (character not open parenthesis)
         1  concatenate character to postFixExpr
         2  popStack (stack, character)
      3  end loop
   3  elseif (character is operator)
         Test priority of token to token at top of stack
      1  stackTop (stack, topToken)
      2  loop (not emptyStack (stack)
           AND priority(character) <= priority(topToken))
         1  popStack (stack, tokenOut)
         2  concatenate tokenOut to postFixExpr
         3  stackTop (stack, topToken)
```

continued

ALGORITHM 3-10 Convert Infix to Postfix *(continued)*

```
      3  end loop
      4  pushStack (stack, token)
   4  else
         Character is operand
      1 Concatenate token to postFixExpr
   5  end if
3  end loop
   Input formula empty. Pop stack to postFix
4  loop (not emptyStack (stack))
   1  popStack (stack, character)
   2  concatenate token to postFixExpr
5  end loop
6  return postFix
end inToPostFix
```

The code follows in Program 3-18.

PROGRAM 3-18 Convert Infix to Postfix

```
 1  /* This program converts an infix formula to a postfix
 2     formula. The infix formula has been edited to ensure
 3     that there are no syntactical errors.
 4         Written by:
 5         Date:
 6  */
 7  #include <stdio.h>
 8  #include <string.h>
 9  #include "stacksADT.h"
10
11  // Prototype Declarations
12  int  priority (char token);
13  bool isOperator (char token);
14
15  int main (void)
16  {
17  // Local Definitions
18     char   postfix [80] = {0};
19     char   temp [2] = {0};
20     char   token;
21     char*  dataPtr;
22     STACK* stack;
23
24  // Statements
25     // Create Stack
26     stack = createStack ();
27
28     // read infix formula and parse char by char
29     printf("Enter an infix formula: ");
```

continued

PROGRAM 3-18 Convert Infix to Postfix *(continued)*

```
30
31    while ((token = getchar ())!= '\n')
32       {
33        if (token == '(')
34           {
35            dataPtr  = (char*) malloc (sizeof(char));
36            *dataPtr = token;
37            pushStack (stack, dataPtr);
38           } // if
39        else if (token == ')')
40           {
41            dataPtr = (char*)popStack (stack);
42            while (*dataPtr != '(')
43               {
44                temp [0]= *dataPtr;
45                strcat (postfix , temp);
46                dataPtr = (char*)popStack (stack);
47               } // while
48           } // else if
49        else if (isOperator (token))
50           {
51            // test priority of token at stack top
52            dataPtr = (char*)stackTop (stack);
53            while (!emptyStack (stack)
54              && priority (token) <= priority (*dataPtr))
55               {
56                dataPtr  = (char*)popStack (stack);
57                temp [0] = *dataPtr;
58                strcat (postfix , temp);
59                dataPtr  = (char*)stackTop (stack);
60               } // while
61            dataPtr  = (char*) malloc (sizeof (char));
62            *dataPtr = token;
63            pushStack (stack , dataPtr);
64           } // else if
65        else                    // character is operand
66           {
67            temp[0]= token;
68            strcat (postfix , temp);
69           } // else
70       } // while get token
71
72    // Infix formula empty. Pop stack to postfix
73    while (!emptyStack (stack))
74       {
75        dataPtr = (char*)popStack (stack);
76        temp[0] = *dataPtr;
77        strcat (postfix , temp);
```

continued

PROGRAM 3-18 Convert Infix to Postfix *(continued)*

```
78          } // while
79
80       // Print the postfix
81       printf ("The postfix formula is: ");
82       puts (postfix);
83
84       // Now destroy the stack
85       destroyStack (stack);
86       return 0;
87    }  // main
88    /* ================== priority ====================
89       Determine priority of operator.
90          Pre  token is a valid operator
91          Post token priority returned
92    */
93    int priority (char token)
94    {
95    // Statements
96       if (token == '*' || token == '/')
97           return 2;
98       if (token == '+' || token == '-')
99           return 1;
100      return 0;
101   }  // priority
102   /* ================== isOperator ====================
103      Determine if token is an operator.
104         Pre  token is a valid operator
105         Post return true if operator; false if not
106   */
107   bool isOperator (char token)
108   {
109   // Statements
110      if (token == '*'
111          || token == '/'
112          || token == '+'
113          || token == '-')
114           return true;
115      return false;
116   }  // isOperator
```

```
Results:
Run 1
   Enter an infix formula: 2+4
   The postfix formula is: 24+
Run 2
   Enter an infix formula: (a+b)*(c-d)/e
   The postfix formula is: ab+cd-*e/
```

Program 3-18 Analysis

An infix expression can contain only three objects: a parenthetical set of operators, a variable identifier or a constant, and an operator. If we have an opening parenthesis,

we put it in the stack. It is eventually paired with a closing parenthesis that signals the end of an expression or a subexpression. Variables and constants go immediately to the output string; thus only operators require more analysis.

Given an operator to be processed, the question is: Should it be pushed into the stack or placed in the output formula? If the stack is empty, the operator is pushed into the stack and we continue.

Infix operators have two priorities. The multiply and divide operators have a higher priority (2) than the add and subtract operators (1). Because the opening parenthesis is also pushed into the operator stack, we give it a priority of 0. We check the priority of the new current token against the priority of the token at the top of the stack at statement 54.

If the new operator's priority is lower than or equal to that of the operator at the top of the stack, the token at the top of the stack is moved to the output string and we loop to recheck the operator at the top of the stack. If the new operator has a higher priority than the operator at the top of the stack, it goes into the stack and we move to the next token in the input formula.

Evaluating Postfix Expressions

Now let's see how we can use stack postponement to evaluate the postfix expressions we developed earlier. For example, given the expression shown below,

```
A B C + *
```

and assuming that `A` is 2, `B` is 4, and `C` is 6, what is the expression value?

The first thing you should notice is that the operands come before the operators. This means that we will have to postpone the use of the operands this time, not the operators. We therefore put them into the stack. When we find an operator, we pop the two operands at the top of the stack and perform the operation. We then push the value back into the stack to be used later. Figure 3-16 traces the operation of our expression. (Note that we push the operand values into the stack, not the operand names. We therefore use the values in the figure.)

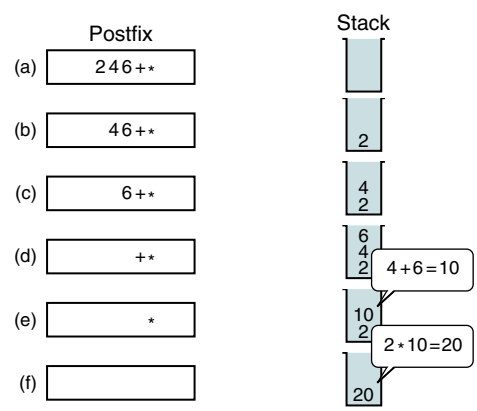


FIGURE 3-16 Evaluation of Postfix Expression

When the expression has been completely evaluated, its value is in the stack. See Algorithm 3-11. The implementation is shown in Program 3-19.

ALGORITHM 3-11 Evaluation of Postfix Expressions

```
Algorithm postFixEvaluate (expr)
This algorithm evaluates a postfix expression and returns its
value.
   Pre    a valid expression
   Post   postfix value computed
   Return value of expression
 1 createStack (stack)
 2 loop (for each character)
   1  if (character is operand)
      1  pushStack (stack, character)
   2  else
      1  popStack (stack, oper2)
      2  popStack (stack, oper1)
      3  operator = character
      4  set value to calculate (oper1, operator, oper2)
      5  pushStack (stack, value)
   3  end if
 3 end loop
 4 popStack (stack, result)
 5 return (result)
end postFixEvaluate
```

PROGRAM 3-19 Evaluate Postfix Expression

```
 1  /* This program evaluates a postfix expression and
 2     returns its value. The postfix expression must be
 3     valid with each operand being only one digit.
 4        Written by:
 5        Date:
 6  */
 7  #include <stdio.h>
 8  #include <stdlib.h>
 9  #include "stacksADT.h"
10
11  // Prototype Declarations
12  bool isOperator (char token);
13  int  calc       (int operand1, int oper, int operand2);
14
15  int main (void)
16  {
17  // Local Definitions
18     char   token;
19     int    operand1;
20     int    operand2;
21     int    value;
```

continued

PROGRAM 3-19 Evaluate Postfix Expression *(continued)*

```
22    int*   dataPtr;
23    STACK* stack;
24
25 // Statements
26    // Create Stack
27    stack = createStack ();
28
29    // read postfix expression, char by char
30    printf("Input formula: ");
31    while ((token = getchar ())!= '\n')
32       {
33        if (!isOperator (token))
34           {
35            dataPtr  = (int*) malloc (sizeof (int));
36            *dataPtr = atoi (&token);
37            pushStack (stack, dataPtr);
38           } // while
39
40        else
41           // character is operand
42           {
43            dataPtr  = (int*)popStack (stack);
44            operand2 = *dataPtr;
45            dataPtr  = (int*)popStack (stack);
46            operand1 = *dataPtr;
47            value    = calc(operand1, token, operand2);
48            dataPtr  = (int*) malloc (sizeof (int));
49            *dataPtr = value;
50            pushStack (stack, dataPtr);
51           } // else
52       } // while
53
54    // The final result is in stack. Pop it print it
55    dataPtr = (int*)popStack (stack);
56    value = *dataPtr;
57    printf ("The result is: %d\n", value);
58
59
60    // Now destroy the stack
61    destroyStack (stack);
62    return 0;
63 }  // main
64 /* ==================== isOperator ==================
65    Validate operator.
66       Pre  token is operator to be validated
67       Post return true if valid, false if not
68 */
69 bool isOperator (char token)
```

continued

PROGRAM 3-19 Evaluate Postfix Expression *(continued)*

```
{
// Statements
   if (token == '*'
       || token == '/'
       || token == '+'
       || token == '-')
      return true;
   return false;
}  // isOperator
/* ==================== calc ==================
   Given two values and operator, determine value of
   formula.
      Pre  operand1 and operand2 are values
           oper is the operator to be used
      Post return result of calculation
*/
int calc (int operand1, int oper, int operand2)
{
// Local Declaration
   int result;

// Statements
   switch (oper)
      {
       case '+' : result  = operand1 + operand2;
                  break;
       case '-' : result  = operand1 - operand2;
                  break;
       case '*' : result  = operand1 * operand2;
                  break;
       case '/' : result  = operand1 / operand2;
                  break;
      } // switch
   return result;
}  // calc
```

```
Results:
Input formula: 52/4+5*2+
The result is 32
```

Program 3-19 Analysis

We have omitted all of the error handling in this algorithm. We assume that the expression is valid and that there is sufficient memory to evaluate it. An implementation obviously needs to guard against these problems.

Given these assumptions, the algorithm is relatively simple. We create a stack and then push operands into the stack until we find an operator. As we find each operator, we pop the two top operands and perform the operation. The result is then pushed into the stack and the next element of the expression is examined. At the end, we pop the value from the stack and return it.

Backtracking

Backtracking is another stack use found in applications such as computer gaming, decision analysis, and expert systems. We examine two backtracking applications in this section: goal seeking and the eight queens problem.

Goal Seeking

Figure 3-17 is an example of a goal-seeking application. One way to portray the problem is to lay out the steps in the form of a graph that contains several alternate paths. Only one of the paths in the figure leads to a desired goal. Whereas we can immediately see the correct path when we look at the figure, the computer needs an algorithm to determine the correct path.

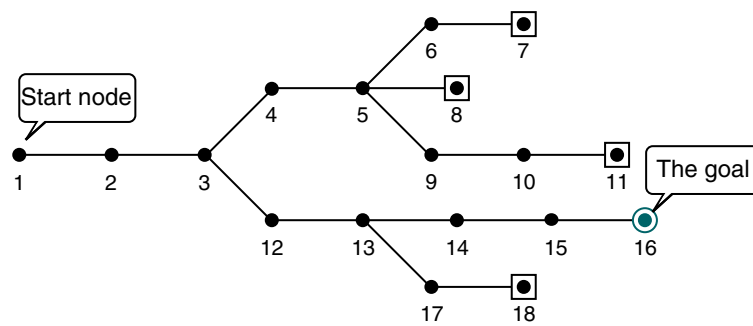


FIGURE 3-17 Backtracking Example

We start at node 1 and move right until we hit a branching node, 3. At this point we take the upper path. We continue until we get to node 5, at which time we again take the upper path. When we arrive at node 7, we can go no further. However, we have not yet reached our goal. We must therefore backtrack to node 5 and take the next path. At node 8 we again backtrack to node 5 and take the third path. Once again, at node 11, we must backtrack, this time way back to node 3. As we follow the path from node 12 we finally arrive at node 16, our goal. Note that we did not examine the path that contains nodes 17 and 18.

By now you should have begun to formulate an algorithm in your head. Every time we get to a decision point, we need to remember where it is so that we can get back to it if necessary. When we backtrack we want to go back to the nearest point before we continue; that is, we don't want to start at the beginning again. For this problem we use the LIFO data structure, the stack.

The question now is what to put into the stack. If we only needed to locate the node that contains the goal, we would just put the branch point nodes into the stack. However, when we are finished we want to print out the path that leads to our goal. Therefore, we must also put the nodes in the valid

path into the stack. Because we are putting two things into the stack, we need to be able to tell them apart. We do that with a flag. If the node is in the path, we push a path token. If we are storing a backtracking point, we set the flag to a backtracking token.

Figure 3-18 contains our stack tracing of the path to our goal in Figure 3-17. We use B to indicate that we have stored a backtracking node. If there is no token, the stack contains a path node.

We start by pushing 1, 2, and 3 into the stack. Because 3 is not our goal, we follow the upper path. However, we need to remember that we made a decision here. Therefore, we push the branch point, 12, into a stack with a backtracking token, as shown in Figure 3-18(a). At 5 we must make another decision. Again, we follow the upper path. This time we have two continue points, 8 and 9, that we need to remember. So, we push both into the stack with backtracking tokens. This point is shown in Figure 3-18(b). At 7 we have reached the end of the path without finding our goal. See Figure 3-15(c). To continue we pop the stack until we are at a backtracking point. We then push the backtracking point into the stack as a path node and move on toward 8. In our diagram we immediately hit another dead end, as shown in Figure 3-18(d). After backtracking to 9, we continue until we get to the dead end at node 11. See Figure 3-18(e). Once again we backtrack, this time to node 12. Following the path to 13, we push a decision node into the stack (B17) and continue on until we find our goal, at node 16.

We now know the path to our goal, we only need to print it out. The print loop simply pops the stack and prints the path. We ignore any backtracking points left in the stack, such as B17 in Figure 3-18(f).

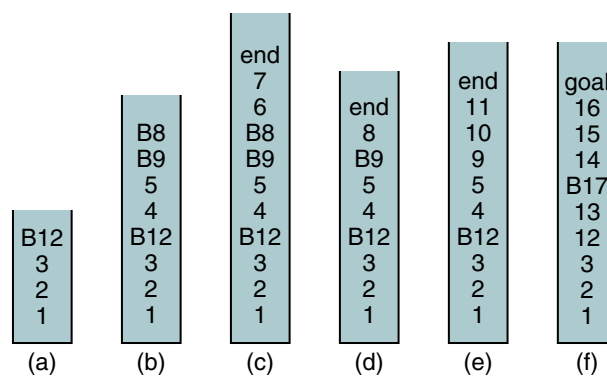


FIGURE 3-18 Backtracking Stack Operation

We present the solution in algorithmic form. Until we study the creation and storage of graphs in Chapter 12, we cannot develop the C code. The pseudocode solution is found in Algorithm 3-12.

ALGORITHM 3-12 Print Path to Goal

```
Algorithm seekGoal (map)
This algorithm determines the path to a desired goal.
   Pre  a graph containing the path
   Post path printed
 1 createStack (stack)
 2 set pMap to starting point
 3 loop (pMap not null AND goalNotFound)
   1   if (pMap is goal)
       1   set goalNotFound to false
   2   else
       1   pushStack (stack, pMap)
       2   if (pMap is a branch point)
           1   loop (more branch points)
               1   create branchPoint node
               2   pushStack (stack, branchPoint)
           2   end loop
       3   end if
       4   advance to next node
   3   end if
 4 end loop
 5 if (emptyStack (stack))
   1   print (There is no path to your goal)
 6 else
   1   print (The path to your goal is:)
   2   loop (not emptyStack (stack))
       1   popStack (stack, pMap)
       2   if (pMap not branchPoint)
           1   print(map point)
       3   end if
   3   end loop
   4   print (End of Path)
 7 end if
 8 return
end seekGoal
```

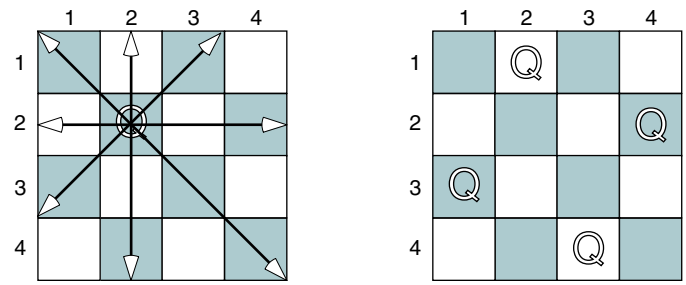
Algorithm 3-12 Analysis

This algorithm demonstrates one of the values of pseudocode and why it is popular when discussing algorithms with users. You will not have enough data structure knowledge until Chapter 11 (graphs) to solve this problem on a computer. Yet even though we don't have a data structure to solve it, we can still develop and understand the algorithm.

Eight Queens Problem

A classic chess problem requires that you place eight queens on the chessboard in such a way that no queen can capture another queen. There are actually several different solutions to this problem. The computer solution to the problem requires that we place a queen on the board and then analyze all of the attack positions to see if there is a queen that could capture the new queen. If there is, then we try another position.[5]

To demonstrate a sample solution, let's analyze how we would place four queens on a 4 × 4 chessboard. The queen's capture rules and one solution are shown in Figure 3-19.



(a) Queen capture rules (b) First four queens solution

FIGURE 3-19 Four Queens Solution

We can solve this problem using a stack and backtracking logic. Because only one queen can be placed in any row, we begin by placing the first queen in row 1, column 1. This location is then pushed into a stack, giving the position shown in Figure 3-20, step 1.

After placing a queen in the first row, we look for a position in the second row. Position 2,1 is not possible because the queen in the first row is guarding this location on the vertical. Likewise, position 2,2 is guarded on the diagonal. We therefore place a queen in the third column in row 2 and push this location into the stack. This position is shown in Figure 3-20, step 2.

We now try to locate a position in row 3, but none are possible. The first column is guarded by the queen in row 1, and the other three positions are guarded by the queen in row 2. At this point we must backtrack to the second row by popping the stack and continue looking for a position for the second-row queen. Because column 4 is not guarded, we place a queen there and push its location into the stack (step 3 in Figure 3-20).

5. A queen can attack another queen if it is in the same row, in the same column, or in the same diagonal.

PROGRAM 3-24 Eight Queens: Print Board *(continued)*

```
47      destroyStack(pOutStack);
48      return;
49  }   // printBoard
```

```
Results:
Welcome to Eight Queens. You may select
a board size from 4 x 4 to 8 x 8. I will
then position a queen in each row of the
board so no queen may capture another
queen. Note: There are no solutions for
boards less than 4 x 4.

Please enter the board size: 4

Place queens in following positions:
Row 1-Col 2:     | Q |   |   |
Row 2-Col 4:     |   |   | Q |
Row 3-Col 1:   Q |   |   |   |
Row 4-Col 3:     |   | Q |   |

We hope you enjoyed Eight Queens.
```

3.6 How Recursion Works

In Chapter 2 we discussed recursion but did not explain how it physically works. Now that we've discussed stacks, we can discuss the physical operation of recursion.

To understand how recursion works, we need to first explore how any call works. When a program calls a subroutine—for example, a function in C—the current module suspends processing and the called subroutine takes over control of the program. When the subroutine completes its processing and returns to the module that called it, the module wakes up and continues its processing. One important point in this interaction is that, unless changed through call by reference, all local data in the calling module are unchanged. Every local variable must be in the same state when processing resumes as it was when processing suspended. Similarly, the parameter list must not be changed. The value of the parameters must be the same before and after a call, unless they are reference parameters.

Let's look at an example of a simple call and see how it works. Figure 3-22 contains an algorithm called `testPower` that prints the value of a number `x` raised to a power `y`. To illustrate the function call, we use a separate algorithm, `power`, to determine the actual value of x^y.

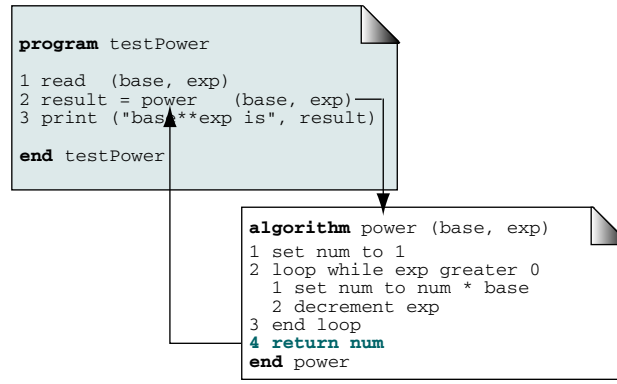


FIGURE 3-22 Call and Return

Our little program contains three local variables: the base number, `base`; the exponent, `exp`; and the answer, `result`. The base and exponent values must not be changed by the call to `power`.[7] When `power` terminates, it returns the value of x^y, which is stored in the local variable, `result`.

When `power` begins to execute, it must know the values of the parameters so that it can process them. It must also know where it needs to return when its processing is done. Finally, because `power` returns a value, it must know where the value is to be placed when it terminates. The physical implementation is determined by the compiler writer, but these data are conceptually placed in a **stackframe**. When `power` is called, the stackframe is created and pushed into a system stack. When it concludes, the stackframe is popped, the local variables are replaced, the return value is stored, and processing resumes in the calling algorithm.

A stackframe contains four different elements:

1. The parameters to be processed by the called algorithm
2. The local variables in the calling algorithm
3. The return statement in the calling algorithm
4. The expression that is to receive the return value (if any)

Now that you understand how a call works, you are ready to see how recursion works. Algorithm 3-14 contains a recursive version of the power algorithm discussed earlier.

7. In this discussion, we assume a recursive compiler. Languages such as COBOL and FORTRAN operate under a different, nonrecursive design in which there are no local variables.

ALGORITHM 3-14 Recursive Power Algorithm

```
Algorithm power (base, exp)
This algorithm computes the value of a number, base, raised
to the power of an exponent, exp.
   Pre    base is the number to be raised
          exp is the exponent
   Post   value of base raised to power exp computed
   Return value of base raised to power exp returned
 1 if (exp is 0)
   1  return (1)
 2 else
   1  return (base * power (base, exp - 1))
 3 end if
end power
```

In Figure 2-4, we traced the calls for the recursive factorial algorithm. This example is logically correct, but it oversimplifies the calls. Figure 3-23 traces the execution of Algorithm 3-14 in its recursive version and shows the contents of the system stack for each call.

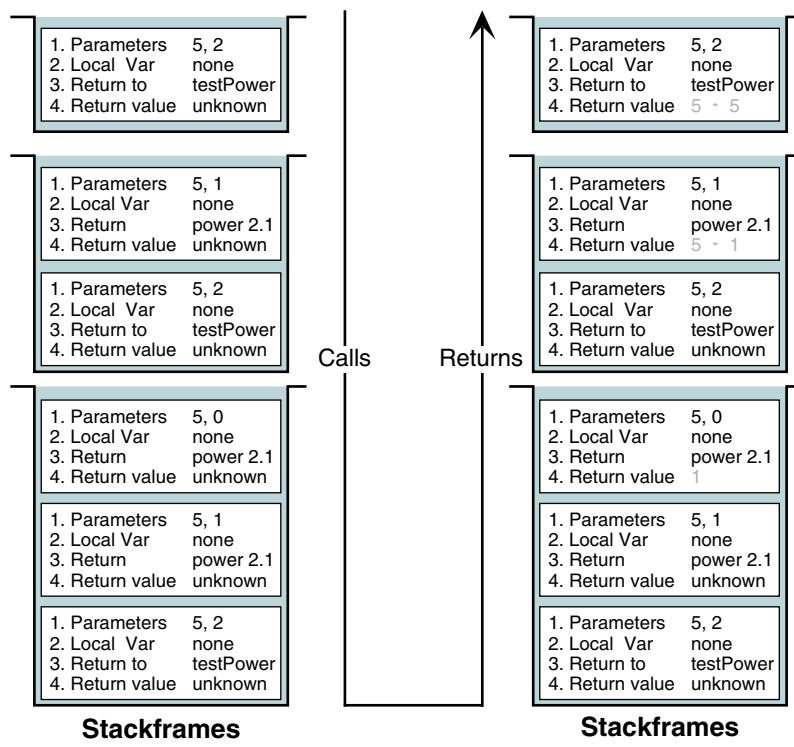


FIGURE 3-23 Stackframes for Power (5, 2)

Referring to Figure 3-23, `power` is called initially by `testPower` with the base and exponent set to 5 and 2, respectively. The first stackframe was created by the original call and contains a return location in the calling algorithm, `testPower`. In the second call, the base is unchanged but the exponent becomes 1. This call creates the second stackframe in Figure 3-23. Note that the parameters are changed and the return address is now `power` 2.1. In the third iteration, the exponent becomes 0. This call is shown in the third stackframe.

At this point we have reached the base case and begin backing out of the recursion. As each return statement is executed, we return to statement 2.1 in `power`, first with the value of 1 (the base case), then with a value of 5 (5×1), and finally with a value of 25 (5×5). When we return the third time, we return to the original calling algorithm, `testPower`.

3.7 Key Terms

backtracking
create stack
data node
destroy stack
empty stack
full stack
head
last in–first out (LIFO)
metadata
overflow
parsing
pop
pop stack
push
push stack
self-referential data structure
stack
stack count
stackframe
stack top
top
underflow

3.8 Summary

- A stack is a linear list in which all additions are restricted to one end, called the top. A stack is also called a LIFO list.
- We have defined eight operations for a stack: create stack, push stack, pop stack, stack top, empty stack, full stack, stack count, and destroy stack.
- Create stack initializes the stack metadata.
- Push stack adds an item to the top of the stack. After the push, the new item becomes the top.
- Each push must ensure that there is room for the new item. If there is no room, the stack is in an overflow state.
- Pop stack removes the item at the top of the stack. After the pop the next item, if any, becomes the top. Each pop must ensure that there is at least one item in the stack. If there is not at least one item, the stack is in an underflow state.
- The stack top operation only retrieves the item at the top of the stack. Each top execution must ensure that there is at least one item in the stack. If there is not at least one item, the stack is in an underflow state.
- Empty stack determines whether the stack is empty and returns a Boolean true if it is.
- Full stack determines whether there is room for at least one more item and returns a Boolean false if there is.
- Stack count returns the number of elements currently in the stack.

Chapter 4
Queues

A queue is a linear list in which data can only be inserted at one end, called the **rear**, and deleted from the other end, called the **front**. These restrictions ensure that the data are processed through the queue in the order in which they are received. In other words, a queue is a **first in–first out (FIFO)** structure.

A queue is the same as a line. In fact, if you were in England, you would not get into a line, you would get into a queue. A line of people waiting for the bus at a bus station is a queue, a list of calls put on hold to be answered by a telephone operator is a queue, and a list of waiting jobs to be processed by a computer is a queue.

Figure 4-1 shows two representations of a queue: one a queue of people and the other a computer queue. Both people and data enter the queue at the rear and progress through the queue until they arrive at the front. Once they are at the front of the queue, they leave the queue and are served.

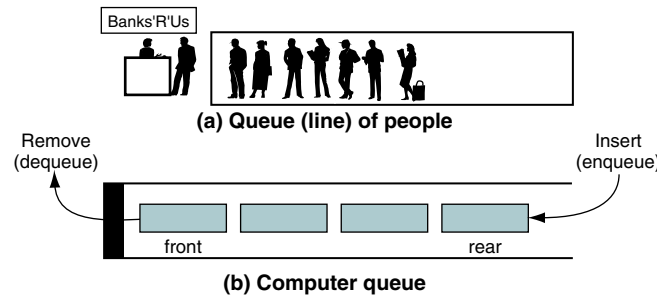


FIGURE 4-1 Queue Concept

4.1 Queue Operations

There are four basic queue operations. Data can be inserted at the rear, deleted from the front, retrieved from the front, and retrieved from the rear. Although there are many similarities between stacks and queues, one significant structural difference is that the queue implementation needs to keep track of the front and the rear of the queue, whereas the stack only needs to worry about one end: the top.

A queue is a linear list in which data can be inserted at one end, called the rear, and deleted from the other end, called the front. It is a first in–first out (FIFO) restricted data structure.

Enqueue

The queue insert operation is known as **enqueue**. After the data have been inserted into the queue, the new element becomes the rear. As we saw with stacks, the only potential problem with enqueue is running out of room for the data. If there is not enough room for another element in the queue, the queue is in an overflow state.

Enqueue inserts an element at the rear of the queue.

Figure 4-2 shows the enqueue operation.

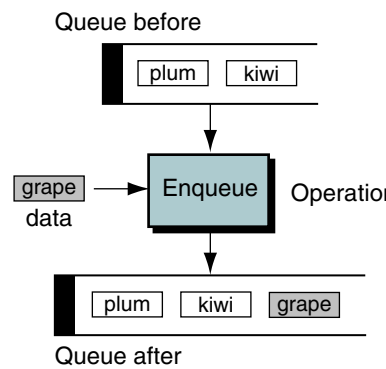


FIGURE 4-2 Enqueue

Dequeue

The queue delete operation is known as **dequeue**. The data at the front of the queue are returned to the user and removed from the queue. If there are no data in the queue when a dequeue is attempted, the queue is in an underflow state.

Dequeue deletes an element at the front of the queue.

The dequeue operation is shown in Figure 4-3.

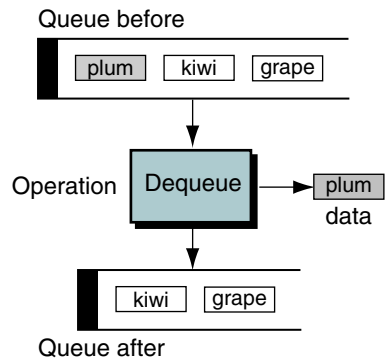


FIGURE 4-3 Dequeue

Queue Front

Data at the front of the queue can be retrieved with **queue front**. It returns the data at the front of the queue without changing the contents of the queue. If there are no data in the queue when a queue front is attempted, then the queue is in an underflow state.

Queue front retrieves the element at the front of the queue.

The queue front operation is shown in Figure 4-4.

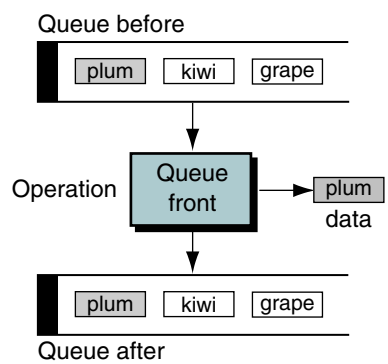


FIGURE 4-4 Queue Front

Queue Rear

A parallel operation to queue front retrieves the data at the rear of the queue. It is known as **queue rear.** As with queue front, if there are no data in the queue when a queue rear is attempted, the queue is in an underflow state.

Queue rear retrieves the element at the rear of the queue.

The queue rear operation is shown in Figure 4-5.

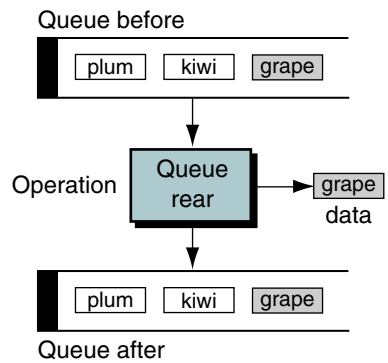


FIGURE 4-5 Queue Rear

Queue Example

Figure 4-6 traces these four operations in an example. We start with an empty queue and enqueue `green` and `blue`. At this point the queue contains two entries. We then dequeue, which removes the entry at the front of the queue, leaving `blue` as the only entry. After enqueuing `red` a queue front operation returns `blue` to the caller but leaves it at the front of the queue. The next operation, queue rear, returns `red` but leaves it in the queue. A dequeue then removes `blue`, leaving `red` as the only entry in the queue. Finally, we dequeue `red`, which results in an empty queue.

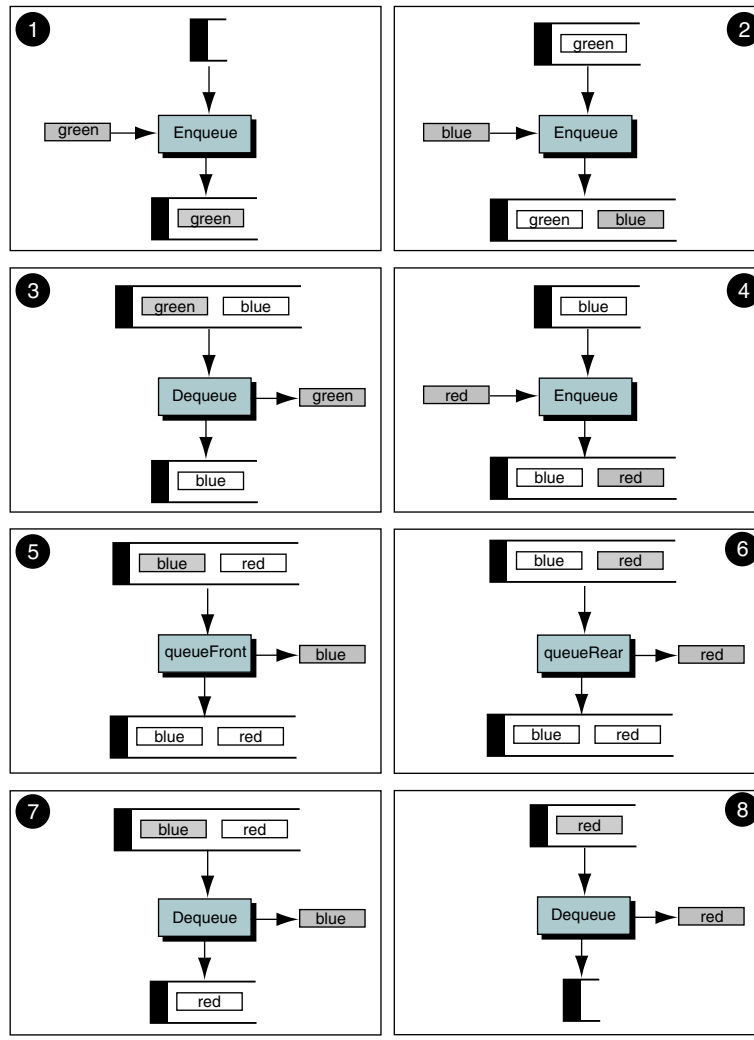


FIGURE 4-6 Queue Example

4.2 Queue Linked List Design

As with a stack, we implement our queue using a linked list. As we have emphasized, the actual implementation may be different. For example, in Appendix F we provide an array implementation for the queue ADT.

Data Structure

We need two different structures to implement the queue: a queue head structure and a data node structure. After it is created, the queue will have one head node and zero or more data nodes, depending on its current state. Figure 4-7 shows the conceptual and physical implementations for our queue structure.

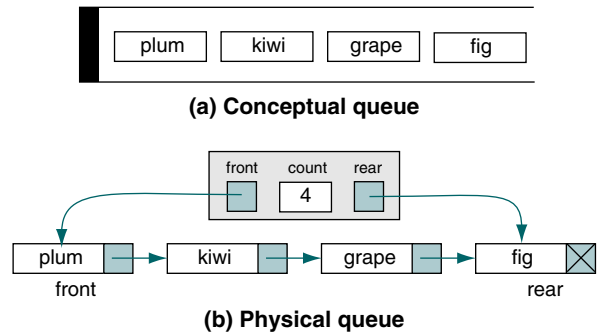


FIGURE 4-7 Conceptual and Physical Queue Implementations

Queue Head

The queue requires two pointers and a count. These fields are stored in the **queue head** structure. Other queue attributes, such as the maximum number of items that were ever present in the queue and the total number of items that have been processed through the queue, can be stored in the head node if such data are relevant to an application. The queue head structure is shown in Figure 4-8.

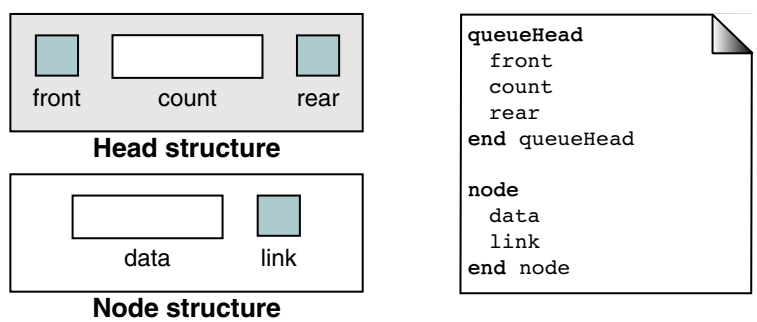


FIGURE 4-8 Queue Data Structure

Queue Data Node

The **queue data node** contains the user data and a link field pointing to the next node, if any. These nodes are stored in dynamic memory and are inserted and deleted as requested by the using program. Its structure is also shown in Figure 4-8.

Queue Algorithms

The queue operations parallel those for a stack, with the addition of an algorithm to look at the data at the rear of the queue. We define these operations here and in the sections that follow. The four basic queue operations are shown in Figure 4-9.

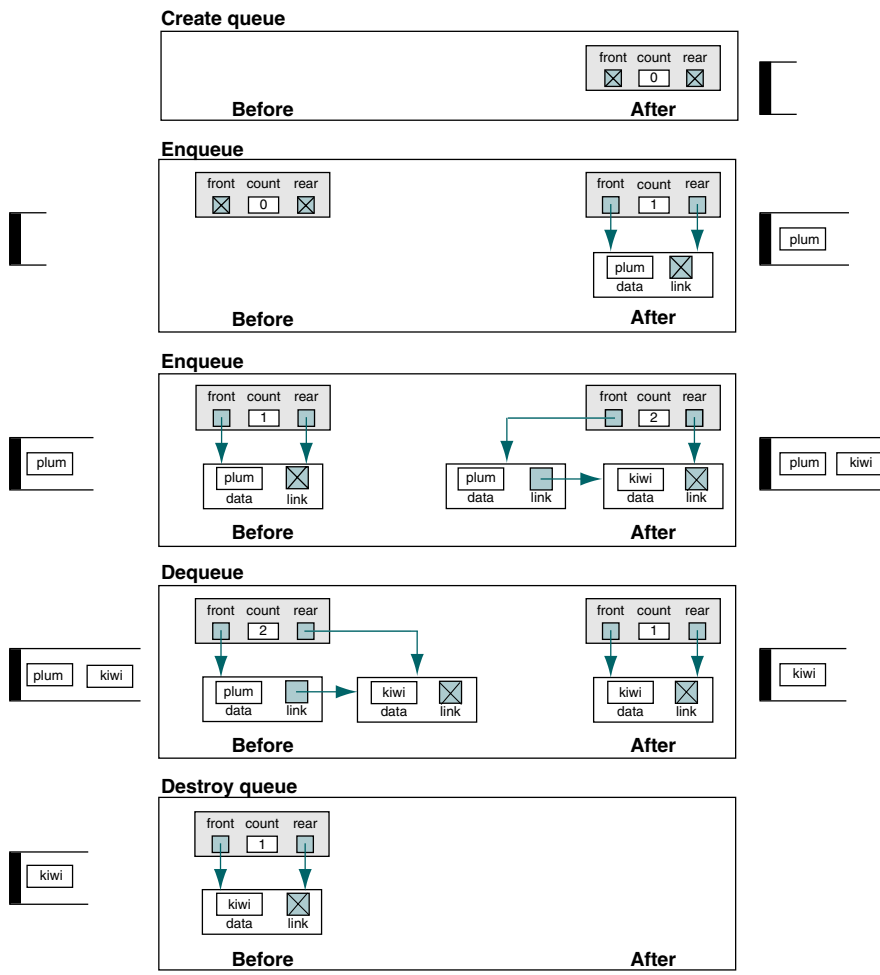


FIGURE 4-9 Basic Queue Functions

Create Queue

The **create queue** operation is rather simple. All we have to do is set the metadata pointers to null and the count to 0. The pseudocode for create queue is shown in Algorithm 4-1.

ALGORITHM 4-1 Create Queue

```
Algorithm createQueue
Creates and initializes queue structure.
  Pre    queue is a metadata structure
  Post   metadata elements have been initialized
  Return queue head
 1 allocate queue head
 2 set queue front to null
 3 set queue rear  to null
 4 set queue count to 0
 5 return queue head
end createQueue
```

Enqueue

The enqueue is a little more complex than inserting data into a stack. To develop the insertion algorithm, we need to analyze two different queue conditions: insertion into an empty queue and insertion into a queue with data. These operations are shown in Figure 4-10.

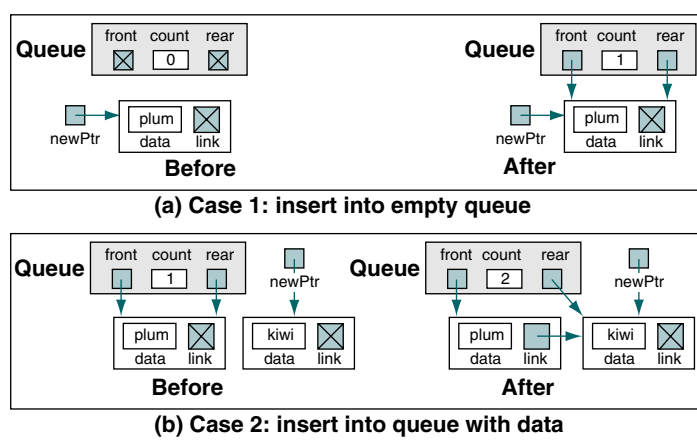


FIGURE 4-10 Enqueue Example

When we insert data into an empty queue, the queue's front and rear pointers must both be set to point to the new node. When we insert data into a queue with data already in it, we must point both the link field in the last

node and the rear pointer to the new node. If the insert was successful, we return a Boolean true; if there is no memory left for the new node, we return a Boolean false. The pseudocode is shown in Algorithm 4-2.

ALGORITHM 4-2 Insert Data into Queue

```
Algorithm enqueue (queue, dataIn)
This algorithm inserts data into a queue.
   Pre    queue is a metadata structure
   Post   dataIn has been inserted
   Return true if successful, false if overflow
 1 if (queue full)
   1  return false
 2 end if
 3 allocate (new node)
 4 move dataIn to new node data
 5 set new node next to null pointer
 6 if (empty queue)
   Inserting into null queue
   1  set queue front to address of new data
 7 else
   Point old rear to new node
   1  set next pointer of rear node to address of new node
 8 end if
 9 set queue rear to address of new node
10 increment queue count
11 return true
end enqueue
```

Dequeue

Although dequeue is also a little more complex than deleting data from a stack, it starts out much the same. We must first ensure that the queue contains data. If the queue is empty, we have underflow and we return false, indicating that the dequeue was not successful.

Given that there are data to be dequeued, we pass the data back through the parameter list and then set the front pointer to the next item in the queue. If we have just dequeued the last item, the queue front pointer automatically becomes a null pointer by assigning it the null pointer from the link field of the last node. However, if the queue is now empty, we must also set the rear pointer to null. These cases are shown in Figure 4-11.

The pseudocode is shown in Algorithm 4-3.

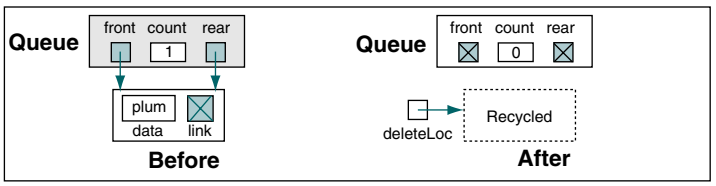
ALGORITHM 4-3 Delete Data from Queue

```
Algorithm dequeue (queue, item)
This algorithm deletes a node from a queue.
  Pre    queue is a metadata structure
```

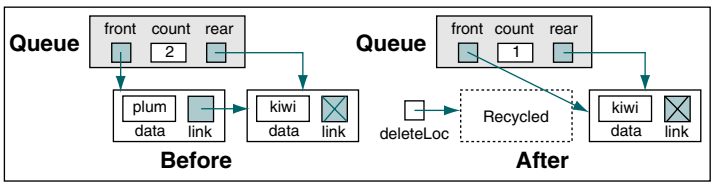
continued

ALGORITHM 4-3 Delete Data from Queue *(continued)*

```
        item is a reference to calling algorithm variable
  Post   data at queue front returned to user through item
         and front element deleted
  Return true if successful, false if underflow
 1 if (queue empty)
   1  return false
 2 end if
 3 move front data to item
 4 if (only 1 node in queue)
   Deleting only item in queue
   1  set queue rear to null
 5 end if
 6 set queue front to queue front next
 7 decrement queue count
 8 return true
end dequeue
```



(a) Case 1: delete only item in queue



(b) Case 2: delete item at front of queue

FIGURE 4-11 Dequeue Examples

Retrieving Queue Data

The only difference between the two retrieve queue operations—queue front and queue rear—is which pointer is used, front or rear. Let's look at queue front. Its logic is identical to that of dequeue except that the data are not deleted from the queue. It first checks for an empty queue and returns false if the queue is empty. If there are data in the queue, it passes the data back through `dataOut` and returns true. The pseudocode is shown in Algorithm 4-4.

ALGORITHM 4-4 Retrieve Data at Front of Queue

```
Algorithm queueFront (queue, dataOut)
Retrieves data at the front of the queue without changing
queue contents.
  Pre    queue is a metadata structure
         dataOut is a reference to calling algorithm variable
  Post   data passed back to caller
  Return true if successful, false if underflow
1 if (queue empty)
  1  return false
2 end if
3 move data at front of queue to dataOut
4 return true
end queueFront
```

To implement queue rear, copy Algorithm 4-4 and move the data at the rear in statement 3.

Empty Queue

Empty queue returns true if the queue is empty and false if the queue contains data. There are several ways to test for an empty queue. Checking the queue count is the easiest. The pseudocode is shown in Algorithm 4-5.

ALGORITHM 4-5 Queue Empty

```
Algorithm emptyQueue (queue)
This algorithm checks to see if a queue is empty.
  Pre    queue is a metadata structure
  Return true if empty, false if queue has data
1 if (queue count equal 0)
  1  return true
2 else
  1  return false
end emptyQueue
```

Full Queue

Like the stack, **full queue** is another structured programming implementation of data hiding. Depending on the language, it may be one of the most difficult algorithms to implement—ANSI C provides no direct way to implement it. The pseudocode is shown in Algorithm 4-6.

ALGORITHM 4-6 Full Queue

```
Algorithm fullQueue (queue)
This algorithm checks to see if a queue is full. The queue
```

continued

ALGORITHM 4-6 Full Queue *(continued)*

```
is full if memory cannot be allocated for another node.
  Pre    queue is a metadata structure
  Return true if full, false if room for another node
1 if (memory not available)
  1  return true
2 else
  1  return false
3 end if
end fullQueue
```

Queue Count

Queue count returns the number of elements currently in the queue by simply returning the count found in the queue head node. The code is shown in Algorithm 4-7.

ALGORITHM 4-7 Queue Count

```
Algorithm queueCount (queue)
This algorithm returns the number of elements in the queue.
  Pre    queue is a metadata structure
  Return queue count
1 return queue count
end queueCount
```

Destroy Queue

Destroy queue deletes all data in the queue. Algorithm 4-8 contains the code to destroy a queue.

ALGORITHM 4-8 Destroy Queue

```
Algorithm destroyQueue (queue)
This algorithm deletes all data from a queue.
  Pre     queue is a metadata structure
  Post    all data have been deleted
1 if (queue not empty)
  1  loop (queue not empty)
     1  delete front node
  2  end loop
2 end if
3 delete head structure
end destroyQueue
```

4.3 Queue ADT

The queue abstract data type (ADT) follows the basic design of the stack abstract data type. We begin by developing the queue data structure and then we develop each of the queue algorithms in Section 4.2.

Queue Structure

The **queue data structure** is shown in Program 4-1. The node structure is identical to the structure we used for a stack. Each node contains a *void* pointer to the data and a link pointer to the next element in the queue. As with the stack, the using program is responsible for allocating memory to store the data.

Program 4-1 also contains the prototype statements for the queue functions. In a production environment, it would be a queue header file, such as `queues.h`.

PROGRAM 4-1 Queue ADT Data Structures

```
1  //Queue ADT Type Defintions
2    typedef struct node
3      {
4       void*        dataPtr;
5       struct node* next;
6      } QUEUE_NODE;
7    typedef struct
8      {
9       QUEUE_NODE* front;
10      QUEUE_NODE* rear;
11      int         count;
12     } QUEUE;
13
14 //Prototype Declarations
15   QUEUE* createQueue  (void);
16   QUEUE* destroyQueue (QUEUE* queue);
17
18   bool  dequeue    (QUEUE* queue, void** itemPtr);
19   bool  enqueue    (QUEUE* queue, void*  itemPtr);
20   bool  queueFront (QUEUE* queue, void** itemPtr);
21   bool  queueRear  (QUEUE* queue, void** itemPtr);
22   int   queueCount (QUEUE* queue);
23
24   bool  emptyQueue (QUEUE* queue);
25   bool  fullQueue  (QUEUE* queue);
26 //End of Queue ADT Definitions
```

Queue ADT Algorithms

This section contains the C code for each of the queue algorithms. As you study them, trace them through Figure 4-6, "Queue Example."

Create Queue

Create queue allocates a node for the queue header. It then initializes the front and rear pointers to null and sets the count to zero. If overflow occurs, the return value is null; if the allocation is successful, it returns the address of the queue head. The pseudocode design is shown in Algorithm 4-1. The code for create queue is developed in Program 4-2.

PROGRAM 4-2 Create Queue

```
/*================= createQueue ================
  Allocates memory for a queue head node from dynamic
  memory and returns its address to the caller.
     Pre    nothing
     Post   head has been allocated and initialized
     Return head if successful; null if overflow
*/
QUEUE* createQueue (void)
{
//Local Definitions
  QUEUE* queue;

//Statements
  queue = (QUEUE*) malloc (sizeof (QUEUE));
  if (queue)
     {
      queue->front  = NULL;
      queue->rear   = NULL;
      queue->count  = 0;
     } // if
  return queue;
} // createQueue
```

Enqueue

The logic for enqueue, as shown in Algorithm 4-2, is straightforward. If memory is available, it creates a new node, inserts it at the rear of the queue, and returns true. If memory is not available, it returns false. The code is shown in Program 4-3.

PROGRAM 4-3 Enqueue

```
/*================= enqueue ================
  This algorithm inserts data into a queue.
     Pre    queue has been created
     Post   data have been inserted
     Return true if successful, false if overflow
*/
```

continued

PROGRAM 4-3 Enqueue *(continued)*

```
 7  bool enqueue (QUEUE* queue, void* itemPtr)
 8  {
 9  //Local Definitions
10    QUEUE_NODE* newPtr;
11
12  //Statements
13    if (!(newPtr =
14       (QUEUE_NODE*)malloc(sizeof(QUEUE_NODE))))
15       return false;
16
17    newPtr->dataPtr = itemPtr;
18    newPtr->next    = NULL;
19
20    if (queue->count == 0)
21       // Inserting into null queue
22       queue->front  = newPtr;
23    else
24       queue->rear->next = newPtr;
25
26    (queue->count)++;
27    queue->rear = newPtr;
28    return true;
29  } // enqueue
```

Program 4-3 Analysis

Because we must maintain both a front and a rear pointer, we need to check to see if we are inserting into a null queue. If we are, we must set both pointers to the data just inserted. If there are already data in the queue, we need to set the next field of the node at the rear of the queue and the rear pointer to the new node. In this case the front pointer is unchanged. Because the rear pointer is updated in either case, we changed it after the *if* statement (see statement 27).

Dequeue

Dequeue follows the basic design developed in Algorithm 4-3. It begins by checking to make sure there are data in the queue. If there are, it passes the address of the data being deleted back to the calling function through a reference parameter, adjusts the pointers, and subtracts one from the queue count. If the delete is successful, it returns true; if it is empty, it returns false. The code is shown in Program 4-4.

PROGRAM 4-4 Dequeue

```
 1  /*================= dequeue ================
 2    This algorithm deletes a node from the queue.
 3       Pre    queue has been created
 4       Post   Data pointer to queue front returned and
```

continued

PROGRAM 4-4 Dequeue *(continued)*

```
 5                front element deleted and recycled.
 6            Return true if successful; false if underflow
 7   */
 8   bool dequeue (QUEUE* queue, void** itemPtr)
 9   {
10   //Local Definitions
11     QUEUE_NODE* deleteLoc;
12
13   //Statements
14     if (!queue->count)
15         return false;
16
17     *itemPtr  = queue->front->dataPtr;
18     deleteLoc = queue->front;
19     if (queue->count == 1)
20        // Deleting only item in queue
21        queue->rear  = queue->front = NULL;
22     else
23        queue->front = queue->front->next;
24     (queue->count)--;
25     free (deleteLoc);
26
27     return true;
28   } // dequeue
```

Program 4-4 Analysis

The logic for dequeue is rather basic. The major concern is that we may have deleted the last element in the queue, in which case we must set it to a null state. This requires that the queue rear be set to a null pointer. Note that we always set the queue front to the next pointer in the node being deleted. If the last node is being deleted, the deleted node's next pointer is guaranteed to be null (because it is the last node in the queue). In this case queue front is automatically set to null. If it is not the last node in the queue, queue front points to the new node at the front of the queue.

Although the code for dequeue is simple enough, the call to it is a little tricky. Because the dequeue data parameter is a *void* pointer to a pointer, the call must cast the data pointer to *void*. The call is shown below.

```
dequeue(queue, (void*)&dataPtr)
```

Because dequeue returns a Boolean success flag, the call should be an expression in a selection statement to test for success or failure.

```
if (dequeue(queue, (void*)&dataPtr))
```

Queue Front

Queue front also passes the address of the data at the front of the queue back to the calling function. It differs from dequeue only in that the queue is not changed. The code is shown in Program 4-5; the pseudocode design is shown in Algorithm 4-4.

PROGRAM 4-5 Queue Front

```
/*================== queueFront =================
  This algorithm retrieves data at front of the
  queue without changing the queue contents.
     Pre    queue is pointer to an initialized queue
     Post   itemPtr passed back to caller
     Return true if successful; false if underflow
*/
bool queueFront (QUEUE* queue, void** itemPtr)
{
//Statements
  if (!queue->count)
      return false;
  else
     {
      *itemPtr = queue->front->dataPtr;
       return true;
     } // else
} // queueFront
```

Queue Rear

The code for queue rear, shown in Program 4-6, is identical to queue front except that the address of the data at the rear of the queue is sent back to the calling function.

PROGRAM 4-6 Queue Rear

```
/*================== queueRear =================
  Retrieves data at the rear of the queue
  without changing the queue contents.
     Pre    queue is pointer to initialized queue
     Post   Data passed back to caller
     Return true if successful; false if underflow
*/
bool queueRear (QUEUE* queue, void** itemPtr)
{
//Statements
  if (!queue->count)
      return true;
```

continued

PROGRAM 4-6 Queue Rear *(continued)*

```
13    else
14       {
15        *itemPtr = queue->rear->dataPtr;
16        return false;
17       } // else
18  } // queueRear
```

Queue Empty

Queue empty is one of the simplest functions in the library. It simply uses the count in the queue header to determine if the queue is empty or contains data, as shown in Program 4-7.

PROGRAM 4-7 Empty Queue

```
1   /*================== emptyQueue =================
2     This algorithm checks to see if queue is empty.
3     Pre    queue is a pointer to a queue head node
4     Return true if empty; false if queue has data
5   */
6   bool emptyQueue (QUEUE* queue)
7   {
8   //Statements
9     return (queue->count == 0);
10  } // emptyQueue
```

Full Queue

As mentioned in the discussion of Algorithm 4-6, full queue is one of the more difficult algorithms to implement because C provides no way to test the amount of space left in dynamic memory. The only way we can tell if there is any room left is to allocate a node and test for success. If the node was allocated successfully, we free it and return false for not full. If there was no memory left, we return true—the queue is full. The code for full queue is shown in Program 4-8.

PROGRAM 4-8 Full Queue

```
1   /*================== fullQueue =================
2     This algorithm checks to see if queue is full. It
3     is full if memory cannot be allocated for next node.
4        Pre    queue is a pointer to a queue head node
5        Return true if full; false if room for a node
6   */
7   bool fullQueue (QUEUE* queue)
8   {
```

continued

PROGRAM 4-8 Full Queue *(continued)*

```
 9  //Local Definitions
10  QUEUE_NODE* temp;
11
12  //Statements
13    temp = (QUEUE_NODE*)malloc(sizeof(*(queue->rear)));
14    if (temp)
15       {
16        free (temp);
17        return true;
18       } // if
19    // Heap full
20    return false;
21  } // fullQueue
```

Queue Count

Queue count simply returns the count found in the queue head node. Its code is shown in Program 4-9.

PROGRAM 4-9 Queue Count

```
 1  /*================== queueCount =================
 2    Returns the number of elements in the queue.
 3       Pre    queue is pointer to the queue head node
 4       Return queue count
 5  */
 6  int queueCount(QUEUE* queue)
 7  {
 8  //Statements
 9    return queue->count;
10  } // queueCount
```

Destroy Queue

The design of **destroy queue** in Algorithm 4-8 is relatively simple; the implementation in Program 4-10 is relatively complex. While it is apparent that we must delete all elements in the queue and free each of their nodes, it is easy to forget to recycle the data space. We begin by checking to make sure that the queue exists. If it doesn't, we simply return a null pointer. If the queue is valid, we cycle through it, deleting all of the elements.

PROGRAM 4-10 Destroy Queue

```
 1  /*================== destroyQueue =================
 2    Deletes all data from a queue and recycles its
 3    memory, then deletes & recycles queue head pointer.
 4       Pre    Queue is a valid queue
```

continued

PROGRAM 4-10 Destroy Queue *(continued)*

```
 5        Post   All data have been deleted and recycled
 6        Return null pointer
 7  */
 8  QUEUE* destroyQueue (QUEUE* queue)
 9  {
10  //Local Definitions
11    QUEUE_NODE* deletePtr;
12
13  //Statements
14    if (queue)
15       {
16        while (queue->front != NULL)
17           {
18            free (queue->front->dataPtr);
19            deletePtr    = queue->front;
20          queue->front = queue->front->next;
21            free (deletePtr);
22           } // while
23        free (queue);
24       } // if
25    return NULL;
26  } // destroyQueue
```

Program 4-10 Analysis In the normal course of events, the queue being destroyed is empty. If it is, the queue front is null and all we need to do is free the queue header and return. If there are data in the queue, however, we first free the data space using the data pointer stored in the queue node, then we delete the queue node itself. Eventually, we get to the end of the queue, at which time we free the queue head node and return.

4.4 Queuing Theory

Queuing theory is a field of applied mathematics that is used to predict the performance of queues. In this section we review a few basic queuing theory concepts. We leave the mathematics of queuing theory to advanced courses on the subject.

> Queuing theory is a field of applied mathematics that is used to predict the performance of queues.

Queues can be divided into single-server and multiserver types. A **single-server queue** can provide service to only one customer at a time. An example of a single-server queue is the hot-food vendor found on street corners in most large cities. **Multiserver queues**, on the other hand, can provide service to many customers at a time. An example of a multiserver queue is a bank in which there is one line with many bank tellers providing service. Note that grocery stores are examples of multiple single-server queues, or *multiqueues*.

The two common elements to all queues are one or more customers who need a service and the a server who provides the service. A **customer** is any person or thing needing service. Besides the examples cited, customers can be as diverse as jobs waiting to be run in a computer and packages being sent by express delivery. The **service** is any activity needed to accomplish the required result. The hot-food vendor provides food as a service; the package handler provides transportation and delivery of packages.

The two factors that most dramatically affect the queue are the arrival rate and the service time. The rate at which customers arrive in the queue for service is known as the **arrival rate**. Depending on the service being provided, the arrival rate may be random or regular. The hot-food vendor's customers more than likely arrive randomly, although the rate may vary widely during the day. Jobs to be processed in the computer, however, may arrive at some regular rate created by a job-scheduling system.

Service time is the average time required to complete the processing of a customer request. If you have ever observed different bank customers while you stood in line, you noted the wide range in time required to process each customer. Obviously, the faster customers arrive and the higher the service time, the longer the queue.

In an ideal situation, customers would arrive at a rate that matches the service time. However, as we all know, things are seldom ideal. Sometimes the server will be idle because there are no customers to be served. At other times there will be many customers waiting to be served. If we can predict the patterns, we may be able to minimize idle servers and waiting customers. Doing so is especially important when customers who have to wait for a long time are apt to switch to an alternative server, such as the hot-food vendor a block down the street.

Queuing theory attempts to predict such patterns. Specifically, it attempts to predict *queue time* (which is defined as the average length of time customers wait in the queue), the average size of the queue, and the maximum queue size. To make these predictions, it needs two factors: the arrival rate and the average service time, which is defined as the average of the total service time between idle periods.

Given queue time and service time, we know *response time*,—a measure of the average time from the point at which customers enter the queue until the moment they leave the server. Response time is an important statistic, especially in online computer systems. A queuing theory model is shown in Figure 4-12.

Once a model of a queue has been built, it can be used to study proposed changes to the system. For example, in the banking queue, if we were able to use automation to reduce the average service time by 15%, how many fewer tellers would we need? Or, given a model of a growing system that is currently under capacity, how long will it be before we need to add another server?

The two factors that most affect the performance of queues are the arrival rate and the service time.

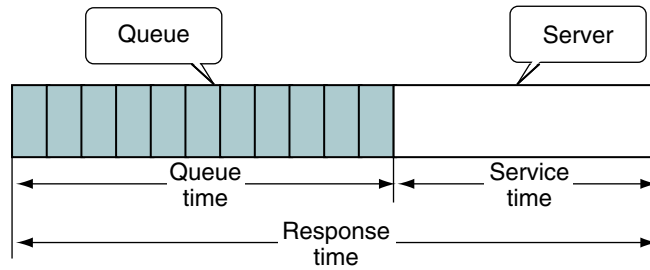


FIGURE 4-12 Queuing Theory Model

4.5 Queue Applications

Queues are one of the more common data-processing structures. They are found in virtually every operating system and every network and in countless other areas. For example, queues are used in business online applications such as processing customer requests, jobs, and orders. In a computer system, a queue is needed to process jobs and for system services such as print spools.

Queues can become quite complex. In this section we demonstrate two queue implementations: The first is an application that is useful in many problems—categorizing data. For this application we show both the design and the implementation. The second is a queue simulation that can be used to study the performance of any queue application. Only the design is developed; the implementation is left as a project at the end of the chapter.

Categorizing Data

It is often necessary to rearrange data without destroying their basic sequence. As a simple example, consider a list of numbers. We need to group the numbers while maintaining their original order in each group. This is an excellent multiple-queue application.

To demonstrate, consider the following list of numbers:

```
3 22 12 6 10 34 65 29 9 30 81 4 5 19 20 57 44 99
```

We want to categorize them into four different groups:

- Group 1: less than 10
- Group 2: between 10 and 19
- Group 3: between 20 and 29
- Group 4: 30 and greater

In other words, we want the list rearranged as shown below.

```
| 3 6 9 4 5 | 12 10 19 | 22 29 20| 34 65 30 81 57 44 99 |
```

This is not sorting. The result is not a sorted list but rather a list categorized according to the specified rules. The numbers in each group have kept their original order.

Categorizing Data Design

The solution is simple. We build a queue for each of the four categories. We then store the numbers in the appropriate queue as we read them. After all the data have been processed, we print each queue to demonstrate that we categorized the data correctly. This design is shown in Algorithm 4-9.

ALGORITHM 4-9 Category Queues

```
Algorithm categorize
  Group a list of numbers into four groups using four queues.
     Written by:
     Date:
1 createQueue (q0to9)
2 createQueue (q10to19)
3 createQueue (q20to29)
4 createQueue (qOver29)
5 fillQueues  (q0to9, q10to19, q20to29, qOver29)
6 printQueues (q0to9, q10to19, q20to29, qOver29)
end categorize
```

Algorithm 4-9 Analysis

The mainline for the category queue simply creates the queues and then calls algorithms to fill and print them. Note that when we complete, we do not destroy the queues. We leave this process for the operating system. If we had more work to do, however, we would delete them to release their memory.

The pseudocode to fill the queue is shown in Algorithm 4-10. We demonstrate the logic for printing the queues when we write the C implementation in the next section.

ALGORITHM 4-10 Fill Category Queues

```
Algorithm fillQueues (q0to9, q10to19, q20to29, qOver29)
This algorithm reads data from the keyboard and places them in
one of four queues.
  Pre  all four queues have been created
  Post queues filled with data
1 loop (not end of data)
```

continued

ALGORITHM 4-10 Fill Category Queues *(continued)*

```
  1  read (number)
  2  if (number < 10)
     1  enqueue (q0to9, number)
  3  elseif (number < 20)
     1  enqueue (q10to19, number)
  4  elseif (number < 30)
     1  enqueue (q20to29, number)
  5  else
     1  enqueue (qOver29, number)
  6  end if
 2 end loop
end fillQueues
```

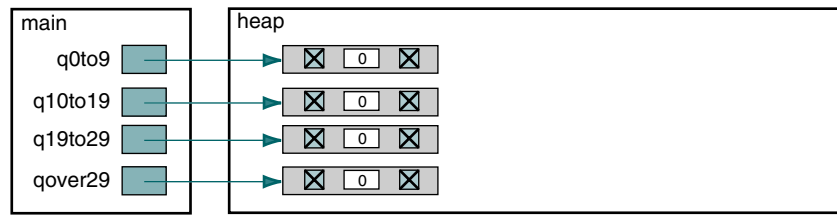
Algorithm 4-10 Analysis What sounded like a long and difficult program is really a short and simple algorithm, thanks to code reusability. The most difficult decision in this algorithm is determining how to write the categorizing code. We simply used a multiway selection implemented with an *elseif* construct. Once we made that decision, the algorithm practically wrote itself.

Categorizing Data—C Implementation

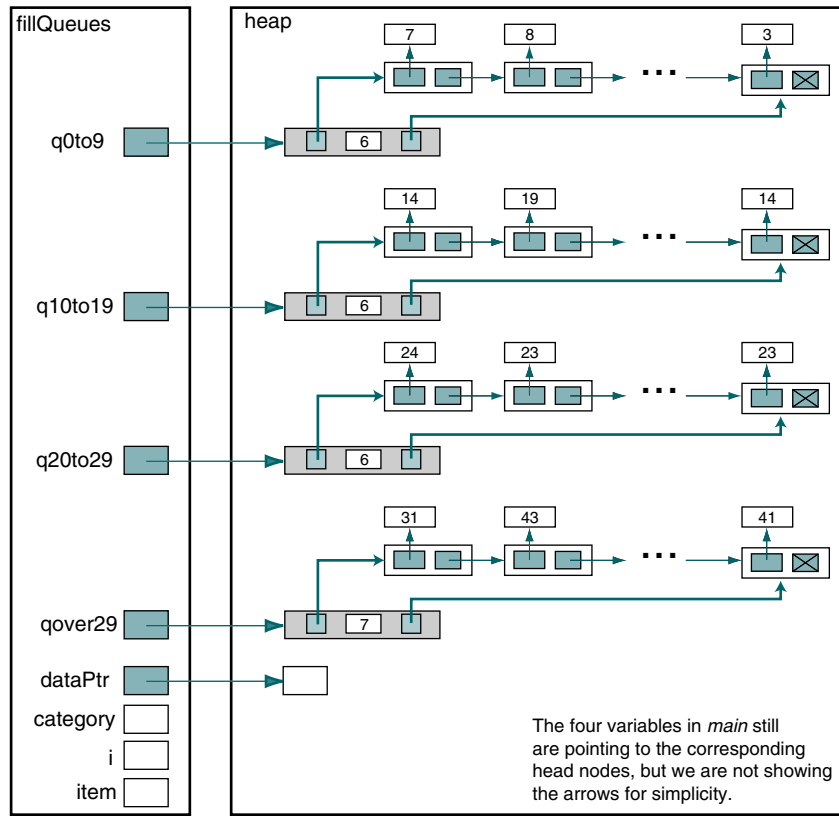
Rather than read the data from a file, however, we simply generate 25 random numbers in the range 0 to 50. The C functions use the abstract data type created in Section 4.3. To help you follow the program, we include Figure 4-13, which shows the structures after they have been created.

Mainline Logic

The mainline logic for the categorizing program begins by creating the queues. Once the queues have been created, it calls a function to create and enqueue the data and another function to print it. The code is shown in Program 4-11.



(a) Before calling fillQueues



(b) After calling fillQueues

FIGURE 4-13 Structures for Categorizing Data

PROGRAM 4-11 Categorizing Data Mainline

```
1  /*Groups numbers into four groups using four queues.
2       Written by:
3       Date:
```

continued

PROGRAM 4-11 Categorizing Data Mainline *(continued)*

```
 4  */
 5  #include <stdio.h>
 6  #include <stdlib.h>
 7  #include <stdbool.h>
 8  #include "queues.h"
 9
10  //Prototype Statements
11  void fillQueues  (QUEUE*, QUEUE*, QUEUE*, QUEUE*);
12  void printQueues (QUEUE*, QUEUE*, QUEUE*, QUEUE*);
13
14  void printOneQueue (QUEUE* pQueue);
15
16  int main (void)
17  {
18  //Local Definitions
19    QUEUE* q0to9;
20    QUEUE* q10to19;
21    QUEUE* q20to29;
22    QUEUE* qOver29;
23
24  //Statements
25    printf("Welcome to a demonstration of categorizing\n"
26           "data. We generate 25 random numbers and then\n"
27           "group them into categories using queues.\n\n");
28
29    q0to9   = createQueue ();
30    q10to19 = createQueue ();
31    q20to29 = createQueue ();
32    qOver29 = createQueue ();
33
34    fillQueues  (q0to9, q10to19, q20to29, qOver29);
35    printQueues (q0to9, q10to19, q20to29, qOver29);
36
37    return 0;
38  } // main
```

Fill Queues

Fill queues uses a random-number generator to create 25 random numbers between 0 and 50. As each number is generated, it is displayed so that we can verify the accuracy of the program, and then each number is inserted into the appropriate queue. The code is in Program 4-12.

PROGRAM 4-12 Categorizing Data: Fill Queues

```
/*================= fillQueues ================
  This function generates data using rand() and
  places them in one of four queues.
     Pre:  All four queues have been created
      Post: Queues filled with data
*/
void fillQueues (QUEUE* q0to9,   QUEUE* q10to19,
                 QUEUE* q20to29, QUEUE* qOver29)
{
//Local Definitions
  int  category;
  int  item;
  int* dataPtr;

//Statements
  printf("Categorizing data:\n");
  srand(79);

  for (int i = 1; i <= 25; i++)
      {
      if (!(dataPtr = (int*) malloc (sizeof (int))))
          printf("Overflow in fillQueues\a\n"),
                 exit(100);

       *dataPtr = item = rand() % 51;
       category = item / 10;
       printf("%3d", item);
       if (!(i % 11))
          // Start new line when line full
          printf("\n");

       switch (category)
          {
           case 0 : enqueue (q0to9, dataPtr);
                    break;
           case 1 : enqueue (q10to19, dataPtr);
                    break;
           case 2 : enqueue (q20to29, dataPtr);
                    break;
           default: enqueue (qOver29, dataPtr);
                    break;
          } // switch
      } // for
  printf("\nEnd of data categorization\n\n");
  return;
} // fillQueues
```

Print Queues

Print queues is a simple driver that calls a function to print one queue for each queue. Its code is shown in Program 4-13.

PROGRAM 4-13 Print Queues

```
/*================= printQueues =================
  This function prints the data in each of the queues.
     Pre  Queues have been filled
     Post Data printed and dequeued
*/
void printQueues (QUEUE* q0to9,   QUEUE* q10to19,
                  QUEUE* q20to29, QUEUE* qOver29)
{
//Statements
  printf("Data   0.. 9:");
  printOneQueue (q0to9);

  printf("Data  10..19:");
  printOneQueue (q10to19);

  printf("Data  20..29:");
  printOneQueue (q20to29);

  printf("Data over 29:");
  printOneQueue (qOver29);

  return;
} // printQueues
```

Print One Queue

Print one queue uses a straightforward approach. Each call prints a queue by dequeuing a node and printing it. The loop continues as long as there are data in the queue. Because we don't have access to the data structure, however, we use empty queue to test for more data to be printed. We could also have used queue count and tested for greater than zero. The code is shown in Program 4-14. It also contains the results from a sample run.

PROGRAM 4-14 Print One Queue

```
/*================= printOneQueue =================
  This function prints the data in one queue,
  ten entries to a line.
     Pre  Queue has been filled
     Post Data deleted and printed. Queue is empty
*/
void printOneQueue (QUEUE* pQueue)
```

continued

PROGRAM 4-14 Print One Queue *(continued)*

```
{
//Local Definitions
  int  lineCount;
  int* dataPtr;

//Statements
  lineCount = 0;
  while (!emptyQueue (pQueue))
      {
       dequeue (pQueue, (void*)&dataPtr);
       if (lineCount++ >= 10)
          {
           lineCount = 1;
           printf ("\n             ");
          } // if
       printf("%3d ", *dataPtr);
      } // while !emptyQueue
  printf("\n");

  return;
} // printOne Queue
```

```
Results:
Welcome to a demonstration of categorizing
data. We generate 25 random numbers and then
group them into categories using queues.

Categorizing data:
 24  7 31 23 26 14 19  8  9  6 43
 16 22  0 39 46 22 38 41 23 19 18
 14  3 41
End of data categorization

Data   0.. 9:  7   8   9   6   0   3
Data  10..19: 14  19  16  19  18  14
Data  20..29: 24  23  26  22  22  23
Data over 29: 31  43  39  46  38  41  41
```

Queue Simulation

An important application of queues is **queue simulation**, a modeling activity used to generate statistics about the performance of queues. Let's build a model of a single-server queue—for example, a saltwater taffy store on a beach boardwalk. The store has one window, and a clerk can service only one customer at a time. The store also ships boxes of taffy anywhere in the country. Because there are many flavors of saltwater taffy and because it takes longer to serve a

Chapter 5
General Linear Lists

A general linear list is a list in which operations, such as retrievals, insertions, changes, and deletions, can be done anywhere in the list, that is, at the beginning, in the middle, or at the end of the list. We use many different types of general lists in our daily lives. We use lists of employees, student lists, and lists of our favorite songs. When we process our song list, we need to be able to search for a song, add a new song, or delete one we gave away. This chapter describes how we can maintain and process general lists. For simplicity, from now on we refer to general linear lists as *lists*.

5.1 Basic Operations

The four basic list operations are insertion, deletion, retrieval, and traversal. Insertion is used to add a new element to the list. Deletion is used to remove an element from the list. Retrieval is used to get the information related to an element without changing the structure of the list. Traversal is used to traverse the list while applying a process to each element.

Insertion

List **insertion** can be ordered or random. **Ordered lists** are maintained in sequence according to the data or, when available, a key that identifies the data. A **key** is one or more fields within a structure that identifies the data. Examples of keys are Social Security number and the universal product code on retail merchandise.

In **random lists** there is no sequential relationship between two elements. Although there are no restrictions on inserting data into a random list, computer algorithms generally insert data at the end of the list. Thus, random lists are sometimes called **chronological lists**. Applications that use random lists are found either in data-gathering applications, in which case the lists are

chronological lists, or in situations in which the applications require randomness, such as simulation studies or games.

Data must be inserted into ordered lists so that the ordering of the list is maintained. Maintaining the order may require inserting the data at the beginning or at the end of the list, but most of the time data are inserted somewhere in the middle of the list. To determine where the data are to be placed, computer scientists use a search algorithm. Insertions are graphically shown in Figure 5-1. The inserted data are identified by the shaded element; in this example it is the third element of the revised list.

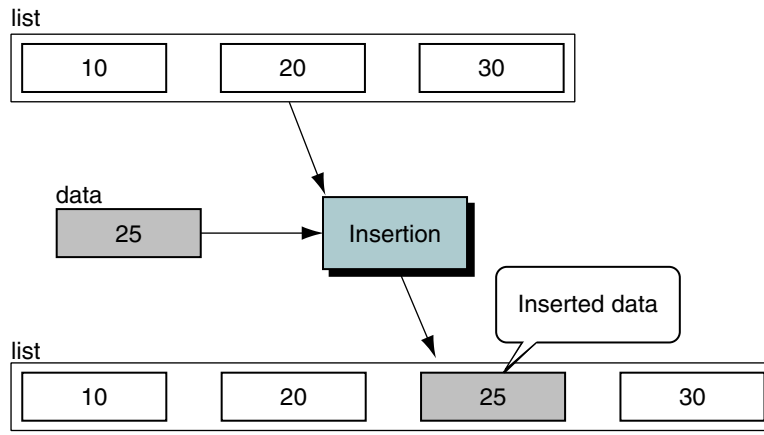


FIGURE 5-1 Insertion

Deletion

Deletion from a list requires that the list be searched to locate the data being deleted. Once located, the data are removed from the list. Figure 5-2 depicts a deletion from a list.

Retrieval

List **retrieval** requires that data be located in a list and presented to the calling module without changing the contents of the list. As with both insertion and deletion, a search algorithm can be used to locate the data to be retrieved from a list. Retrieving data from a list is shown in Figure 5-3.

Traversal

List **traversal** processes each element in a list in sequence. It requires a looping algorithm rather than a search. Each execution of the loop processes one element in the list. The loop terminates when all elements have been processed.

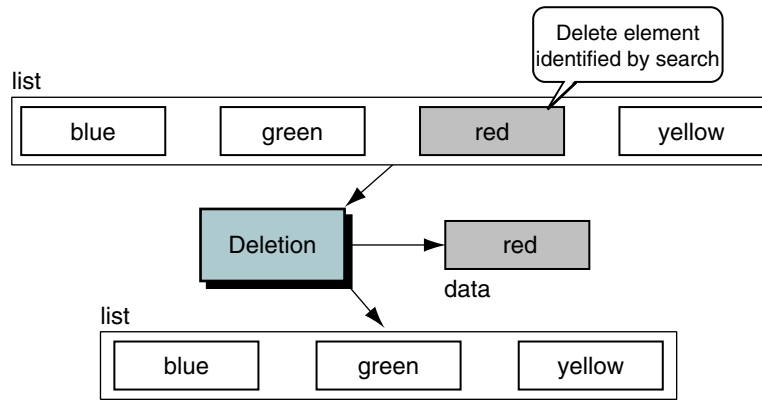


FIGURE 5-2 Deletion

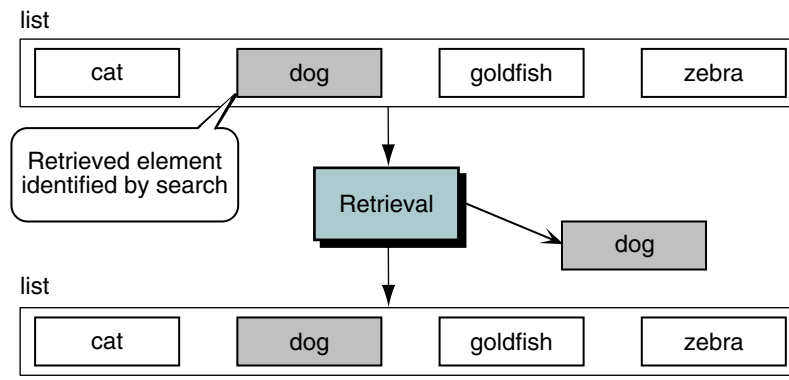


FIGURE 5-3 Retrieval

5.2 Implementation

Several data structures can be used to implement a list; we use a linked list. A linked list is a good structure for a list because data are easily inserted and deleted at the beginning, in the middle, or at the end of the list. Figure 5-4 shows the conceptual view of a list and its implementation as a linked list.

Data Structure

To implement a list, we need two different structures, a head node and data node, as shown in Figure 5-5.

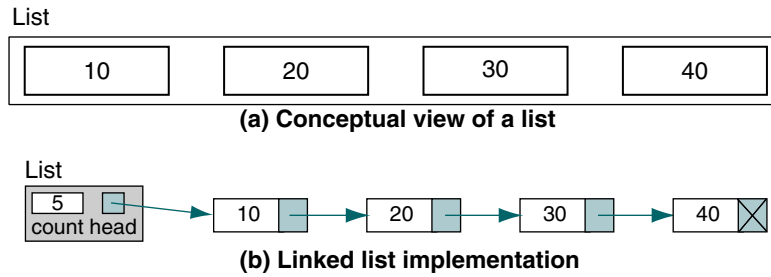


FIGURE 5-4 Linked List Implementation of a List

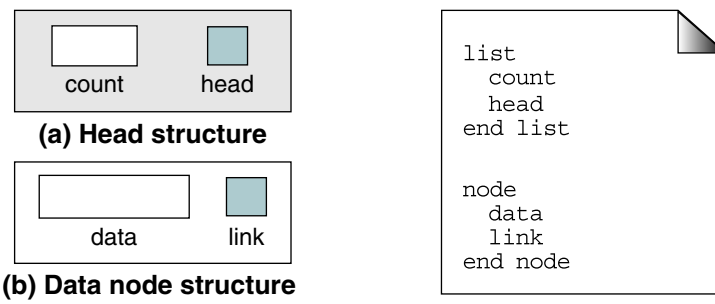


FIGURE 5-5 Head Node and Data Node

Head Node

Although only a single pointer is needed to identify the list, we often find it convenient to create a head node structure that stores the head pointer and other data about the list. When a node contains data about a list, the data are known as **metadata**; that is, they are data about the data in the list. For example, the head structure in Figure 5-4 contains one piece of metadata: `count`, an integer that contains the number of nodes currently in the list. Other metadata, such as the greatest number of nodes during the processing of the list, are often included when they serve a useful purpose.

Data Node

The data type for the list depends entirely on the application. A typical data type is shown below. The data types must be tailored to the particular application being created. We include a key field for applications that require searching by key.

```
data
   key
   field1
   field2
      ...
   fieldN
end data
```

Algorithms

We define 10 operations for a list, which should be sufficient to solve any problem. If an application requires additional list operations, they can be easily added. For each operation we define its name and provide a brief description and its calling parameters. We then develop algorithms for each operation.

Create List

Create list allocates the head structure and initializes the metadata for the list. At this time there are only two metadata entries; later we will add more to expand the capabilities of the list. Figure 5-6 shows the header before and after it is initialized by create list.

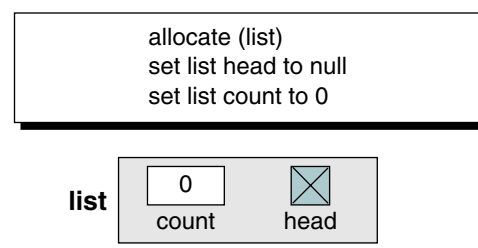


FIGURE 5-6 Create List

The pseudocode for create list is shown in Algorithm 5-1.

ALGORITHM 5-1 Create List

```
Algorithm createList (list)
Initializes metadata for list.
  Pre    list is metadata structure passed by reference
  Post   metadata initialized
1 allocate (list)
2 set list head  to null
3 set list count to 0
end createList
```

Insert Node

Insert node adds data to a list. We need only its logical predecessor to insert a node into the list. Given the predecessor, there are three steps to the insertion:

1. Allocate memory for the new node and move data to the node.
2. Point the new node to its successor.
3. Point the new node's predecessor to the new node.

These steps appear to be simple, but a little analysis is needed to fully understand how to implement them. To insert a node into a list, we need to know the location of the node that precedes the new node. This node is identified by a predecessor pointer that can be in one of two states: it can contain the address of a node or it can be null. When the predecessor pointer is null, it means that there is no predecessor to the data being added. The logical conclusion is that we are either adding to an empty list or are at the beginning of the list. If the predecessor is not null, we are adding somewhere after the first node—that is, in the middle of the list or at the end of the list. Let's discuss each of these situations in turn.

Insert into Empty List

When the head pointer of the list is null, the list is empty. This situation is shown in Figure 5-7. All that is necessary to add a node to an empty list is to assign the list head pointer the address of the new node and make sure that its link field is a null pointer. Although we could use a constant to set the link field of the new node, we use the null pointer contained in the list head. Why we use the list head null pointer will become apparent in the next section.

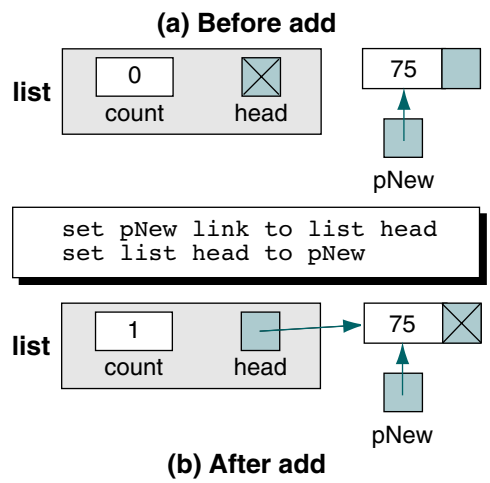


FIGURE 5-7 Add Node to Empty List

The pseudocode statements to insert a node into an empty list are shown here.

```
set pNew link to list head                (Null pointer)
set list head to pNew                     (First node)
```

Note the order of the statements. We must first point the new node to its successor; then we can change the head pointer. If we reverse these statements, we end up with the new node pointing to itself, which puts our program into a never-ending loop when we process the list.

Insert at Beginning

We add at the beginning of the list anytime we need to insert a node before the first node of the list. We determine that we are adding at the beginning of the list by testing the predecessor pointer. If it is a null pointer, there is no predecessor, so we are at the beginning of the list.

To insert a node at the beginning of the list, we simply point the new node to the first node of the list and then set the head pointer to point to the new first node. We know the address of the new node. The question at this point is how we can find the address of the first node currently in the list so that we can point the new node to it. The answer is simple: the first node's address is stored in the head pointer. The pseudocode statements to insert at the beginning of the list are shown here.

```
set pNew link to list head        (To current first node)
set list head to pNew             (To new first node)
```

If we compare these two statements with the statements to insert into an empty list, we will see that they are the same. They are the same because, logically, inserting into an empty list is the same as inserting at the beginning of a list. We can therefore use the same logic to cover both situations. Adding at the beginning of the list is shown in Figure 5-8.

Insert in Middle

When we add a node anywhere in the middle of the list, the predecessor pointer (`pPre`) contains an address. This case is shown in Figure 5-9.

To insert a node between two nodes, we point the new node to its successor and then point its predecessor to the new node. The address of the new node's successor can be found in the predecessor's link field. The pseudocode statements to insert a node in the middle of the list are shown here:

```
set pNew link to pPre link          (New to successor)
set pPre link to pNew               (Predecessor to new)
```

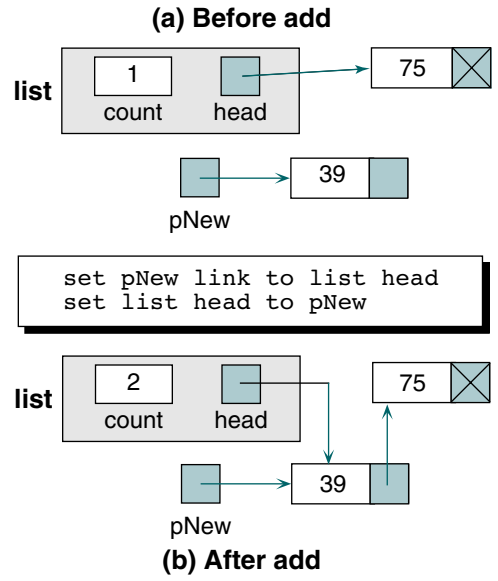


FIGURE 5-8 Add Node at Beginning

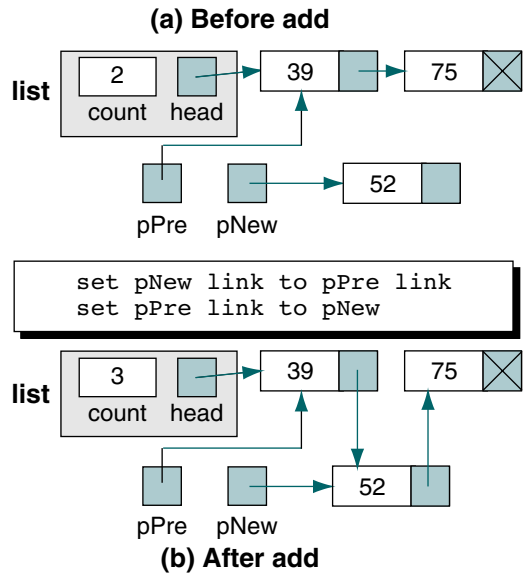


FIGURE 5-9 Add Node in Middle

Insert at End

When we are adding at the end of the list, we only need to point the predecessor to the new node. There is no successor to point to. It is necessary, however, to set the new node's link field to a null pointer. The statements to insert a node at the end of a list are shown here:

```
set pNew link to null pointer
set pPre link to pNew                (Predecessor to new)
```

Rather than have special logic in the algorithm for inserting at the end, we can take advantage of the existing linked list structure. We know that the last node in the list has a null link pointer. If we use this pointer rather than a null pointer constant, the code becomes exactly the same as the code for inserting in the middle of the list. The revised code follows. Compare it with the code for inserting in the middle of the list.

```
set pNew link to pPre link           (New to null)
set pPre link to pNew                (Predecessor to new)
```

Figure 5-10 shows the logic for inserting at the end of a list.

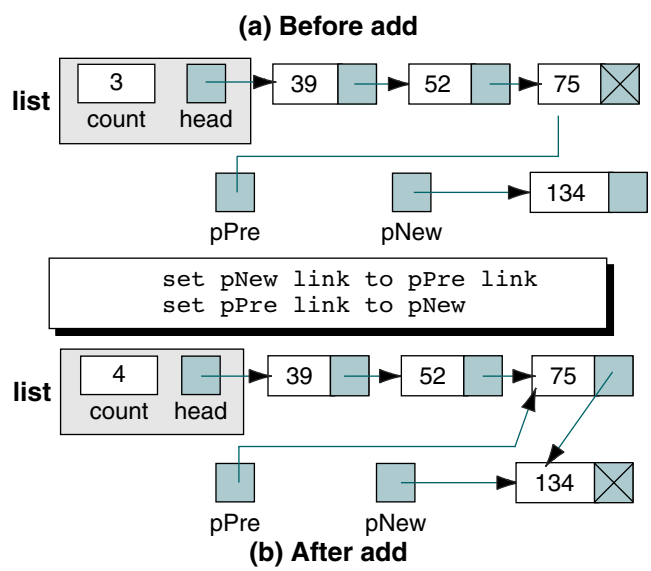


FIGURE 5-10 Add Node at End

Insert Node Algorithm

Now let's write the algorithm that puts it all together and inserts a node into the list. We are given a pointer to the list, the predecessor, and the data to be

inserted. We allocate memory for the new node and adjust the link pointers appropriately. When the algorithm is complete, it returns a Boolean—true if it was successful and false if there was no memory for the insert. The pseudocode is shown in Algorithm 5-2.

ALGORITHM 5-2 Insert List Node

```
Algorithm insertNode (list, pPre, dataIn)
Inserts data into a new node in the list.
  Pre    list is metadata structure to a valid list
         pPre is pointer to data's logical predecessor
         dataIn contains data to be inserted
  Post   data have been inserted in sequence
  Return true if successful, false if memory overflow
 1 allocate (pNew)
 2 set pNew data to dataIn
 3 if (pPre null)
   Adding before first node or to empty list.
   1  set pNew link to list head
   2  set list head to pNew
 4 else
   Adding in middle or at end.
   1  set pNew link to pPre link
   2  set pPre link to pNew
 5 end if
 6 return true
end insertNode
```

Algorithm 5-2 Analysis

We have discussed all of the logic in this algorithm except for the memory allocation. When memory is exhausted, the insert is in an **overflow** state. The action taken depends on the application. Although the application generally needs to abort the program, that decision should not be made in the insert algorithm. We therefore return a Boolean indicating whether we were successful and let the calling module decide whether it needs to abort or whether some other action is appropriate.

Delete Node

The **delete node** algorithm logically removes a node from the list by changing various link pointers and then physically deleting the node from dynamic memory. To logically delete a node, we must first locate the node itself. A delete node is located by knowing its address and its predecessor's address. We will discuss location concepts shortly. Once we locate the node to be deleted, we can simply change its predecessor's link field to point to the deleted node's successor. We then recycle the node back to dynamic memory. We need to be concerned, however, about deleting the only node in a list. Deleting the only node results in an empty list, and so we must be careful that in this case the head is set to a null pointer.

The delete situations parallel those for add. We can delete the only node, the first node, a node in the middle of the list, or the last node of a list. As we

explain below, these four situations reduce to only two combinations: delete the first node and delete any other node. In all cases the node to be deleted is identified by a pointer (`pLoc`).

Delete First Node

When we delete the first node, we must reset the head pointer to point to the first node's successor and then recycle the memory for the deleted note. We can tell we are deleting the first node by testing the predecessor. If the predecessor is a null pointer, we are deleting the first node. This situation is diagrammed in Figure 5-11.

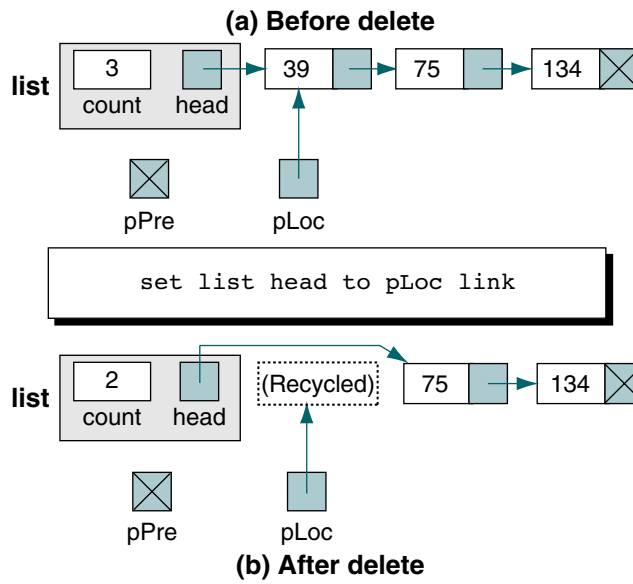


FIGURE 5-11 Delete First Node

The statements to delete the first node are shown in the next example. `Recycle` is the pseudocode command to return a node's space to dynamic memory.

```
set list head to pLoc link
recycle (pLoc)
```

If we examine this logic carefully, we note that it also applies when we are deleting the only node in the list. If the first node is the only node, then its link field is a null pointer. Because we move its link field (a null pointer) to the head pointer, the result is by definition an empty list.

General Delete Case

We call deleting any node other than the first node a general case because the same logic applies to deleting any node either in the middle or at the end of the list. For both of these cases, we simply point the predecessor node to the successor of the node being deleted. The logic is shown in Figure 5-12.

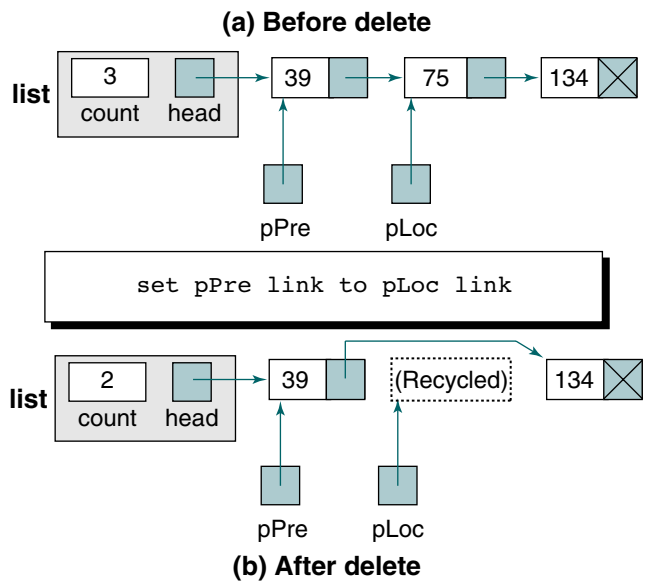


FIGURE 5-12 Delete General Case

We delete the last node automatically. When the node being deleted is the last node of the list, its null pointer is moved to the predecessor's link field, making the predecessor the new logical end of the list. After the pointers have been adjusted, the current node is recycled.

The delete general case pseudocode is shown here:

```
set pPre link to pLoc link
recycle (pLoc)
```

Delete Node Algorithm

The logic to delete a node is shown in Algorithm 5-3. The algorithm is given a pointer to the list, to the node to be deleted, and to the delete node's predecessor. It copies the deleted node's data to a data out area in the calling program and then adjusts the pointers before releasing the node's memory.

ALGORITHM 5-3 List Delete Node

```
Algorithm deleteNode (list, pPre, pLoc, dataOut)
Deletes data from list & returns it to calling module.
  Pre  list is metadata structure to a valid list
       Pre is a pointer to predecessor node
       pLoc is a pointer to node to be deleted
       dataOut is variable to receive deleted data
  Post data have been deleted and returned to caller
 1 move pLoc data to dataOut
 2 if (pPre null)
   Deleting first node
   1  set list head to pLoc link
 3 else
   Deleting other nodes
   1  set pPre link to pLoc link
 4 end if
 5 recycle (pLoc)
end deleteNode
```

Algorithm 5-3 Analysis

We need to discuss two points in this algorithm. The first, and most important, is that the node to be deleted must be identified before it is called. The algorithm assumes that the predecessor and current pointers are properly set. If they aren't, the algorithm will most likely fail. Even if it doesn't fail, the data will be wrong. (It's better that the algorithm fail than that it report invalid results.)

Second, when we discussed the individual logic cases earlier, we placed the recycle statement after each delete statement. In the implementation we moved it to the end of the algorithm. When the same statements appear in both the true and false blocks of a selection statement, they should be moved out of the selection logic. Moving common statements is similar to factoring common expressions in algebra. The result is a program that is smaller and easier to maintain.

List Search

A list search is used by several algorithms to locate data in a list. To insert data, we need to know the logical predecessor to the new data. To delete data, we need to find the node to be deleted and identify its logical predecessor. To retrieve data from a list, we need to search the list and find the data. In addition, many user applications require that lists be searched to locate data.

We must use a sequential search because there is no physical relationship among the nodes. The classic sequential search[1] returns the location of an element when it is found and the address of the last element when it is not found. Because our list is ordered, we need to return the location of the element when it is found and the location *where it should be placed* when it is not found. Knuth[2] calls this search "sequential search in ordered table." We simply call it an ordered list search.

---

1. See Chapter 13.
2. Donald E. Knuth, *The Art of Computer Programming,* Vol. 3, *Sorting and Searching,* Second Edition (Reading, MA: Addison-Wesley, 1998), 398.

To search a list on a key, we need a key field. For simple lists the key and the data can be the same field. For more complex structures, we need a separate key field. We reproduce the data node structure from "Data Node" in Section 5-2 for your convenience:

```
data
   key
   field1
   field2
      ...
   fieldN
end data
```

Given a target key, the ordered list search attempts to locate the requested node in the list. If a node in the list matches the target value, the search returns true; if there are no key matches, it returns false. The predecessor and current pointers are set according to the rules in Table 5-1.

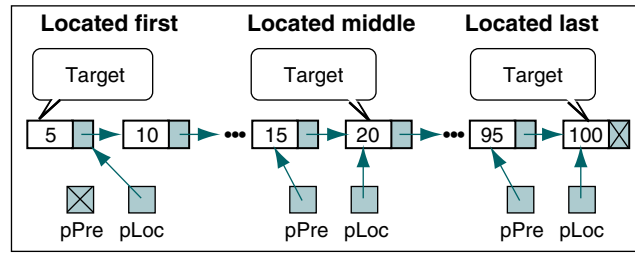
Condition	pPre	pLoc	Return
Target < first node	Null	First node	False
Target = first node	Null	First node	True
First < target < last	Largest node < target	First node > target	False
Target = middle node	Node's predecessor	Equal node	True
Target = last node	Last's predecessor	Last node	True
Target > last node	Last node	Null	False

TABLE 5-1 List Search Results

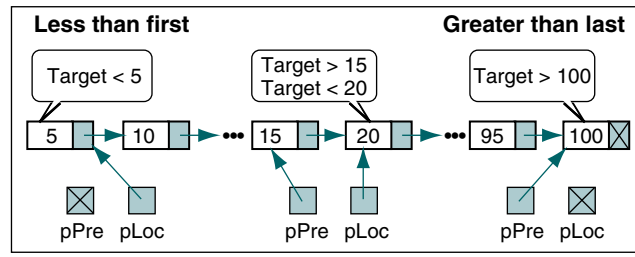
Each of these conditions is also shown in Figure 5-13.

We start at the beginning and search the list sequentially until the target value is no longer greater than the current node's key. At this point the target value is either less than or equal to the current node's key while the predecessor is pointing to the node immediately before the current node. We then test the current node and set the return value to true if the target value is equal to the list value or false if it is less (it cannot be greater) and terminate the search. The pseudocode for this search is shown in Algorithm 5-4.

In the previous discussion we assumed that the list was sequentially ordered on a key. It is often necessary, however, to search the list on a list attribute rather than on the key. For example, given a list of information about the employees in a company, we might want to find employees who have an engineering degree or employees who speak Japanese.



(a) Successful searches (return true)



(b) Unsuccessful searches (return false)

FIGURE 5-13 Ordered List Search

ALGORITHM 5-4 Search List

```
Algorithm searchList (list, pPre, pLoc, target)
Searches list and passes back address of node containing
target and its logical predecessor.
  Pre   list is metadata structure to a valid list
        pPre is pointer variable for predecessor
        pLoc is pointer variable for current node
        target is the key being sought
  Post  pLoc points to first node with equal/greater key
        -or- null if target > key of last node
        pPre points to largest node smaller than key
        -or- null if target < key of first node
  Return true if found, false if not found
1 set pPre to null
2 set pLoc to list head
3 loop (pLoc not null AND target > pLoc key)
  1  set pPre to pLoc
  2  set pLoc to pLoc link
4 end loop
5 if (pLoc null)
  Set return value
  1  set found to false
```

continued

ALGORITHM 5-4 Search List *(continued)*

```
 6 else
   1  if (target equal pLoc key)
      1  set found to true
   2  else
      1  set found to false
   3  end if
 7 end if
 8 return found
end searchList
```

Algorithm 5-4 Analysis

Examine the loop statement carefully. Note that there are two tests. The first test protects us from running off the end of the list; the second test stops the loop when we find the target or, if the target doesn't exist, when we find a node larger than the target. It is important that the null pointer test be done first. If the loop is at the end of the list, the current pointer is no longer valid. Testing the key first would give unpredictable results.[3]

We could make the search slightly more efficient if we had a rear pointer. A rear pointer is a metadata field that contains the address of the last node in the list. With a rear pointer, we could test the last node to make sure that the target wasn't larger than its key value. If the target were larger, we would simply skip the loop, setting the predecessor pointer to the last node and the current pointer to a null pointer. Once we know that the target is not greater than the last node, we don't need to worry about running off the end of the list.

To search a list on any field other than the key, we use a simple sequential search. The problem with nonkey searches, however, is that multiple elements often satisfy the search criteria. Although we might have only one employee who speaks Japanese, we might just as well have zero or many who do. One simple solution is to return a list of all elements that satisfy the criteria.

Retrieve Node

Now that we know how to locate a node in the list, we are ready to study retrieve node. Retrieve node uses search node to locate the data in the list. If the data are found, it moves the data to the output area in the calling module and returns true. If they are not found, it returns false. The pseudocode is shown in Algorithm 5-5.

Empty List

Processing logic often depends on there being data in a list. We provide **empty list**, a simple module that returns a Boolean indicating that there are data in the list or that it is empty (Algorithm 5-6).

3. Note that some languages, such as Pascal, test both expressions in the condition before evaluating the results. In these situations the loop limit test condition must be broken up into two separate expressions.

ALGORITHM 5-5 Retrieve List Node

```
Algorithm retrieveNode (list, key, dataOut)
Retrieves data from a list.
  Pre    list is metadata structure to a valid list
         key is target of data to be retrieved
         dataOut is variable to receive retrieved data
  Post   data placed in dataOut
          -or- error returned if not found
  Return true if successful, false if data not found
1 set found to searchList (list, pPre, pLoc, key)
2 if (found)
  1 move pLoc data to dataOut
3 end if
4 return found
end retrieveNode
```

ALGORITHM 5-6 Empty List

```
Algorithm emtpyList (list)
Returns Boolean indicating whether the list is empty.
  Pre    list is metadata structure to a valid list
  Return true if list empty, false if list contains data
1 if (list count equal 0)
  1 return true
2 else
  1 return false
end emptyList
```

Algorithm 5-6 Analysis

One of the questions often raised by students studying program design is "Why write a module when it contains only one statement—isn't it more efficient to simply code the statement inline?" Although it is definitely more efficient to write the code inline, it is not a better design for a generalized module. Remember that we are implementing generalized code in which the user does not necessarily know the data structure. For example, virtually all systems have an algorithm that tests for end of file. This test is quite simple, most likely only one line of code. It is necessary, however, because the programmer doesn't know the file structure and therefore cannot check the end-of-file status without support.

Full List

At first glance, **full list** appears to be as simple as empty list. As we saw with stacks, however, it turns out to be a relatively complex algorithm to implement. Very few languages provide the programmer with the capability to test how much memory is left in dynamic memory—C does not. The pseudocode is shown in Algorithm 5-7.

ALGORITHM 5-7 Full List

```
Algorithm fullList (list)
Returns Boolean indicating whether or not the list is full.
  Pre    list is metadata structure to a valid list
  Return false if room for new node; true if memory full
 1 if (memory full)
   1  return true
 2 else
   2  return false
 3 end if
 4 return true
end fullList
```

List Count

List count is another simple, one-line module. It is necessary because the calling module has no direct access to the list structure. Its implementation is shown in Algorithm 5-8.

ALGORITHM 5-8 List Count

```
Algorithm listCount (list)
Returns integer representing number of nodes in list.
  Pre    list is metadata structure to a valid list
  Return count for number of nodes in list
 1 return (list count)
end listCount
```

Traverse List

Algorithms that traverse a list start at the first node and examine each node in succession until the last node has been processed. Traversal logic is used by several different types of algorithms, such as changing a value in each node, printing the list, summing a field in the list, or calculating the average of a field. Any application that requires that the entire list be processed uses a traversal.

To traverse the list, we need a walking pointer, a pointer that moves from node to node as each element is processed. Assuming a list with a head structure, the following pseudocode uses a walking pointer to traverse the list. Each loop modifies the pointer to move to the next node in sequence as we traverse the list.

```
set pWalker to list head
loop (more nodes)
   process (pWalker data)
   set pWalker to next link
end loop
```

We begin by setting the walking pointer to the first node in the list. Then, using a loop, we continue until all of the data have been processed. Each loop calls a process module and passes it the data and then advances the walking pointer to the next node. When the last node has been processed, the loop terminates.

We have two possible approaches in designing the traverse list implementation. In one approach the user controls the loop, calling traverse to get the next element in the list. In the other approach, the traverse module controls the loop, calling a user-supplied algorithm to process the data. We implement the first option because it provides the programmer with more flexibility. For example, if the application needs to process only half the list, the loop can simply terminate. The second design would loop through the second half of the data unnecessarily. Figure 5-14 is a graphic representation of a list traversal.

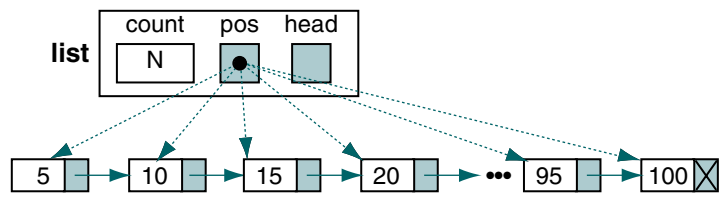


FIGURE 5-14 List Traversal

Because we need to remember where we are in the list from one call to the next, we need to add a current position pointer to the head structure. It keeps track of the node processed after the last call. The head structure is shown in Figure 5-14. The figure also shows how the position pointer is modified to move from one node to the next as the traversal algorithm is called.

Each call also needs to know whether we are starting from the beginning of the list or continuing from the last node processed. This information is communicated through the parameter list. The basic logic to traverse a list is shown in Algorithm 5-9. We name it `getNext` because it is called to get the next node in the traversal.

ALGORITHM 5-9 Traverse List

```
Algorithm getNext (list, fromWhere, dataOut)
Traverses a list. Each call returns the location of an
element in the list.
  Pre    list is metadata structure to a valid list
         fromWhere is 0 to start at the first element
         dataOut is reference to data variable
  Post   dataOut contains data and true returned
         -or- if end of list, returns false
```

continued

ALGORITHM 5-9 Traverse List *(continued)*

```
  Return true if next element located
         false if end of list
1 if (empty list)
  1  return false
2 if (fromWhere is beginning)
  Start from first
  1  set list pos to list head
  2  move current list data to dataOut
  3  return true
3 else
  Continue from pos
  1  if (end of list)
     End of List
     1  return false
   2 else
     1  set list pos to next node
     2  move current list data to dataOut
     3  return true
   3 end if
4 end if
end getNext
```

Algorithm 5-9 Analysis There are two major blocks of code in this algorithm. Statement 2 handles the processing when we start from the beginning of the list; statement 3 handles the processing when we continue from the current location. When we are starting at the beginning, we set the head structure's position pointer to the first node in the list and pass the first node's data back to the calling module.

When we are continuing the list traversal, we must ensure that there are more data in the list. If we are at the end of the list, we set success to false and terminate. If there are more data, we set the current position to the next node and pass its data back to the calling module.

One final word of caution: This design supports only one traversal of any given list at a time. If an application needs more than one traversal at a time, a different design is needed.

Destroy List

When a list is no longer needed but the application is not done, the list should be destroyed. **Destroy list** deletes any nodes still in the list and recycles their memory. It then sets the metadata to a null list condition. The code for destroy list is shown in Algorithm 5-10.

ALGORITHM 5-10 Destroy List

```
Algorithm destroyList (pList)
Deletes all data in list.
  Pre    list is metadata structure to a valid list
```

continued

Chapter 6
Introduction to Trees

The study of trees in mathematics can be traced to Gustav Kirchhoff in the middle nineteenth century and several years later to Arthur Cayley, who used trees to study the structure of algebraic formulas. Cayley's work undoubtedly laid the framework for Grace Hopper's use of trees in 1951 to represent arithmetic expressions. Hopper's work bears a strong resemblance to today's binary tree formats.[1]

Trees are used extensively in computer science to represent algebraic formulas; as an efficient method for searching large, dynamic lists; and for such diverse applications as artificial intelligence systems and encoding algorithms. In this chapter we discuss the basic concepts behind the computer science application of trees. Then, in the following four chapters, we develop the application of trees for specific problems.

6.1 Basic Tree Concepts

A **tree** consists of a finite set of elements, called **nodes**, and a finite set of directed lines, called **branches**, that connect the nodes. The number of branches associated with a node is the **degree** of the node. When the branch is directed toward the node, it is an **indegree** branch; when the branch is directed away from the node, it is an **outdegree** branch. The sum of the indegree and outdegree branches is the degree of the node.

A tree consists of a finite set of elements, called nodes, and a finite set of directed lines, called branches, that connect the nodes.

1. Donald E. Knuth, *The Art of Computer Programming* Vol. 1, *Fundamental Algorithms,* 2nd ed. (Reading, MA: Addison-Wesley, 1972), 405, 458.

If the tree is not empty, the first node is called the **root**. The indegree of the root is, by definition, zero. With the exception of the root, all of the nodes in a tree must have an indegree of exactly one; that is, they may have only one predecessor. All nodes in the tree can have zero, one, or more branches leaving them; that is, they may have an outdegree of zero, one, or more (zero or more successors). Figure 6-1 is a representation of a tree.

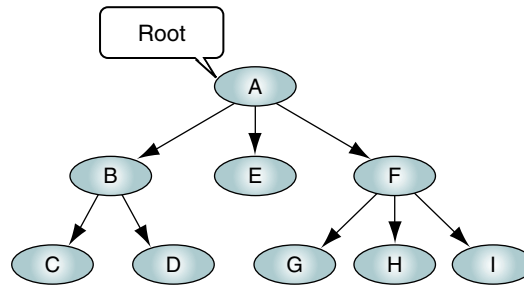


FIGURE 6-1 Tree

Terminology

In addition to *root*, many different terms are used to describe the attributes of a tree. A **leaf** is any node with an outdegree of zero, that is, a node with no successors. A node that is not a root or a leaf is known as an **internal node** because it is found in the middle portion of a tree.

A node is a **parent** if it has successor nodes—that is, if it has an outdegree greater than zero. Conversely, a node with a predecessor is a **child**. A child node has an indegree of one. Two or more nodes with the same parent are **siblings**. Fortunately, we don't have to worry about aunts, uncles, nieces, nephews, and cousins. Although some literature uses the term *grandparent*, we do not. We prefer the more general term *ancestor*. An **ancestor** is any node in the path from the root to the node. A **descendent** is any node in the path below the parent node; that is, all nodes in the paths from a given node to a leaf are descendents of that node. Figure 6-2 shows the usage of these terms.

Several terms drawn from mathematics or created by computer scientists are used to describe attributes of trees and their nodes. A **path** is a sequence of nodes in which each node is adjacent to the next one. Every node in the tree can be reached by following a unique path starting from the root. In Figure 6-2 the path from the root to the leaf `I` is designated as `AFI`. It includes two distinct branches, `AF` and `FI`.

The **level** of a node is its distance from the root. Because the root has a zero distance from itself, the root is at level 0. The children of the root are at level 1, their children are at level 2, and so forth. Note the relationship between levels and siblings in Figure 6-2. Siblings are always at the same level, but all nodes in a level are not necessarily siblings. For example, at level 2, `C` and `D` are siblings, as are `G`, `H`, and `I`. However, `D` and `G` are not siblings because they have different parents.

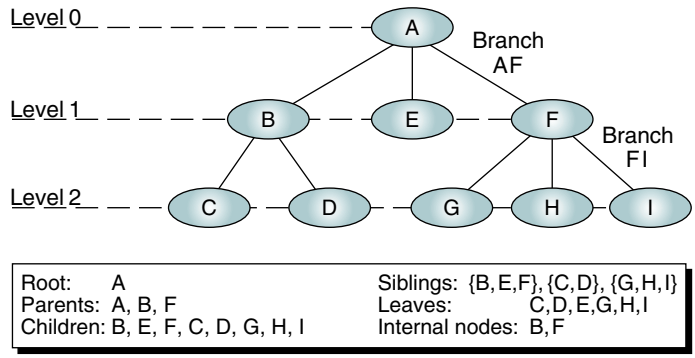


FIGURE 6-2 Tree Nomenclature

The **height** of the tree is the level of the leaf in the longest path from the root plus 1. By definition the height of an empty tree is –1. Figure 6-2 contains nodes at three levels: 0, 1, and 2. Its height is 3. Because the tree is drawn upside down, some texts refer to the **depth** of a tree rather than its height.

The level of a node is its distance from the root. The height of a tree is the level of the leaf in the longest path from the root plus 1.

A tree may be divided into subtrees. A **subtree** is any connected structure below the root. The first node in a subtree is known as the root of the subtree and is used to name the subtree. Subtrees can also be further subdivided into subtrees. In Figure 6-3, BCD is a subtree, as are E and FGHI. Note that by this definition, a single node is a subtree. Thus, the subtree B can be divided into two subtrees, C and D, and the subtree F contains the subtrees G, H, and I.

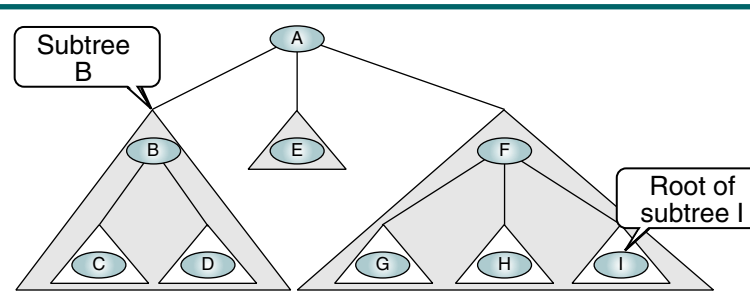


FIGURE 6-3 Subtrees

The concept of subtrees leads us to a recursive definition of a tree: A tree is a set of nodes that either: (1) is empty or (2) has a designated node, called

the root, from which hierarchically descend zero or more subtrees, which are also trees.

A tree is a set of nodes that either:
1. Is empty, or
2. Has a designated node, called the root, from which hierarchically descend zero or more subtrees, which are also trees

User Representation

There are three different user representations for trees. The first is the organization chart format, which is basically the notation we use to represent trees in our figures. The term we use for this notation is **general tree**. The general tree representation of a computer's components is shown in Figure 6-4; it is discussed in Section 6.3.

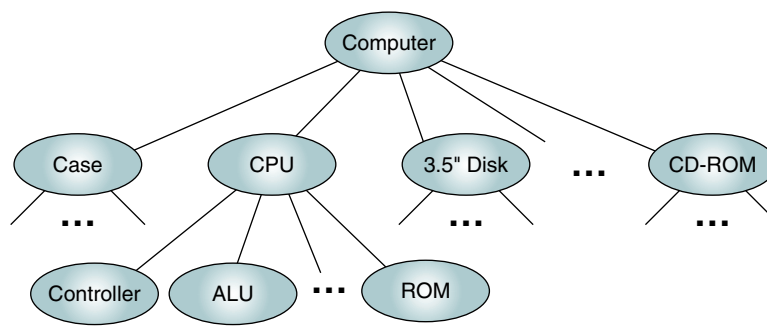


FIGURE 6-4 Computer Parts List as a General Tree

The second representational notation is the indented list. You will find it most often used in bill-of-materials systems in which a parts list represents the assembly structure of an item. The graphical representation of a computer system's components in Figure 6-4 clearly shows the relationship among the various components of a computer, but graphical representations are not easily generated from a database system. The bill-of-materials format was therefore created to show the same information using a textual parts list format. In a bill of materials, each assembly component is shown indented below its assembly. Some bills of materials even show the level number of each component. Because a bill of materials shows which components are assembled into each assembly, it is sometimes called a **goezinta** (goes into) list. Table 6-1 shows the computer bill of materials in an indented parts list format.

There is another common bill of materials with which you should be familiar. When you write a structured program, the entire program can be considered an assembly of related functions. Your structure chart is a general tree representing the relationship among the functions.

Part number	Description
301	Computer
301-1	Case
…	…
301-2	CPU
301-2-1	Controller
301-2-2	ALU
…	…
301-2-9	ROM
301-3	3.5" Disk
…	…
301-9	CD-ROM
…	…

TABLE 6-1 Computer Bill of Materials

The third user format is the parenthetical listing. This format is used with algebraic expressions. When a tree is represented in parenthetical notation, each open parenthesis indicates the start of a new level; each closing parenthesis completes the current level and moves up one level in the tree. Consider the tree shown in Figure 6-1. Its parenthetical notation is

```
A (B (C D) E F (G H I))
```

To convert a general tree to its parenthetical notation, we use the code in Algorithm 6-1.

ALGORITHM 6-1 Convert General Tree to Parenthetical Notation

```
Algorithm ConvertToParen (root, output)
Convert a general tree to parenthetical notation.
   Pre  root is a pointer to a tree node
   Post output contains parenthetical notation
 1 Place root in output
 2 if (root is a parent)
   1  Place an open parenthesis in the output
   2  ConvertToParen (root's first child)
   3  loop (more siblings)
      1  ConvertToParen (root's next child)
```

continued

ALGORITHM 6-1 Convert General Tree to Parenthetical Notation *(continued)*

```
   4   end loop
   5   Place close parenthesis in the output
 3 end if
 4 return
end ConvertToParen
```

Using this algorithm with Figure 6-1, we start by placing the root (A) in the output. Because A is a parent, we insert an opening parenthesis in the output and then process the subtree B by first placing its root (B) in the output. Again, because B is a parent, we place an open parenthesis in the output and recursively call ConvertToParen to insert C. This time, however, the root has no children. The algorithm therefore terminates and returns to statement 2.4. After placing D in the output, we return to the loop statement to discover that there are no more siblings. We now place a closing parenthesis in the output and return to statement 2.4 to continue placing B's siblings in the output.

After I's children have been completely processed, we add a closing parenthesis (2.5), which takes us up one level to complete the processing of B's siblings and the addition of a closing parenthesis. Note that when the processing is done correctly, the parentheses are balanced. If they are not balanced, you have made a mistake in the conversion.

6.2 Binary Trees

A **binary tree** is a tree in which no node can have more than two subtrees; the maximum outdegree for a node is two. In other words, a node can have zero, one, or two subtrees. These subtrees are designated as the **left subtree** and the **right subtree**. Figure 6-5 shows a binary tree with its two subtrees. Note that each subtree is itself a binary tree.

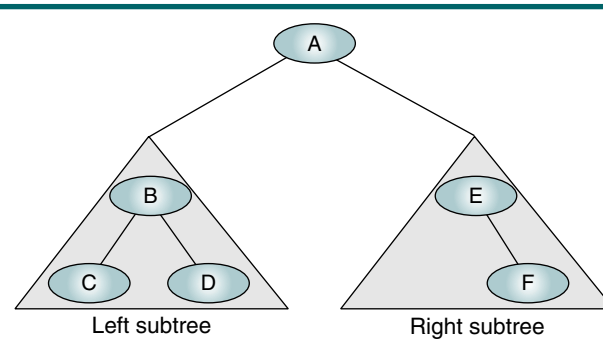


FIGURE 6-5 Binary Tree

To better understand the structure of binary trees, study Figure 6-6.

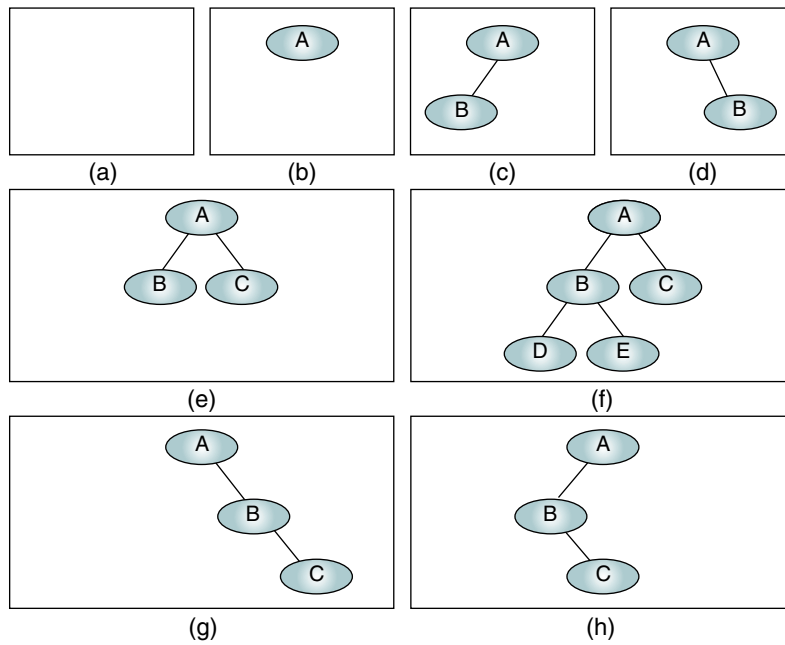


FIGURE 6-6 Collection of Binary Trees

Figure 6-6 contains eight binary trees, the first of which is a null tree. A **null tree** is a tree with no nodes, as shown in Figure 6-6(a). As you study this figure, note that symmetry is not a tree requirement.

A node in a binary tree can have no more than two subtrees.

Properties

We now define several properties for binary trees that distinguish them from general trees.

Height of Binary Trees

The height of binary trees can be mathematically predicted.

Maximum Height

Given that we need to store N nodes in a binary tree, the maximum height, H_{max}, is

$$H_{max} = N$$

Example Given three nodes to be stored in a binary tree, what is the maximum height? In this example N is 3. Therefore, the maximum height is 3. There are four different trees that can be drawn to satisfy a maximum height of 3. A tree with a maximum height is rare. It occurs when all of the nodes in the entire tree have only one successor, as shown in Figure 6-6(g) and Figure 6-6(h).

Minimum Height

The minimum height of the tree, H_{min}, is determined by the following formula:

$$H_{min} = \lfloor \log_2 N \rfloor + 1$$

Example Given three nodes to be stored in a binary tree, what is the minimum height? Again, N is 3. Therefore, the minimum height is 2.

Minimum Nodes. We can turn the calculation around and determine the minimum number of nodes in a tree of a specified height. Given a height of the binary tree, H, the minimum number of nodes in the tree are given as

$$N_{min} = H$$

Example Given a tree of height 3, what is the minimum number of nodes that can be stored? Using the formula for N_{min}, the answer is H, or 3 nodes. This case is seen in Figure 6-6(g) and 6-6(h).

Maximum Nodes

The formula for the maximum number of nodes is derived from the fact that each node can have only two descendents. Given a height of the binary tree, H, the maximum number of nodes in the tree is given as

$$N_{max} = 2^H - 1$$

Example Given a tree of height 3, what is the maximum number of nodes that can be stored? N_{max} is 7.

Balance

The distance of a node from the root determines how efficiently it can be located. The children of any node in a tree can be accessed by following only one branch path, the one that leads to the desired node. The nodes at level 1, which are children of the root, can be accessed by following only one branch. Similarly, the nodes at level 2 of a tree can all be accessed by following only two branches from the root. It stands to reason, therefore, that the shorter the tree, the easier it is to locate any desired node in the tree.

This concept leads us to a very important characteristic of a binary tree—, its **balance**. To determine whether a tree is balanced, we calculate its balance factor. The **balance factor** of a binary tree is the difference in height between its left and right subtrees. If we define the height of the left subtree as H_L and the height of the right subtree as H_R, the balance factor of the tree, B, is determined by the following formula:

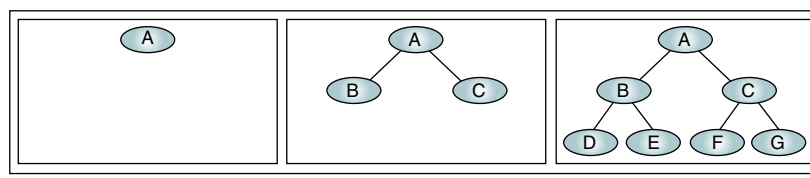
$$B = H_L - H_R$$

Using this formula, the balances of the eight trees in Figure 6-6 are (a) 0 by definition, (b) 0, (c) 1, (d) –1, (e) 0, (f) 1, (g) –2, and (h) 2.

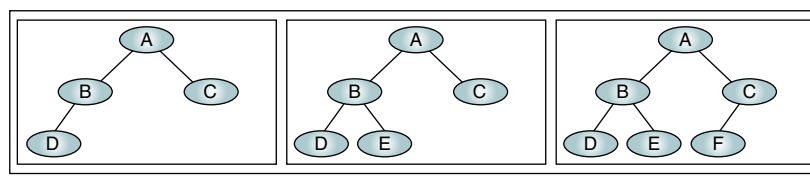
In a **balanced binary tree**, the height of its subtrees differs by no more than one (its balance factor is –1, 0, or +1), and its subtrees are also balanced. As we shall see, this definition was created by Adelson-Veskii and Landis in their definition of an AVL tree.

Complete and Nearly Complete Binary Trees

A **complete tree** has the maximum number of entries for its height (see formula N_{max} in "Height of Binary Trees"). The maximum number is reached when the last level is full (see Figure 6-7). A tree is considered **nearly complete** if it has the minimum height for its nodes (see formula H_{min}) and all nodes in the last level are found on the left. Complete and nearly complete trees are shown in Figure 6-7.



(a) Complete trees (at levels 0, 1, and 2)



(b) Nearly complete trees (at level 2)

FIGURE 6-7 Complete and Nearly Complete Trees

Binary Tree Traversals

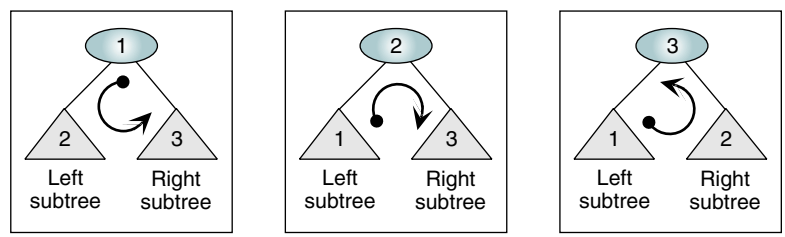
A **binary tree traversal** requires that each node of the tree be processed once and only once in a predetermined sequence. The two general approaches to the traversal sequence are depth first and breadth first. In the **depth-first traversal**, the

processing proceeds along a path from the root through one child to the most distant descendent of that first child before processing a second child. In other words, in the depth-first traversal, we process all of the descendents of a child before going on to the next child.

In a **breadth-first traversal**, the processing proceeds horizontally from the root to all of its children, then to its children's children, and so forth until all nodes have been processed. In other words, in the breadth-first traversal, each level is completely processed before the next level is started.

Depth-first Traversals

Given that a binary tree consists of a root, a left subtree, and a right subtree, we can define six different depth-first traversal sequences. Computer scientists have assigned three of these sequences standard names in the literature; the other three are unnamed but are easily derived. The standard traversals are shown in Figure 6-8.



(a) Preorder traversal **(b) Inorder traversal** **(c) Postorder traversal**

FIGURE 6-8 Binary Tree Traversals

The traditional designation of the traversals uses a designation of `node` (N) for the root, `left` (L) for the left subtree, and `right` (R) for the right subtree. To demonstrate the different traversal sequences for a binary tree, we use Figure 6-9.

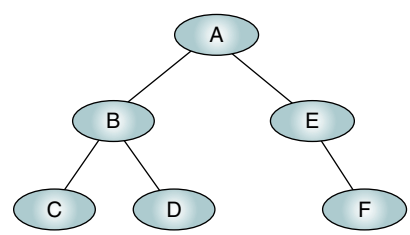


FIGURE 6-9 Binary Tree for Traversals

Preorder Traversal (NLR)

In the **preorder traversal**, the root node is processed first, followed by the left subtree and then the right subtree. It draws its name from the Latin prefix *pre*, which means to go before. Thus, the root goes before the subtrees.

> In the preorder traversal, the root is processed first, before its subtrees.

Given the recursive characteristics of trees, it is only natural to implement tree traversals recursively. First we process the root, then the left subtree, and then the right subtree. The left subtree is in turn processed recursively, as is the right subtree. The code for the preorder traversal is shown in Algorithm 6-2.

ALGORITHM 6-2 Preorder Traversal of a Binary Tree

```
Algorithm preOrder (root)
Traverse a binary tree in node-left-right sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
 1 if (root is not null)
   1  process (root)
   2  preOrder (leftSubtree)
   3  preOrder (rightSubtree)
 2 end if
end preOrder
```

Figure 6-9 contains a binary tree with each node named. The processing sequence for a preorder traversal processes this tree as follows: First we process the root A. After the root, we process the left subtree. To process the left subtree, we first process its root, B, then its left subtree and right subtree in order. When B's left and right subtrees have been processed in order, we are then ready to process A's right subtree, E. To process the subtree E, we first process the root and then the left subtree and the right subtree. Because there is no left subtree, we continue immediately with the right subtree, which completes the tree.

Figure 6-10 shows another way to visualize the traversal of the tree. Imagine that we are walking around the tree, starting on the left of the root and keeping as close to the nodes as possible. In the preorder traversal we process the node when we meet it for the first time (on the left of the node). This is shown as a black box on the left of the node. The path is shown as a line following a route completely around the tree and back to the root.

Figure 6-11 shows the recursive algorithmic traversal of the tree. The first call processes the root of the tree, A. It then recursively calls itself to process the root of the subtree B, as shown in Figure 6-11(b). The third call, shown in Figure 6-11(c), processes node C, which is also subtree C. At this point we call preorder with a null pointer, which results in an immediate return to subtree C

to process its right subtree. Because C's right subtree is also null, we return to node B so that we can process its right tree, D, in Figure 6-11(d). After processing node D, we make two more calls, one with D's null left pointer and one with its null right pointer. Because subtree B has now been completely processed, we return to the tree root and process its right subtree, E, in Figure 6-11(e). After a call to E's null left subtree, we call E's right subtree, F, in Figure 6-11(f). Although the tree is completely processed at this point, we still have two more calls to make: one to F's null left subtree and one to its null right subtree. We can now back out of the tree, returning first to E and then to A, which concludes the traversal of the tree.

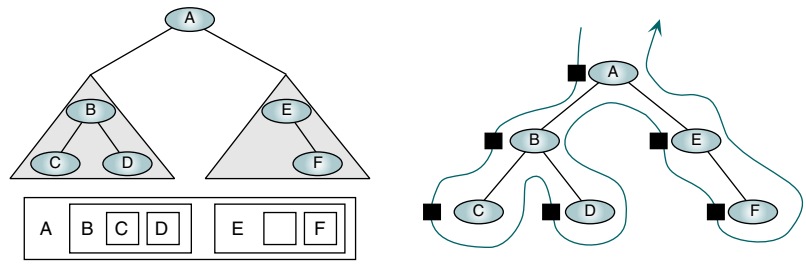


FIGURE 6-10 Preorder Traversal—A B C D E F

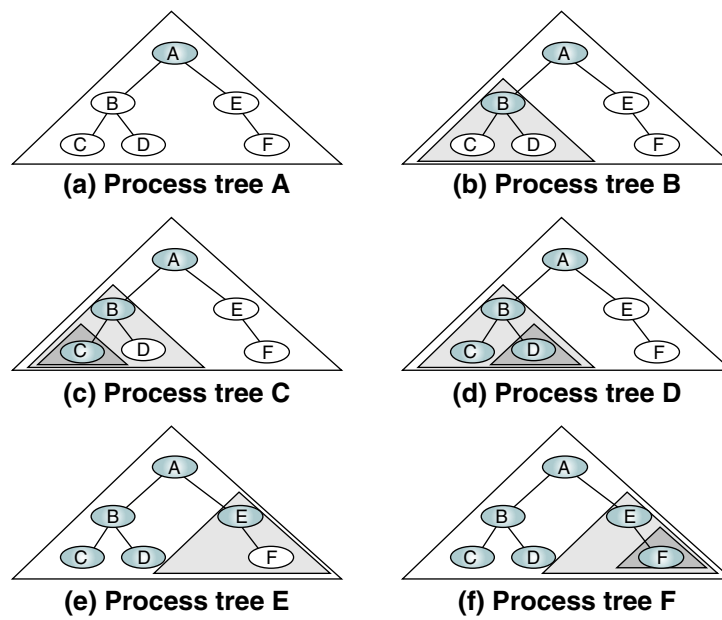


FIGURE 6-11 Algorithmic Traversal of Binary Tree

Inorder Traversal (LNR)

The **inorder traversal** processes the left subtree first, then the root, and finally the right subtree. The meaning of the prefix *in* is that the root is processed *in between* the subtrees. Once again we implement the algorithm recursively, as shown in Algorithm 6-3.

ALGORITHM 6-3 Inorder Traversal of a Binary Tree

```
Algorithm inOrder (root)
Traverse a binary tree in left-node-right sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
 1  if (root is not null)
    1   inOrder (leftSubTree)
    2   process (root)
    3   inOrder (rightSubTree)
 2  end if
end inOrder
```

Because the left subtree must be processed first, we trace from the root to the far-left leaf node before processing any nodes. After processing the far-left subtree, C, we process its parent node, B. We are now ready to process the right subtree, D. Processing D completes the processing of the root's left subtree, and we are now ready to process the root, A, followed by its right subtree. Because the right subtree, E, has no left child, we can process its root immediately followed by its right subtree, F. The complete sequence for inorder processing is shown in Figure 6-12.

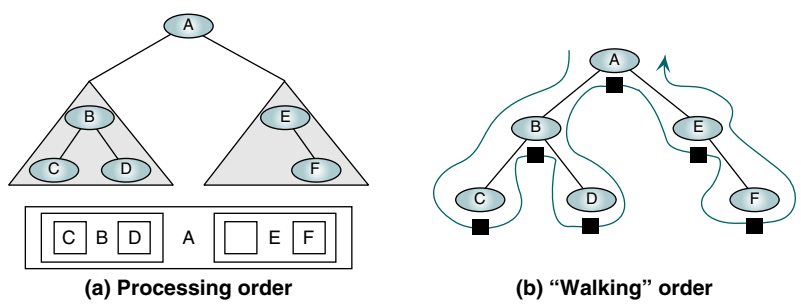


FIGURE 6-12 Inorder Traversal—C B D A E F

To walk around the tree in inorder sequence, we follow the same path but process each node when we meet it for the second time (the bottom of the node). This processing route is shown in Figure 6-12(b).

In the inorder traversal, the root is processed between its subtrees.

Postorder Traversal (LRN)

The last of the standard traversals is the **postorder traversal**. It processes the root node after (*post*) the left and right subtrees have been processed. It starts by locating the far-left leaf and processing it. It then processes its right sibling, including its subtrees (if any). Finally, it processes the root node.

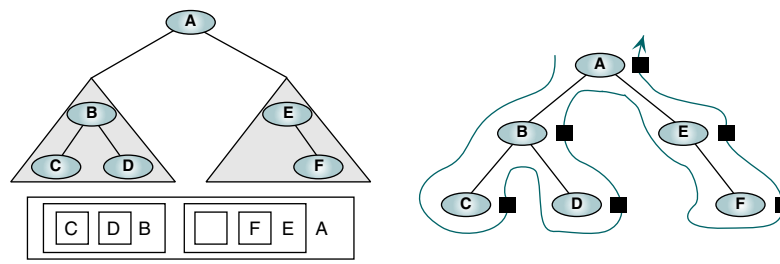
> In the postorder traversal, the root is processed after its subtrees.

The recursive postorder traversal logic is shown in Algorithm 6-4.

ALGORITHM 6-4 Postorder Traversal of a Binary Tree

```
Algorithm postOrder (root)
Traverse a binary tree in left-right-node sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
 1  if (root is not null)
    1   postOrder (left subtree)
    2   postOrder (right subtree)
    3   process (root)
 2  end if
end postOrder
```

In the tree walk for a postorder traversal, we move the processing block to the right of the node so that we process it as we meet the node for the third time. The postorder traversal is shown in Figure 6-13. Note that we took the same path in all three walks; only the time of the processing changed.



(a) Processing order (b) "Walking" order

FIGURE 6-13 Postorder Traversal—C D B F E A

Breadth-first Traversals

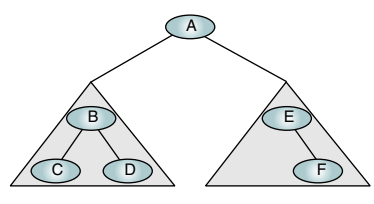
In the breadth-first traversal of a binary tree, we process all of the children of a node before proceeding with the next level. In other words, given a root at level *n*, we process all nodes at level *n* before proceeding with the nodes at

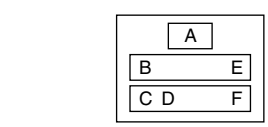
level $n + 1$. To traverse a tree in depth-first order, we used a stack. (Remember that recursion uses a stack.) To traverse a tree in breadth-first order, we use a queue. The pseudocode for a breadth-first traversal of our binary tree is shown in Algorithm 6-5.

ALGORITHM 6-5 Breadth-first Tree Traversal

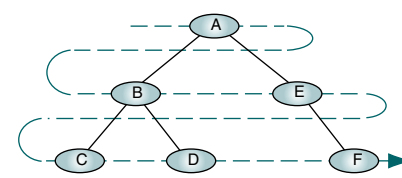
```
Algorithm breadthFirst (root)
Process tree using breadth-first traversal.
   Pre    root is node to be processed
   Post   tree has been processed
 1 set currentNode to root
 2 createQueue (bfQueue)
 3 loop (currentNode not null)
   1  process (currentNode)
   2  if (left subtree not null)
      1  enqueue (bfQueue, left subtree)
   3  end if
   4  if (right subtree not null)
      1  enqueue (bfQueue, right subtree)
   5  end if
   6  if (not emptyQueue(bfQueue))
      1  set currentNode to dequeue (bfQueue)
   7  else
      1  set currentNode to null
   8  end if
 4 end loop
 5 destroyQueue (bfQueue)
end breadthFirst
```

Like the depth-first traversals, we can trace the traversal with a walk. This time, however, the walk proceeds in a horizontal fashion, first across the root level, then across level 1, then across level 2, and so forth until the entire tree is traversed. The breadth-first traversal is shown in Figure 6-14.





(a) Processing order



(b) "Walking" order

FIGURE 6-14 Breadth-first Traversal

Expression Trees

One interesting application of binary trees is expression trees. An **expression** is a sequence of tokens that follow prescribed rules. A **token** may be either an operand or an operator. In this discussion we consider only binary arithmetic operators in the form operand–operator–operand. The standard arithmetic operators are +, -, *, and /.

An **expression tree** is a binary tree with the following properties:

- Each leaf is an operand.
- The root and internal nodes are operators.
- Subtrees are subexpressions, with the root being an operator.

Figure 6-15 contains an infix expression and its expression tree.

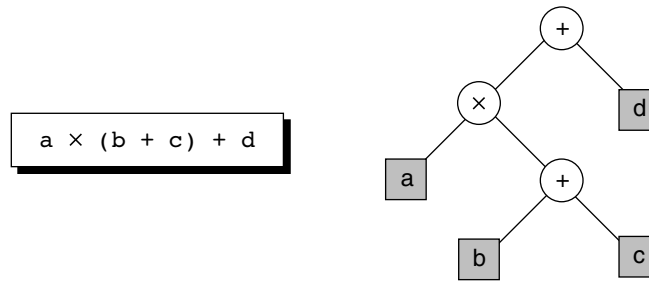


FIGURE 6-15 Infix Expression and Its Expression Tree

For an expression tree, the three standard depth-first traversals represent the three different expression formats: infix, postfix, and prefix. The inorder traversal produces the infix expression, the postorder traversal produces the postfix expression, and the preorder traversal produces the prefix expression.

Infix Traversal

To demonstrate the **infix traversal** of an expression tree, let's write an algorithm that traverses the tree and prints the expression. When we print the infix expression tree, we must add an opening parenthesis at the beginning of each expression and a closing parenthesis at the end of each expression. Because the root of the tree and each of its subtrees represents a subexpression, we print the opening parenthesis when we start a tree or a subtree and the closing parenthesis when we have processed all of its children. Figure 6-16 shows the placement of the parentheses as we walk through the tree using an inorder traversal.

The pseudocode for the infix expression tree traversal is shown in Algorithm 6-6.

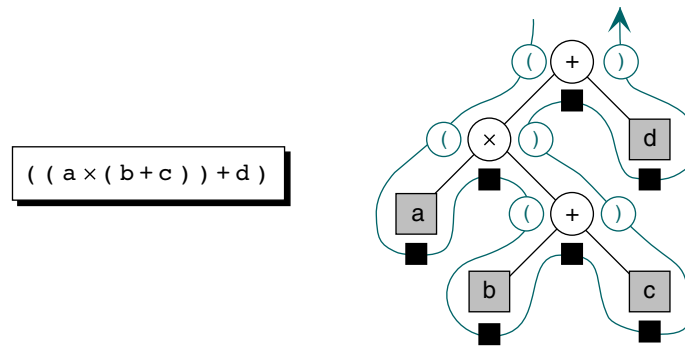


FIGURE 6-16 Infix Traversal of an Expression Tree

ALGORITHM 6-6 Infix Expression Tree Traversal

```
Algorithm infix (tree)
Print the infix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the infix expression has been printed
1  if (tree not empty)
   1   if (tree token is an operand)
       1   print (tree-token)
   2   else
       1   print (open parenthesis)
       2   infix (tree left subtree)
       3   print (tree token)
       4   infix (tree right subtree)
       5   print (close parenthesis)
   3   end if
2  end if
end infix
```

Postfix Traversal

The **postfix traversal** of an expression uses the basic postorder traversal of any binary tree. Note that it does not require parentheses. The pseudocode is shown in Algorithm 6-7.

ALGORITHM 6-7 Postfix Traversal of an Expression Tree

```
Algorithm postfix (tree)
Print the postfix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the postfix expression has been printed
1  if (tree not empty)
```

continued

ALGORITHM 6-7 Postfix Traversal of an Expression Tree *(continued)*

```
   1  postfix (tree left subtree)
   2  postfix (tree right subtree)
   3  print   (tree token)
 2 end if
end postfix
```

Prefix Traversal

The final expression tree traversal is the **prefix traversal**. It uses the standard preorder tree traversal. Again, no parentheses are necessary. The pseudocode is shown in Algorithm 6-8.

ALGORITHM 6-8 Prefix Traversal of an Expression Tree

```
Algorithm prefix (tree)
Print the prefix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the prefix expression has been printed
 1 if (tree not empty)
   1  print  (tree token)
   2  prefix (tree left subtree)
   3  prefix (tree right subtree)
 2 end if
end prefix
```

Huffman Code

The American Standard Code for Information Interchange (ASCII) is a fixed-length code; that is, the character length does not vary. Each ASCII character consists of 7 bits.[2] Although the character E occurs more frequently than the character Z, both are assigned the same number of bits. This consistency means that every character uses the maximum number of bits.

Huffman code, on the other hand, makes character storage more efficient. In Huffman code we assign shorter codes to characters that occur more frequently and longer codes to those that occur less frequently. For example, E and T, two characters that occur frequently in the English language, could be assigned one bit each. A, O, R, and N, which also occur frequently but less frequently than E and T, could be assigned two bits each. S, U, I, D, M, C, and G are the next most frequent and could be assigned three bits each, and so forth. In a given piece of text, only some of the characters require the maximum bit length. When used in a network transmission, the overall length of the transmission is shorter if Huffman-encoded characters are transmitted

2. When ASCII code is stored in an 8-bit byte, only the first 128 values are considered ASCII. The second 128 characters are used for special characters and graphics and are considered "extended ASCII."

Chapter 7
Binary Search Trees

We now turn our attention to search trees, with an in-depth discussion of two standard tree structures: binary search trees in this chapter and AVL trees in Chapter 8. Both are used when data need to be ordered. They differ primarily in that AVL trees are balanced, whereas binary search trees are not.

In the design of the linear list structure, we had two choices: an array or a linked list. The array structure provides a very efficient search algorithm, the binary search, but its insertion and deletion algorithms are very inefficient. On the other hand, the linked list structure provides efficient insertion and deletion, but its search algorithm is very inefficient. What we need is a structure that provides an efficient search algorithm and at the same time efficient insert and delete algorithms. The binary search tree and the AVL tree provide that structure.

7.1 Basic Concepts

A **binary search tree (BST)** is a binary tree with the following properties:

- All items in the left subtree are less than the root.
- All items in the right subtree are greater than or equal to the root.
- Each subtree is itself a binary search tree.

> In a binary search tree, the left subtree contains key values less than the root and the right subtree contains key values greater than or equal to the root.

Generally, the information represented by each node is a record rather than a single data element. When the binary search tree definition is applied to a record, the sequencing properties refer to the key of the record. Figure 7-1 reflects the properties of a binary tree in which K is the key.

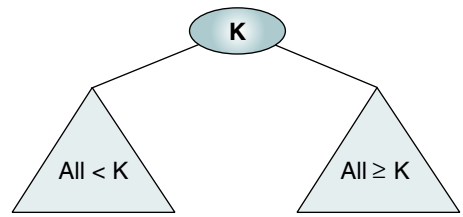


FIGURE 7-1 Binary Search Tree

Figure 7-2 contains five binary search trees. As you study them, note that the trees in Figures 7-2(a) and (b) are complete and balanced, the tree in Figure 7-2(d) is nearly complete and balanced, and the trees in Figures 7-2(c) and (e) are neither complete nor balanced.

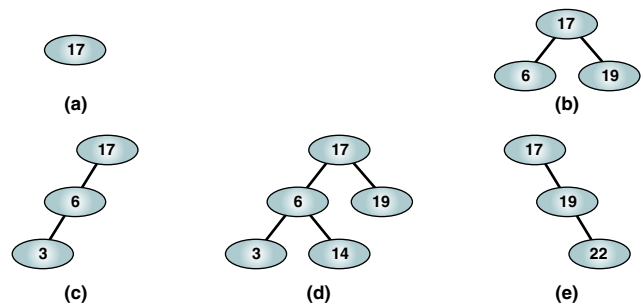


FIGURE 7-2 Valid Binary Search Trees

Now let's look at some binary trees that do not have the properties of a binary search tree. Examine the binary trees in Figure 7-3. The first tree, Figure 7-3(a), breaks the first rule: all items in the left subtree must be less than the root. The key in the left subtree (22) is greater than the key in the root (17). The second tree, Figure 7-3(b), breaks the second rule: all items in the right subtree must be greater than or equal to the root. The key in the right subtree (11) is less than the key in the root (17). Figure 7-3(c) breaks the third rule: each subtree must be a binary search tree. In this tree the left subtree key (6) is less than the root (17), and the right subtree key (19) is greater than the root. However, the left subtree is not a valid binary search tree because it breaks the first rule: its left subtree (11) is greater than the root (6). Figure 7-3(d) also breaks one of the three rules. Do you see which one? (*Hint:* What is the largest key in the left subtree?)

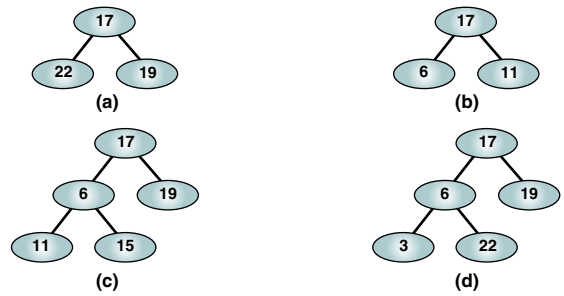


FIGURE 7-3 Invalid Binary Search Trees

7.2 BST Operations

We now examine several operations used on binary search trees. We start with the traversal algorithms we studied in Chapter 6. We then look at some simple search algorithms and conclude with the algorithms that build a binary search tree.

Traversals

The **binary tree traversal** operations are identical to the ones in Chapter 6. Our interest here is not in the operation itself but rather in the results it produces. Let's begin by traversing the tree in Figure 7-4.

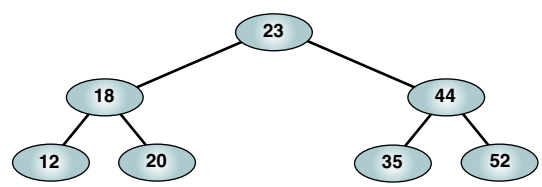


FIGURE 7-4 Example of a Binary Search Tree

If we traverse the tree using a preorder traversal, we get the results shown below.

```
23 18 12 20 44 35 52
```

Although this traversal is valid, it is not very useful. Let's try a postorder traversal and see if it is more useful.

```
12 20 18 35 52 44 23
```

Again, this sequence holds little promise of being useful. Let's try an inorder traversal.

```
12 18 20 23 35 44 52
```

This traversal has some very practical use: the inorder traversal of a binary search tree produces a sequenced list. What happens if you traverse the tree using a right-node-left sequence? Try it and see.[1]

> The inorder traversal of a binary search tree produces a sequenced list.

Searches

In this section we study three search algorithms: find the smallest node, find the largest node, and find a requested node (BST search).

Find the Smallest Node

As we examine the binary search tree in Figure 7-4, we note that the node with the smallest value (12) is the far-left leaf node in the tree. The **find smallest node** operation, therefore, simply follows the left branches until we get to a leaf. Algorithm 7-1 contains the pseudocode to find the smallest node in a binary search tree.

ALGORITHM 7-1 Find Smallest Node in a BST

```
Algorithm findSmallestBST (root)
This algorithm finds the smallest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of smallest node
1  if (left subtree empty)
   1   return (root)
2  end if
3  return findSmallestBST (left subtree)
end findSmallestBST
```

Algorithm 7-1 Analysis As is typical with trees, this algorithm is recursive. The first call starts with the root of the tree, as shown in Figure 7-5. We then follow a path down the left subtrees. If the left

1. As you can see, the right-node-left traversal traverses the tree in a descending sequence. This is not a standard traversal, but it can be very useful.

subtree is not null, we must keep looking farther to the left. We do so with a recursive call to `findSmallestBST` with the left subtree. The base case in this algorithm occurs when we find an empty left subtree. At this point we return the address of the current node, which is the node containing 12 as the base case. Because the recursive call is part of a return statement, as we move back up the tree we continue to return the node address to the smallest node until we finally return it to the initiating module.

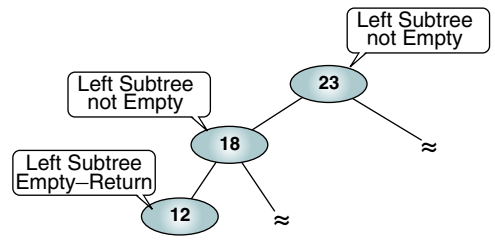


FIGURE 7-5 Find Smallest Node in a BST

Find the Largest Node

The logic to the **find largest node** operation in a binary search tree is the reverse of finding the smallest node. This time we start at the tree root and follow the right branches to the far-right node in the tree, which by definition must be the largest. The pseudocode is shown in Algorithm 7-2.

ALGORITHM 7-2 Find Largest Node in a BST

```
Algorithm findLargestBST (root)
This algorithm finds the largest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of largest node returned
1  if (right subtree empty)
   1   return (root)
2  end if
3  return findLargestBST (right subtree)
end findLargestBST
```

BST Search

We now examine the most important feature of binary search trees, the **binary tree search**, which locates a specific node in the tree. To help us understand how the BST search works, let's revisit the binary search algorithm, as shown in Figure 7-6. This figure traces each of the possible search paths from the middle element in the array. Starting with 23, the binary search examines either 18 or 44, depending on the search key. From 18 it examines either 12 or 20; from 44 it examines either 35 or 52. As is clear in the figure, tracing all possible search paths follows the same paths we see in a binary search tree.

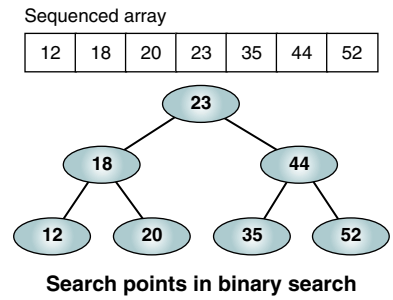


FIGURE 7-6 BST and the Binary Search

Now let's reverse the process. Find a given node in a binary search tree. Assume we are looking for node 20. We begin by comparing the search argument, 20, with the value in the tree root. Because 20 is less than the root value, 23, and because we know that all values less than the root lie in its left subtree, we go left. We now compare the search argument with the value in the subtree, 18. This time the search argument is greater than the root value, 18. Because we know that values greater than the tree root must lie in its right subtree, we go right and find our desired value. This logic is shown in Algorithm 7-3.

ALGORITHM 7-3 Search BST

```
Algorithm searchBST (root, targetKey)
Search a binary search tree for a given value.
   Pre    root is the root to a binary tree or subtree
          targetKey is the key value requested
   Return the node address if the value is found
          null if the node is not in the tree
 1 if (empty tree)
      Not found
   1  return null
 2 end if
 3 if (targetKey < root)
   1  return searchBST (left subtree, targetKey)
 4 else if (targetKey > root)
   1  return searchBST (right subtree, targetKey)
 5 else
      Found target key
   1  return root
 6 end if
end searchBST
```

Algorithm 7-3 Analysis We implement the BST search using recursion. In this algorithm there are two base cases: either we find the search argument in the tree, in which case we return the

address of its node (statement 5.1), or the search argument doesn't exist, in which case we return null (statement 1.1).

Study the returns at statements 3.1 and 4.1 carefully. Note that they are returning the value given by the recursive call, which as we saw earlier is either null or the address of the node we are trying to locate. These statements are necessary to pass the located address back through the recursion to the original requester. Figure 7-7 traces the path through the binary search tree from Figure 7-4, as we search for node 20 using Algorithm 7-3.

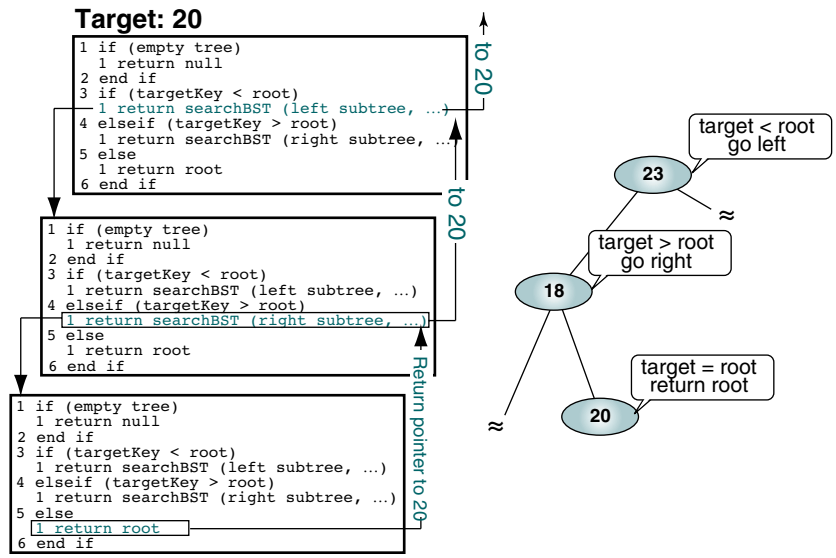


FIGURE 7-7 Searching a BST

Insertion

The **insert node** function adds data to a BST. To insert data all we need to do is follow the branches to an empty subtree and then insert the new node. In other words, all inserts take place at a leaf or at a leaflike node—a node that has only one null subtree.

All BST insertions take place at a leaf or a leaflike node.

Figure 7-8 shows our binary search tree after we have inserted two nodes. We first added node 19. To locate its insertion point, we searched the tree through the path 23, 18, and 20 to a null left branch. After locating the insertion point, we inserted the new node as the left subtree of 20. We then added 38. This time we searched the tree through 23, 44, and 35 to a null right subtree and inserted the new node.

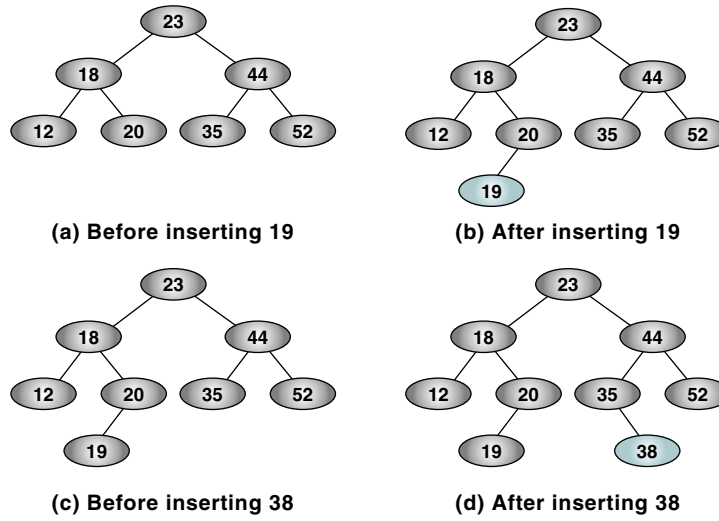


FIGURE 7-8 BST Insertion

Insertions of both 19 and 38 were made at a leaf node. If we inserted a duplicate of the root, 23, it would become the left subtree of 35. Remember that in a binary search tree, nodes with equal values are found in the right subtree. The path for its insertion would therefore be 23, 44, and 35. In this case the insertion takes place at a leaflike node. Although 35 has a right subtree, its left subtree is null. We would therefore place the new node, 23, as the left subtree of 35.

We are now ready to develop the insert algorithm. We can write an elegant algorithm that inserts the data into the tree using recursion. If the tree or subtree is empty, we simply insert the data at the root. If we are not at an empty tree, we determine which branch we need to follow and call recursively to determine whether we are at a leaf yet. The pseudocode is shown in Algorithm 7-4.

ALGORITHM 7-4 Add Node to BST

```
Algorithm addBST (root, newNode)
Insert node containing new data into BST using recursion.
   Pre    root is address of current node in a BST
          newNode is address of node containing data
   Post   newNode inserted into the tree
   Return address of potential new tree root
1  if (empty tree)
   1   set root to newNode
   2   return newNode
2  end if
```

continued

ALGORITHM 7-4 Add Node to BST *(continued)*

```
   Locate null subtree for insertion
3  if (newNode < root)
   1   return addBST (left subtree, newNode)
4  else
   1   return addBST (right subtree, newNode)
5  end if
end addBST
```

Algorithm 7-4 Analysis The algorithm is quite elegant, but it is not easy to see how the new node is inserted at the correct location. To help, let's insert a node into a tree as shown in Figure 7-9.

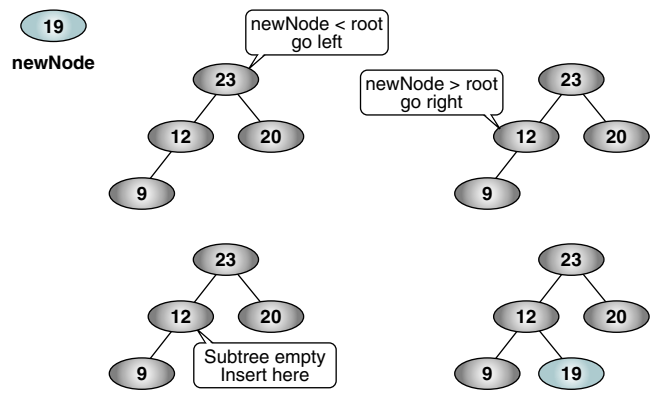


FIGURE 7-9 Trace of Recursive BST Insert

To insert 19 into the tree, we start with `root`. Because the new node's key is less than the root key, we recursively call `addBST` using the left subtree (12). The new node's key is now greater than the root key, so we again call recursively with the right subtree, which is null. At this point we discover that the subtree is null, so we insert the new node, replacing the null subtree as 12's right subtree.

Deletion

To **delete a node** from a binary search tree, we must first locate it. There are four possible cases when we delete a node:

1. The node to be deleted has no children. In this case, all we need to do is delete the node.
2. The node to be deleted has only a right subtree. We delete the node and attach the right subtree to the deleted node's parent.
3. The node to be deleted has only a left subtree. We delete the node and attach the left subtree to the deleted node's parent.
4. The node to be deleted has two subtrees. It is possible to delete a node from the middle of a tree, but the result tends to create very unbalanced

trees. Rather than simply delete the node, therefore, we try to maintain the existing structure as much as possible by finding data to take the place of the deleted data. This can be done in one of two ways: (1) we can find the largest node in the deleted node's left subtree and move its data to replace the deleted node's data or (2) we can find the smallest node on the deleted node's right subtree and move its data to replace the deleted node's data. Regardless of which logic we use, we will be moving data from a leaf or a leaflike node that can then be deleted. Prove to yourself that either of these moves preserves the integrity of the binary search tree.

The pseudocode for the binary search tree delete is shown in Algorithm 7-5.

ALGORITHM 7-5 Delete Node from BST

```
Algorithm deleteBST (root, dltKey)
This algorithm deletes a node from a BST.
   Pre    root is reference to node to be deleted
          dltKey is key of node to be deleted
   Post   node deleted
          if dltKey not found, root unchanged
   Return true if node deleted, false if not found
 1 if (empty tree)
   1  return false
 2 end if
 3 if (dltKey < root)
   1  return deleteBST (left subtree, dltKey)
 4 else if (dltKey > root)
   1  return deleteBST (right subtree, dltKey)
 5 else
      Delete node found--test for leaf node
   1  If (no left subtree)
      1  make right subtree the root
      2  return true
   2  else if (no right subtree)
      1  make left subtree the root
      2  return true
   3  else
         Node to be deleted not a leaf. Find largest node on
         left subtree.
      1  save root in deleteNode
      2  set largest to largestBST (left subtree)
      3  move data in largest to deleteNode
      4  return deleteBST (left subtree of deleteNode,
                           key of largest
   4  end if
 6 end if
end deleteBST
```

Algorithm 7-5 Analysis

You need to study this algorithm carefully to fully understand it. First, note that it is a recursive algorithm. There are two base cases: First, we do not find the node. In that

case the root pointer is null. This case is handled in statement 1.1. The second case occurs after we have deleted the node, at either statement 5.1.2 or statement 5.2.2.

The first two cases on page 313 have been combined in one case in the actual implementation of the algorithm. If the left subtree is null, we can simply connect the right subtree to the parent (`root`). If the right subtree is null, we are connecting a null subtree, which is correct. If the right subtree is not null, its data are connected to the deleted node's parent, which is Case 2. On the other hand, if the left subtree is not null, we test to see whether the right subtree is null. If the right subtree is null, we can move the left subtree pointer to its parent.

The most difficult logic in this algorithm occurs when the node is not a leaf. You need to study this situation carefully to fully understand it. We begin by searching for the largest node on the left subtree and move its data to replace the data to be deleted. We then call the algorithm recursively, giving it a new delete target, the key of the leaf node that contains the data we moved to the internal or root node. This guarantees that when we find the target key this time, at least one of its subtrees is null. We trace this logic in Figure 7-10.

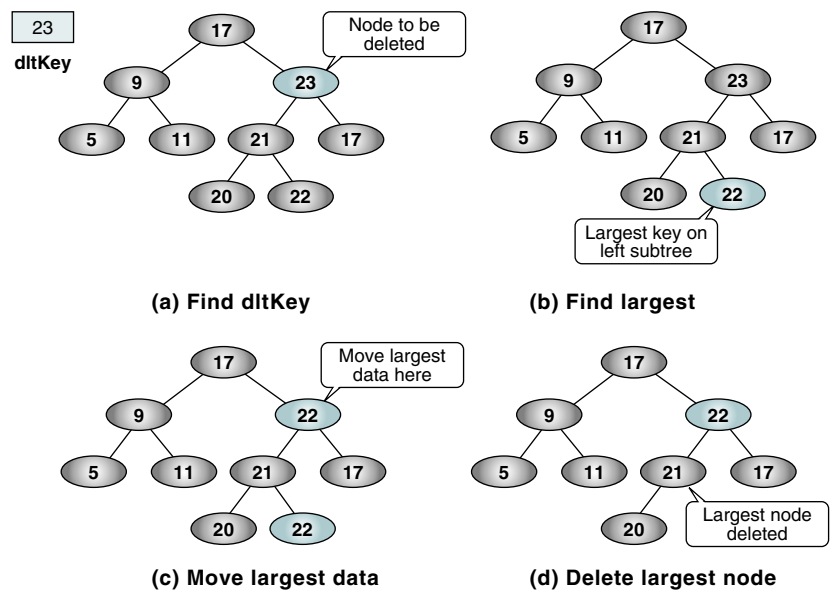


FIGURE 7-10 Delete BST Test Cases

7.3 Binary Search Tree ADT

No programming language has intrinsic operations for a binary search tree. Its operations must be simulated using C functions. The model for the BST abstract data type is the ADT we developed for lists in Chapter 5. Both structures use dynamic memory to store nodes containing a pointer to the application data. Both structures require pointers to identify each node's successor. Whereas the list contains only one pointer to each successor, the BST tree uses two—one for the left subtree and one for the right subtree. Because there is

Chapter 12
Sorting

One of the most common applications in computer science is **sort**, the process through which data are arranged according to their values. We are surrounded by data. If data were not ordered, we would spend hours trying to find a single piece of information. Imagine the difficulty of finding someone's telephone number in a telephone book that had no internal order.

The history of sorting goes back to the roots of computing. Herman Hollerith's electric tabulating machine was used to tally the 1890 U.S. Census and was one of the first modern sorting machines. Sorting was also on the scene when general-purpose computers first came into use. According to Knuth, "There is evidence that a sorting routine was the first program ever written for a stored program computer."[1] Although computer scientists have not developed a major new algorithm in more than 30 years (the newest algorithm in this book is heap sort, which was developed in 1964), sorting is still one of the most important concepts in computing today.

12.1 Sort Concepts

We discuss six internal sorts in this chapter: insertion sort, bubble sort, selection sort, shell sort, heap sort, and quick sort. The first three are useful only for sorting very small lists, but they form the basis of the last three, which are useful general-purpose sorting concepts. After discussing the internal sorts, we will introduce the basic concepts used in external sorts.

Sorting is one of the most common data-processing applications.

1. Donald E. Knuth, *The Art of Computer Programming,* vol. 3, *Sorting and Searching*, Second Edition (Reading, MA: Addison-Wesley, 1998), 385.

Sorts are generally classified as either internal or external sorts. In an **internal sort**, all of the data are held in primary memory during the sorting process. An **external sort** uses primary memory for the data currently being sorted and secondary storage for any data that does not fit in primary memory. For example, a file of 20,000 records may be sorted using an array that holds only 1000 records. During the sorting process, only 1000 records are therefore in memory at any one time; the other 19,000 are kept in a file in secondary storage.

Sorting algorithms are classified as either internal or external.

Internal sorting algorithms have been grouped into several different classifications depending on their general approach to sorting. Knuth identified five different classifications: insertion, selection, exchanging, merging, and distribution sorts.[2] In this text we cover the first three. Distribution sorts, although interesting, have minimal use in computers. The different sorts are shown in Figure 12-1.

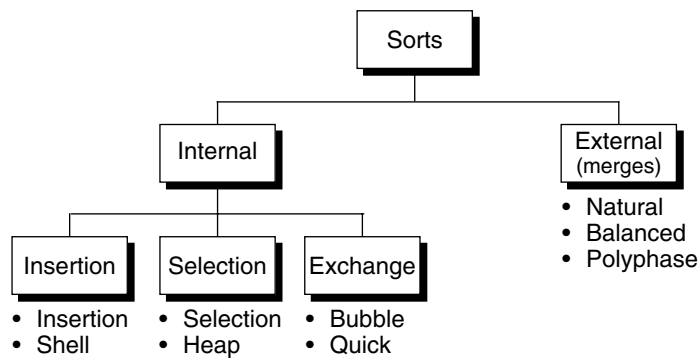


FIGURE 12-1 Sort Classifications

Sort Order

Data may be sorted in either ascending sequence or descending sequence. The **sort order** identifies the sequence of the sorted data, ascending or descending. If the order of the sort is not specified, it is assumed to be ascending. Examples of common data sorted in ascending sequence are the dictionary and the telephone book. Examples of common descending data are percentages of games won in a sporting event such as baseball or grade-point averages for honor students.

2. See Knuth, *Art of Computer Programming,* 73–179.

Sort Stability

Sort stability is an attribute of a sort, indicating that data with equal keys maintain their relative input order in the output. Stability is best seen in an example. In Figure 12-2(a) the unsorted data contain three entries with identical keys (212). If the data are sorted with a stable sort, the order in Figure 12-2(b) is guaranteed. That is, 212 `green` is guaranteed to be the first of the three in the output, 212 `yellow` is guaranteed to be the second, and 212 `blue` is guaranteed to be the last. If the sort is not stable, records with identical keys may occur in any order, including the stable order shown in Figure 12-2(b). Figure 12-2(c) is one example of the six different sequences that could occur in an unstable sort. Note that in this example `blue` comes out first even though it was the last of the equal keys. Of the sort algorithms we will discuss in this text, the straight insertion sort and the bubble sort are stable; the others are unstable.

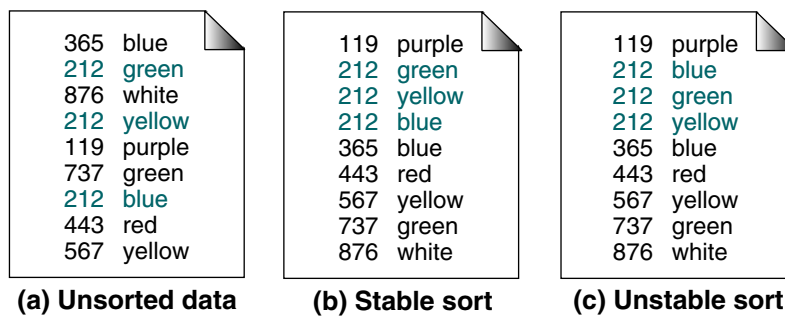


FIGURE 12-2 Sort Stability

Sort Efficiency

Sort efficiency is a measure of the relative efficiency of a sort. It is usually an estimate of the number of comparisons and moves required to order an unordered list. We discuss the sort efficiency of each of the internal sorts we cover in this chapter. Generally speaking, however, the best possible sorting algorithms are on the order of $n \log n$; that is, they are $O(n \log n)$ sorts.[3] Three of the sorts we study are $O(n^2)$. The best, quick sort, is $O(n \log n)$.

Passes

During the sorting process, the data are traversed many times. Each traversal of the data is referred to as a **sort pass**. Depending on the algorithm, the sort

3. As a project, we discuss an interesting sort—the radix sort—which is $O(n)$. However, its extensive overhead limits its general use.

pass may traverse the whole list or just a section of the list. Also, a characteristic of a sort pass is the placement of one or more elements in a sorted list.

Sorts and ADTs

Sort algorithms can be easily adapted to an ADT implementation using the techniques discussed in Section 1.5, "Generic Code for ADTs." There is one significant difference between the ADTs in previous chapters and sort ADTs: in sort ADTs the data type is defined and filled in the application. It is then passed to the ADT for sorting.

To sort data of an unknown type and size, the ADT needs four parameters, as shown in the following prototype statement:

```
bool sortADT (void* ary,
              int   sizeofElem,
              int   numElem,
              int  (*compare)(void* arg1, void* arg2));
```

The data to be sorted is received as a *void* pointer. To locate an element in the array, we can treat the array as an array of characters and use pointer arithmetic to move from one element to another. For example, to address the fifth element of the array, we use the following statement:

```
(char*)ary + (walker * sizeofElem)
```

The number of elements entry (`numElem`) is needed to identify the physical end of the list. And, finally, a compare function is needed, as we have seen in all of the key-sequenced ADTs.

All sort algorithms require that array elements be exchanged. Because we don't know the physical type, the exchanges must treat each element as an array of characters and move them one at a time. The following code segment shows how to exchange two elements:

```
// Move smallest to hold area
for (int i = 0; i < sizeofElem; i++)
     *(holdData + i) = *(smallest + i);
// Move current to smallest location
movePtr = (char*)ary + (current * sizeofElem);
for (int i = 0; i < sizeofElem; i++)
     *(smallest + i) = *(movePtr + i);
// Move hold area current location
for (int i = 0; i < sizeofElem; i++)
     *(movePtr + i)  = *(holdData + i);
```

In this chapter we implement the sort algorithms using integer data so that we can concentrate on the sort design. We leave ADT implementations for problems and projects at the end of the chapter.

12.2 Selection Sorts

Selection sorts are among the most intuitive of all sorts. Given a list of data to be sorted, we simply select the smallest item and place it in a sorted list. These steps are then repeated until we have sorted all of the data. In this section we study two selection sorts: the straight selection sort and the heap sort.

> In each pass of the selection sort, the smallest element is selected from the unsorted sublist and exchanged with the element at the beginning of the unsorted list.

Straight Selection Sort

In the **straight selection sort**, the list at any moment is divided into two sublists, sorted and unsorted, which are divided by an imaginary wall. We select the smallest element from the unsorted sublist and exchange it with the element at the beginning of the unsorted data. After each selection and exchange, the wall between the two sublists moves one element, increasing the number of sorted elements and decreasing the number of unsorted ones. Each time we move one element from the unsorted sublist to the sorted sublist, we say that we have completed one sort pass. If we have a list of n elements, therefore, we need $n - 1$ passes to completely rearrange the data. The selection sort is graphically presented in Figure 12-3.

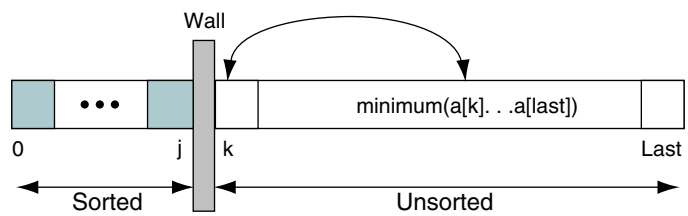


FIGURE 12-3 Selection Sort Concept

Figure 12-4 traces our set of six integers as we sort them. It shows how the wall between the sorted and unsorted sublists moves in each pass. As you study the figure, you see that the array is sorted after five passes, one less than the number of elements in the array. Thus, if we use a loop to control the sorting, our loop has one less iteration than the number of elements in the array.

Selection Sort Algorithm

If you knew nothing about sorting and were asked to sort a list on paper, you would undoubtedly scan the list to locate the smallest item and then copy it to a second list. You would then repeat the process of locating the smallest remaining item in the list and copying it to the new list until you had copied all items to the sorted list.

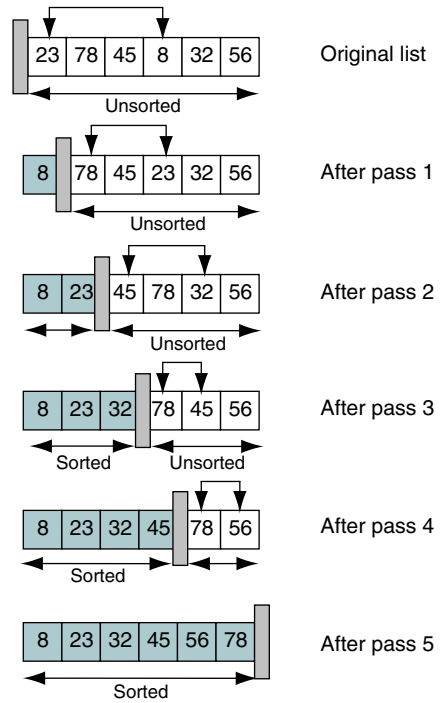


FIGURE 12-4 Selection Sort Example

With the exception of using two areas, this is exactly how the selection sort works. Starting with the first item in the list, the algorithm scans the list for the smallest element and exchanges it with the item at the beginning of the list. Each selection and exchange is one sort pass. After advancing the index (wall), the sort continues until the list is completely sorted. The pseudocode is shown in Algorithm 12-1.

ALGORITHM 12-1 Selection Sort

```
Algorithm selectionSort (list, last)
Sorts list array by selecting smallest element in
unsorted portion of array and exchanging it with element
at the beginning of the unsorted list.
   Pre  list must contain at least one item
        last contains index to last element in the list
   Post list has been rearranged smallest to largest
1  set current to 0
2  loop (until last element sorted)
   1   set smallest to current
```

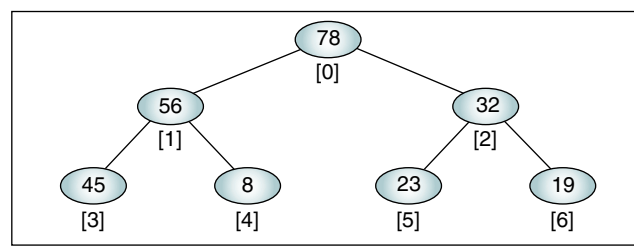
continued

ALGORITHM 12-1 Selection Sort *(continued)*

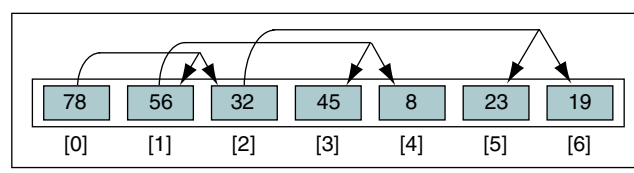
```
   2  set walker   to current + 1
   3  loop (walker <= last)
      1  if (walker key < smallest key)
         1  set smallest to walker
      2  increment walker
   4  end loop
      Smallest selected: exchange with current element.
   5  exchange (current, smallest)
   6  increment current
3  end loop
end selectionSort
```

Heap Sort

In Chapter 9 we studied heaps. Recall that a heap is a tree structure in which the root contains the largest (or smallest) element in the tree. (You may want to review Chapter 9 before studying heap sort.) As a quick review, Figure 12-5 shows a heap in its tree form and in its array form.



(a) Heap in its tree form



(b) Heap in its array form

FIGURE 12-5 Heap Representations

The **heap sort** algorithm is an improved version of the straight selection sort. The straight selection sort algorithm scans the unsorted elements and selects the smallest element. Finding the smallest element among the n elements requires $n - 1$ comparisons. This part of the selection sort makes it very slow.

The heap sort also selects an element from the unsorted portion of the list, but it is the largest element. Because the heap is a tree structure, however, we don't need to scan the entire list to locate the largest key. Rather, we

reheap, which moves the largest element to the root by following tree branches. This ability to follow branches makes the heap sort much faster than the straight selection sort.

Heap sort begins by turning the array to be sorted into a heap. The array is turned into a heap only once for each sort. We then exchange the root, which is the largest element in the heap, with the last element in the unsorted list. This exchange results in the largest element's being added to the beginning of the sorted list. We then reheap down to reconstruct the heap and exchange again. The reheap and exchange process continues until the entire list is sorted.

> The heap sort is an improved version of the selection sort in which the largest element (the root) is selected and exchanged with the last element in the unsorted list.

The heap sort process is shown in Figure 12-6. We first turn the unordered list into a heap. You should verify for yourself that the array is actually a heap.

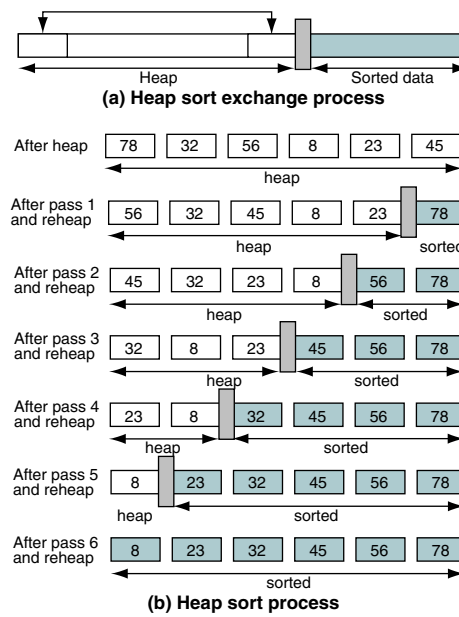


FIGURE 12-6 Heap Sort Process

Because the largest element (78) is at the top of the heap, we can exchange it with the last element in the heap and move the heap wall one element to the left. This exchange places the largest element in its correct location at the end of the array, but it destroys the heap. We therefore rebuild the heap. The smaller heap now has its largest element (56) at the top. We

exchange 56 with the element at the end of the heap (23), which places 56 in its correct location in the sorted array. The reheap and exchange processing continues until we have sorted the entire array.

Heap Sort Algorithm

Algorithm 12-2 contains the pseudocode for heap sort. It uses two algorithms defined in Chapter 9: Algorithm 9-1, "Reheap Up," and Algorithm 9-2, "Reheap Down."

ALGORITHM 12-2 Heap Sort

```
Algorithm heapSort (heap, last)
Sort an array, using a heap.
   Pre  heap array is filled
        last is index to last element in array
   Post heap array has been sorted
   Create heap
 1 set walker to 1
 2 loop (heap built)
   1   reheapUp (heap, walker)
   2   increment walker
 3 end loop
   Heap created. Now sort it.
 4 set sorted to last
 5 loop (until all data sorted)
   1   exchange (heap, 0, sorted)
   2   decrement sorted
   3   reheapDown (heap, 0, sorted)
 6 end loop
end heapSort
```

Algorithm 12-2 Analysis Whereas the heap-array structure is relatively complex, the heap sort algorithm is simple. The algorithm begins by using the reheap up algorithm to turn the array into a heap. It then sorts the array by exchanging the element at the top of the heap with the element at the end of the heap and rebuilding the heap using the reheap down algorithm.

Selection Sort Efficiency

In this section we examine the sort efficiency for the selection sorts.

Straight Selection Sort

The code for the straight selection sort is found in Algorithm 12-1, "Selection Sort." It contains the two loops shown below.

```
2   loop (until last element sorted)
    ...
    3   loop (walker <= last)
```

The outer loop executes $n - 1$ times. The inner loop also executes $n - 1$ times. This is a classic example of the quadratic loop. Its search effort therefore, using big-O notation, is $O(n^2)$.

The straight selection sort efficiency is $O(n^2)$.

Heap Sort

The heap sort pseudocode is shown in Algorithm 12-2. Ignoring the effort required to build the heap initially, the sort contains two loops. The first is a simple iterative loop; the second is a recursive loop:

```
5   loop (until all data sorted)
    ...
    3   reheapDown (heap, 0, sorted)
```

The outer loop starts at the end of the array and moves through the heap one element at a time until it reaches the first element. It therefore loops n times. The inner loop follows a branch down a binary tree from the root to a leaf or until the parent and child data are in heap order. The probability of the data being in order before we reach the leaf is very small, so we ignore it. The difficult part of this analysis is that for each of the outer loops, the heap becomes smaller, shortening the path from the root to a leaf. Again, except for the largest of heaps, this factor is rather minor and is eliminated in big-O analysis; therefore, we ignore it also.

Following the branches of a binary tree from a root to a leaf requires log n loops. The sort effort, the outer loop times the inner loop, for the heap sort is therefore

$$n(\log n)$$

When we include the processing to create the original heap, the big-O notation is the same. Creating the heap requires n log n loops through the data. When factored into the sort effort, it becomes a coefficient, which is then dropped to determine the final sort effort.

The heap sort efficiency is $O(n \log n)$.

Summary

Our analysis leads to the conclusion that the heap sort, $O(n \log n)$, is more efficient than the straight selection sort, $O(n^2)$. Table 12-1 offers a comparison of these two sorts. Depending on the algorithm's implementation, the run time can be affected.

n	Number of loops	
	Straight selection	Heap
25	625	116
100	10,000	664
500	250,000	4482
1000	1,000,000	9965
2000	4,000,000	10,965

TABLE 12-1 Comparison of Selection Sorts

Selection Sort Implementation

We now turn our attention to implementing the selection sort algorithms in C. We first implement the straight selection sort and then the heap sort.

Selection Sort C Code

The code for the selection sort is shown in Program 12-1. Its design is found in Algorithm 12-1.

PROGRAM 12-1 Selection Sort

```
/* ================== selectionSort ====================
   Sorts list [1…last] by selecting smallest element in
   unsorted portion of array and exchanging it with
   element at beginning of the unsorted list.
      Pre  list must contain at least one item
           last contains index to last list element
      Post list has been sorted smallest to largest
*/
void selectionSort  (int list[ ], int last)
{
// Local Declarations
   int smallest;
   int holdData;

// Statements
   for (int current = 0; current < last; current++)
       {
        smallest = current;
        for (int walker = current + 1;
                 walker <= last;
                 walker++)
            if (list[ walker ] < list[ smallest ])
```

continued

PROGRAM 12-4 Shell Sort *(continued)*

```
28                walker = ( walker - incre );
29               } //  while
30             // Insert hold in proper position
31             list [walker + incre] = hold;
32            } // for walk
33          // End of pass--calculate next increment.
34          incre = incre / 2;
35        } // while
36    return;
37  } // shellSort
```

Program 12-4 Analysis Although more complex than the straight insertion sort, the shell sort is by no means a difficult algorithm. There are only two additional complexities over the straight insertion sort. First, rather than a single array, we have an array of partitions. Second, whenever an exchange is made, we must fall back and verify the order of the partition.

12.4 Exchange Sorts

The third category of sorts, **exchange sorts**, contains the most common sort taught in computer science—the bubble sort—and the most efficient general-purpose sort, quick sort. In exchange sorts we exchange elements that are out of order until the entire list is sorted. Although virtually every sorting method uses some form of exchange, the sorts in this section use it extensively.

Bubble Sort

In the **bubble sort**, the list at any moment is divided into two sublists: sorted and unsorted. The smallest element is bubbled from the unsorted sublist and moved to the sorted sublist. After moving the smallest to the sorted list, the wall moves one element to the right, increasing the number of sorted elements and decreasing the number of unsorted ones (Figure 12-12). Each time an element moves from the unsorted sublist to the sorted sublist, one sort pass is completed. Given a list of n elements, the bubble sort requires up to $n - 1$ passes to sort the data.

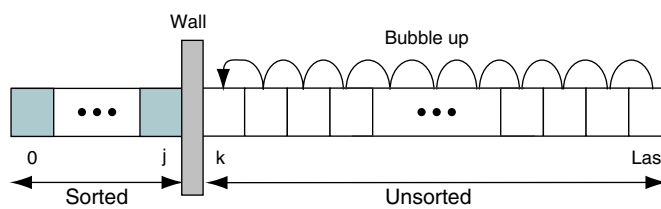


FIGURE 12-12 Bubble Sort Concept

Figure 12-13 shows how the wall moves one element in each pass. Looking at the first pass, we start with 32 and compare it with 56. Because 32 is less than 56, we exchange the two and step down one element. We then compare 32 and 8. Because 32 is not less than 8, we do not exchange these elements. We step down one element and compare 45 and 8. They are out of sequence, so we exchange them and step down again. Because we moved 8 down, it is now compared with 78, and these two elements are exchanged. Finally, 8 is compared with 23 and exchanged. This series of exchanges places 8 in the first location, and the wall is moved up one position.

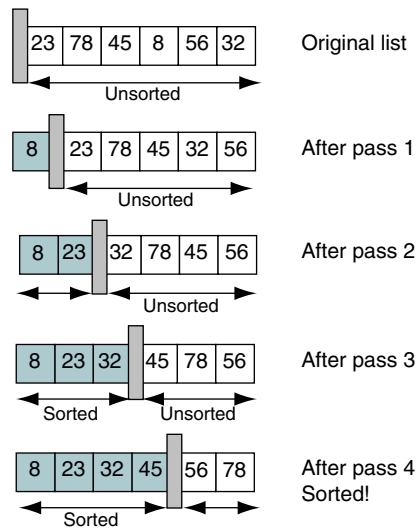


FIGURE 12-13 Bubble Sort Example

Bubble Sort Algorithm

Like the insertion and selection sorts, the bubble sort is quite simple. In each pass through the data, the smallest element is bubbled to the beginning of the unsorted segment of the array. In the process, adjacent elements that are out of order are exchanged, partially ordering the data. When the smallest element is encountered, it is automatically bubbled to the beginning of the unsorted list. The sort then continues by making another pass through the unsorted list. The code for the bubble sort is shown in Algorithm 12-5.

ALGORITHM 12-5 Bubble Sort

```
Algorithm  bubbleSort (list, last)
Sort an array using bubble sort. Adjacent
elements are compared and exchanged until list is
completely ordered.
```

continued

ALGORITHM 12-5 Bubble Sort *(continued)*

```
   Pre  list must contain at least one item
        last contains index to last element in the list
   Post list has been rearranged in sequence low to high
 1 set current to 0
 2 set sorted  to false
 3 loop (current <= last AND sorted false)
      Each iteration is one sort pass.
   1  set walker to last
   2  set sorted to true
   3  loop (walker > current)
      1  if (walker data < walker - 1 data)
            Any exchange means list is not sorted.
         1  set sorted to false
         2  exchange (list, walker, walker - 1)
      2  end if
      3  decrement walker
   4  end loop
   5  increment current
 4 end loop
end bubbleSort
```

Algorithm 12-5 Analysis

If you have studied other bubble sort algorithms, you may have noticed a slight improvement in this version of the sort. If an exchange is not made in a pass (statement 3), we know that the list is already sorted and the sort can stop. At statement 3.2 we set a Boolean, **`sorted`**, to true. If at any time during the pass an exchange is made, **`sorted`** is changed to false, indicating that the list was not sorted when the pass began.

Another difference you may have noticed is that we started from the high end and bubbled down. As a historical note, the bubble sort was originally written to "bubble up" the highest element in the list. From an efficiency point of view, it makes no difference whether the largest element is bubbled down or the smallest element is bubbled up. From a consistency point of view, however, comparisons between the sorts are easier if all three of our basic sorts (insertion, selection, and exchange) work in the same manner. For that reason we have chosen to bubble the lowest key in each pass.

Quick Sort

In the bubble sort, consecutive items are compared and possibly exchanged on each pass through the list, which means that many exchanges may be needed to move an element to its correct position. **Quick sort** is an exchange sort developed by C. A. R. Hoare in 1962. It is more efficient than the bubble sort because a typical exchange involves elements that are far apart, so fewer exchanges are required to correctly position an element.

Each iteration of the quick sort selects an element, known as **pivot**, and divides the list into three groups: a partition of elements whose keys are less than the pivot's key, the pivot element that is placed in its ultimately correct location in the list, and a partition of elements greater than or equal to the pivot's key. The sorting then continues by quick sorting the left

partition followed by quick sorting the right partition. This partitioning is shown in Figure 12-14.

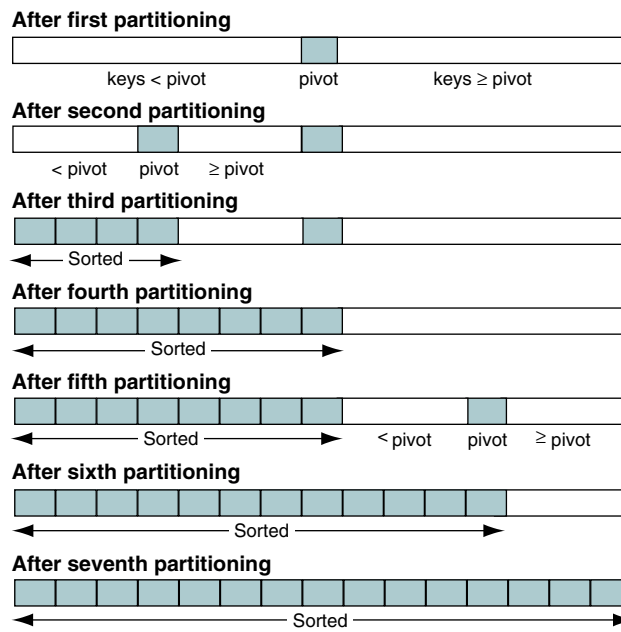


FIGURE 12-14 Quick Sort Partitions

Hoare's original algorithm selected the pivot key as the first element in the list. In 1969, R. C. Singleton improved the sort by selecting the pivot key as the median value of three elements: left, right, and an element in the middle of the list. Once the median value is determined, it is exchanged with the left element. We implement Singleton's variation of quick sort.

Quick sort is an exchange sort in which a pivot key is placed in its correct position in the array while rearranging other elements widely dispersed across the list.

Knuth suggests that when the sort partition becomes small a straight insertion sort be used to complete the sorting of the partition.[9] Although the mathematics to optimally choose the minimum partition size are quite complex, the partition size should be relatively small; we recommend 16.

9. Knuth, *Art of Computer Programming*, "Algorithm Q," 115.

Quick Sort Algorithm

We are now ready to develop the algorithm. As you may have anticipated from our discussion, we use a recursive algorithm for the quick sort. In addition to the basic algorithm, two supporting algorithms are required, one to determine the pivot element and one for the straight insertion sort. We discuss these two algorithms first.

Straight Insertion Module

The straight insertion sort is a minor variation on the algorithm developed earlier, which always sorted a list beginning at location 0. Because the partition to be sorted in the quick sort can be found anywhere in the array, we must be able to sort partitions beginning at locations other than 0. We therefore add a parameter that specifies the starting location of the location to be sorted in addition to its ending location. The modified code is shown in Algorithm 12-6.

ALGORITHM 12-6 Quick Sort's Straight Insertion Sort Module

```
Algorithm quickInsertion (list, first, last)
Sort array list using insertion sort. The list is
divided into sorted and unsorted lists. With each pass, the
first element in the unsorted list is inserted into the
sorted list. This is a special version of the insertion
sort modified for use with quick sort.
   Pre  list must contain at least one element
        first is an index to first element in the list
        last is an index to last element in the list
   Post list has been rearranged
1  set current to first + 1
2  loop (until current partition sorted)
   1  move current element to hold
   2  set walker to current - 1
   3  loop (walker >= first AND hold key < walker key)
      1  move walker element one element right
      2  decrement walker
   4  end loop
   5  move hold to walker + 1 element
   6  increment current
3  end loop
end quickInsertion
```

Determine Median of Three

The logic to select the median location requires three tests. First we test the left and middle elements; if they are out of sequence, we exchange them. Then we test the left and right elements; if they are out of sequence, we exchange them. Finally, we test the middle and right elements; if they are out of sequence, we exchange them. At this point the three elements are in order.

```
left element <= middle element <= right element
```

We see, therefore, that this logic is based on the logical proposition that if *a* is less than *b,* and *b* is less than *c*, then *a* is less than *c*. Finally, before we leave the algorithm we exchange the left and middle elements, thus positioning the median valued element at the left of the array. The pseudocode is shown in Algorithm 12-7.

ALGORITHM 12-7 Median Left

```
Algorithm medianLeft (sortData, left, right)
Find the median value of an array and place it in the first
(left) location.
   Pre   sortData is an array of at least three elements
         left and right are the boundaries of the array
   Post  median value located and placed left
   Rearrange sortData so median value is in middle location.
1  set mid to (left + right) / 2
2  if (left key > mid key)
   1   exchange (sortData, left, mid)
3  end if
4  if (left key > right key)
   1   exchange (sortData, left, right)
5  end if
6  if (mid key > right key)
   1   exchange (sortData, mid, right)
7  end if
   Median is in middle location. Exchange with left.
8  exchange (sortData, left, mid)
end medianLeft
```

Algorithm 12-7 Analysis

The logic to determine a median value can become unintelligible very quickly. The beauty of this algorithm is its simplicity. It approaches the determination of the median value by performing a very simple sort on the three elements, placing the median in the middle location. It then exchanges the middle element with the left element.

Quick Sort Algorithm

We now turn our attention to the quick sort algorithm itself. It contains an interesting program design that you will find useful in other array applications. To determine the correct position for the pivot element, we work from the two ends of the array toward the middle. Because we have used the median value of three elements to determine the pivot element, the pivot may end up near the middle of the array, although this is not guaranteed. Quick sort is most efficient when the pivot's location is the middle of the array. The technique of working from the ends to the middle is shown in Figure 12-15.

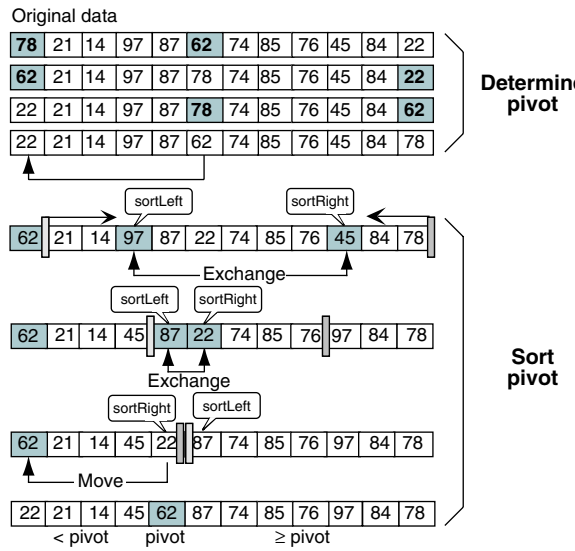


FIGURE 12-15 Quick Sort Pivot

As you study Figure 12-15, note that before the exchanges start, the median element is in the middle position and the smallest of the three elements used to determine the median is in the right location. After calling the median left algorithm, the median is in the left position and the smallest is in the middle location. The pivot key is then moved to a hold area to facilitate the processing. This move is actually the first part of an exchange that puts the pivot key in its correct location.

To help follow the sort, envision two walls, one on the left just after the pivot and one on the right (see Figure 12-15). We start at the left wall and move right while looking for an element that belongs on the right of the pivot. We locate one at element 97. After finding an element on the left that belongs on the right, we start at the right wall and move left while looking for an element that belongs on the left of the pivot. We find one at element 45. At this point we exchange the two elements, move the walls to the left of element 87 and the right of element 76, and repeat the process.

This time, as we move to the right, the first left element, 87, belongs on the right. When we move down from the right, we find that element 22 belongs on the left. Again, we exchange the left and right elements and move the wall. At this point we note that the walls have crossed. We have thus located the correct position for the pivot element. We move the data in the pivot's location to the first element of the array and then move the pivot back to the array, completing the exchange we started when we moved the pivot key to the hold area. The list has now been rearranged so that the pivot element (62) is in its correct location in the array relative to all of the other

elements in the array. The elements on the left of the pivot are all less than the pivot, and the elements on the right of the pivot are all greater than the pivot. Any equal data is found on the right of the pivot.

After placing the pivot key in its correct location, we recursively call quick sort to sort the left partition. When the left partition is completely sorted, we recursively call quick sort to sort the right partition. When both the left partition and the right partition have been sorted, the list is completely sorted. The data in Figure 12-15 are completely sorted using only quick sort in Figure 12-16.

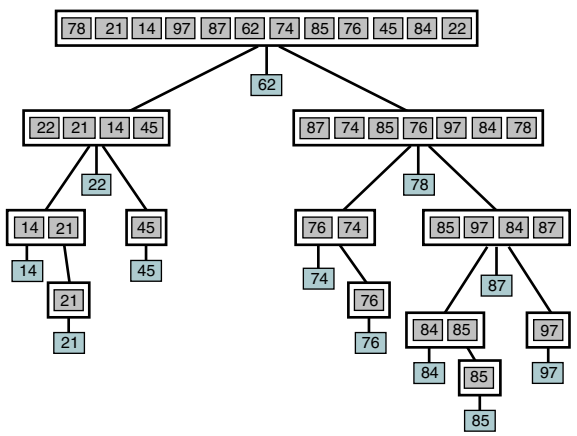


FIGURE 12-16 Quick Sort Operation

The quick sort pseudocode is shown in Algorithm 12-8.

ALGORITHM 12-8 Quick Sort

```
Algorithm quickSort (list, left, right)
An array, list, is sorted using recursion.
   Pre  list is an array of data to be sorted
        left and right identify the first and last
        elements of the list, respectively
   Post list is sorted
 1 if ((right - left) > minSize)
      Quick sort
   1  medianLeft (list, left, right)
   2  set pivot     to left element
   3  set sortLeft  to left + 1
   4  set sortRight to right
   5  loop (sortLeft    <= sortRight)
         Find key on left that belongs on right
```

continued

ALGORITHM 12-8 Quick Sort *(continued)*

```
      1  loop (sortLeft key < pivot key)
         1  increment sortLeft
      2  end loop
         Find key on right that belongs on left
      3  loop (sortRight key >= pivot key)
         1  decrement sortRight
      4  end loop
      5  if (sortLeft <= sortRight)
         1  exchange(list, sortLeft, sortRight)
         2  increment sortLeft
         3  decrement sortRight
      6  end if
   6  end loop
      Prepare for next pass
   7  move sortLeft - 1 element to left element
   8  move pivot element to sortLeft - 1 element
   9  if  (left < sortRight)
      1  quickSort (list, left, sortRight - 1)
  10  end if
  11  if (sortLeft < right)
      1  quickSort (list, sortLeft, right)
  12  end if
 2 else
   1  insertionSort (list, left, right)
 3 end if
end quickSort
```

Algorithm 12-8 Analysis

In addition to the design that works from both ends to the middle, two aspects of this algorithm merit further discussion. The loops used to determine the sort left and sort right elements (statements 1.5.1 and 1.5.3) test only one condition. Most loops that process an array must also test for the end of the array, but we can omit the test in this algorithm based on selection of the pivot key. The pivot key is guaranteed to be the median value of three elements: the first, the last, and the one in the middle. Therefore, the median key cannot be less than the far-left key nor greater than the far-right key. At worst it is equal to the far-left or far-right key.

Assume that we have the worst case: all of the elements in a partition have the same key value. In this case the pivot key is equal to the far-left and the far-right keys. When we start on the left to move up the list, we stop immediately because the pivot key is equal to the **`sortLeft`** key. When we then start on the right and move down, we move beyond the beginning of the list when all keys are equal. However, because the two elements have crossed, we do not exchange elements. Rather, we move to statement 1.7. In this case we never use the sort right index that has moved off the beginning of the array.

Hoare's original algorithm was not recursive. Because he had to maintain a stack, he incorporated logic to ensure that the stack size was kept to a minimum. Rather than simply sort the left partition, he determined which partition was larger and put it in the stack while he sorted the smaller partition. We are not concerned with minimizing the stack size for two reasons. First, we have chosen the pivot key to be the median value.

Therefore, the size of the two partitions should be generally the same, thus minimizing the number of recursive calls. More important, because we are using recursion, we do not need to determine the size of the stack in advance. We simply call on the system to provide stack space.

Exchange Sort Efficiency

In the exchange sorts, we find what Knuth called the best general-purpose sort: quick sort. Let's determine their sort efforts to see why.

Bubble Sort

The code for the bubble sort is shown in Algorithm 12-5, "Bubble Sort." As we saw with the straight insertion and the straight selection sorts, it uses two loops to sort the data. The loops are shown below:

```
3   loop (current <= last AND sorted false)
    ...
    3   loop (walker > current)
```

The outer loop tests two conditions: the current index and a sorted flag. Assuming that the list is not sorted until the last pass, we loop through the array n times. The number of loops in the inner loop depends on the current location in the outer loop. It therefore loops through half the list on the average. The total number of loops is the product of both loops, making the bubble sort efficiency

$$n\left(\frac{n+1}{2}\right)$$

In big-O notation the bubble sort efficiency is $O(n^2)$.

The bubble sort efficiency is $O(n^2)$.

Quick Sort

The code for the quick sort is shown in Algorithm 12-8, "Quick Sort." A quick look at the algorithm reveals that there are five loops (three iterative loops and two recursive loops). The algorithm also contains the straight insertion sort as a subroutine. Because it is possible to use the quick sort to completely sort the data—that is, it is possible to eliminate the insertion sort—we analyze the quick sort portion as though the insertion sort weren't used. The quick sort loops are shown below:

```
    5  loop (sortLeft <= sortRight)
       1   loop (sortLeft key < pivot key)
           ...
```

continued

```
      2   end loop
      3   loop (sortRight key >= pivot key)
          ...
      4   end loop
      5   ...
   6  end loop
      ...
   9  if (left < sortRight)
      1   quickSort (list, left, sortRight - 1)
  10  end if
  11  if (sortLeft < right)
      1   quickSort (list, sortLeft, right)
  12  if (sortLeft < right)
```

Recall that each pass in the quick sort divides the list into three parts: a list of elements smaller than the pivot key, the pivot key, and a list of elements greater than the pivot key. The first loop (statement 5), in conjunction with the two nested loops (statements 5.1 and 5.3), looks at each element in the portion of the array being sorted. Statement 5.1 loops through the left portion of the list; statement 5.3 loops through the right portion of the list. Together, therefore, they loop through the list n times.

Similarly, the two recursive loops process one portion of the array each, either on the left or the right of the pivot key. The question is how many times they are called. Recall that we said that quick sort is most efficient when the pivot key is in the middle of the array. That's why we used a median value. Assuming that it is located relatively close to the center, we see that we have divided the list into two sublists of roughly the same size. Because we are dividing by 2, the number of loops is logarithmic. The total sort effort is therefore the product of the first loop times the recursive loops, or $n \log n$.

The quick sort efficiency is $O(n \log n)$.

Algorithmics does not explain why we use the straight insertion sort when the list is small. The answer lies in the algorithm code and the overhead of recursion. When the list becomes sufficiently small, it is simply more efficient to use a straight insertion sort.

Summary

Our analysis leads to the conclusion that the quick sort's efficiency is the same as that of the heap sort. This is true because big-O notation is only an approximation of the actual sort efficiency. Although both are on the order of $n \log n$, if we were to develop more-accurate formulas to reflect their actual efficiency, we would see that the quick sort is actually more efficient. Table 12-3 summarizes the six sorts we discuss in this chapter.

n	Number of loops		
	Straight insertion Straight selection Bubble sorts	Shell	Heap and quick
25	625	55	116
100	10,000	316	664
500	250,000	2364	4482
1000	1,000,000	5623	9965
2000	4,000,000	13,374	10,965

TABLE 12-3 Sort Comparisons

We recommend that you use the straight insertion sort for small lists and the quick sort for large lists. Although the shell sort is an interesting sort that played an important role in the history of sorts, we would not recommend it in most cases. As you will see when we discuss external sorting, algorithms such as the heap sort play an important niche role in special situations and for that reason also belong in your sorting tool kit.

Exchange Sort Implementation

Let's look at the C code for the bubble sort and the quick sort algorithms.

Bubble Sort Code

The bubble sort code is shown in Program 12-5.

PROGRAM 12-5 Bubble Sort

```
 1  /* ================== bubbleSort ====================
 2     Sort list using bubble sort. Adjacent elements are
 3     compared and exchanged until list is ordered.
 4        Pre  list must contain at least one item
 5             last contains index to last list element
 6        Post list has been sorted in ascending sequence
 7  */
 8  void bubbleSort (int list [], int last)
 9  {
10  // Local Definitions
11     int temp;
12
```

continued

Chapter 13
Searching

One of the most common and time-consuming operations in computer science is **searching,** the process used to find the location of a target among a list of objects. In this chapter we study several search algorithms. We begin with list searching and a discussion of the two basic search algorithms: the sequential search—including three interesting variations—and the binary search.

After reviewing the concepts of list searches, we discuss hashed list searching, in which the data key is algorithmically manipulated to calculate the location of the data in the list. An integral part of virtually all hashed list algorithms is collision resolution, which we discuss in the last section.

Although we discuss the list search algorithms using an array structure, the same concepts can be found in linked list searches. The sequential search, along with the ordered list variation, is most commonly used to locate data in a list (Chapter 5). The binary search tree (Chapter 7) is actually a structure built to provide the efficiency of the binary search of a tree structure. These searches are covered in their respective chapters.

13.1 List Searches

The algorithm used to search a list depends to a large extent on the structure of the list. In this section we study searches that work with arrays. The two basic searches for arrays are the sequential search and the binary search. The sequential search can be used to locate an item in any array. The binary search, on the other hand, requires an ordered list. The basic search concept is shown in Figure 13-1.

Sequential Search

The **sequential search** is used whenever the list is not ordered. Generally, you use this technique only for small lists or lists that are not searched often. In other cases you should first sort the list and then search it using the binary search, discussed later.



FIGURE 13-1 Search Concept

In the sequential search, we start searching for the target at the beginning of the list and continue until we find the target or we are sure that it is not in the list. This approach gives us two possibilities: either we find it or we reach the end of the list. In Figure 13-2 we trace the steps to find the value 14. We first check the data at index 0, then 1, and then 2 before finding 14 in the fourth element (index 3).

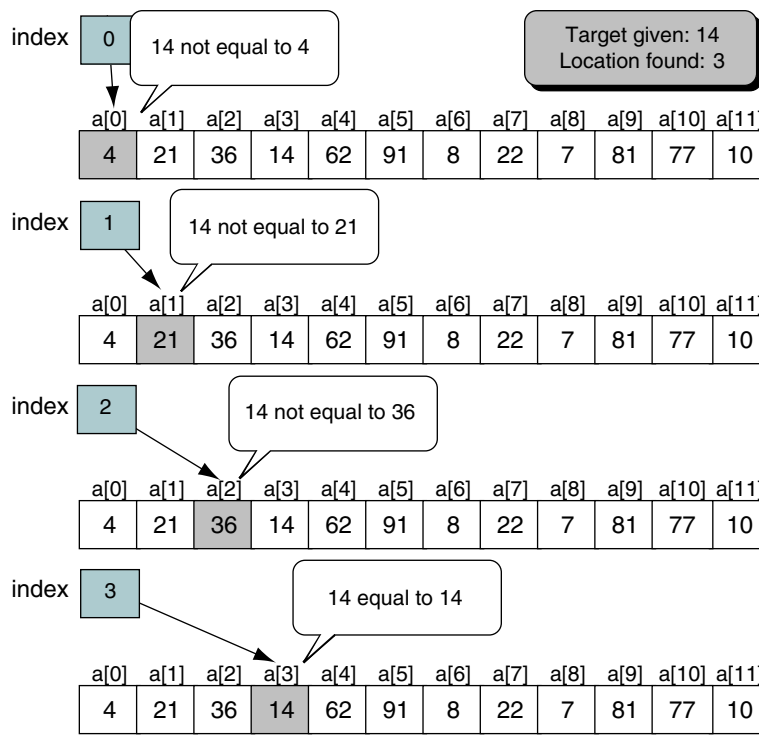
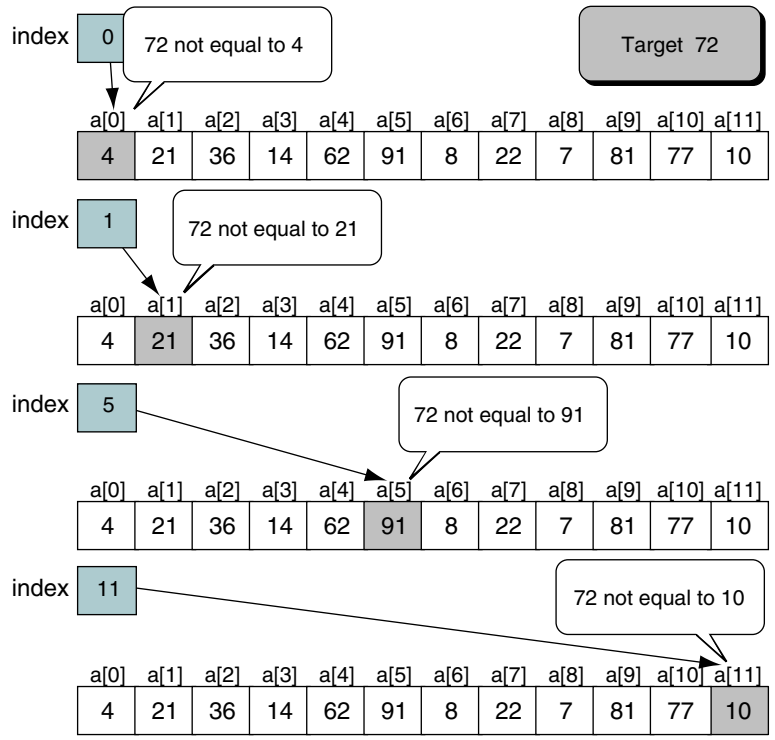


FIGURE 13-2 Successful Search of an Unordered List

But what if the target were not in the list? In that case we would have to examine each element until we reach the end of the list. Figure 13-3 traces the search for a target of 72. At the end of the list, we discover that the target does not exist.



Note: Not all test points are shown.

FIGURE 13-3 Unsuccessful Search in Unordered List

Sequential Search Algorithm

The sequential search algorithm needs to tell the calling algorithm two things: First, did it find the data it was looking for? And, second, if it did, at what index are the target data found? To answer these questions, the search algorithm requires four parameters: (1) the list we are searching, (2) an index to the last element in the list,[1] (3) the target, and (4) the address where the found element's index location is to be stored. To tell the calling algorithm whether the data were found, we return a Boolean—true if we found it or false if we didn't find it.

Although we could write a sequential search algorithm without passing the index to the last element, if we did so the search would have to know how

1. As an alternative to the index to the last element, the size of the list may be passed.

many elements are in the list. To make the function as flexible as possible, we pass the index of the last data value in the array. Generalizing the algorithm by passing the index to the last item is also a good structured design technique. With this information we are now ready to code Algorithm 13-1.

ALGORITHM 13-1 Sequential Search

```
Algorithm seqSearch (list, last, target, locn)
Locate the target in an unordered list of elements.
   Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found: index stored in locn & found true
          if not found: last stored in locn & found false
   Return found true or false
 1 set looker to 0
 2 loop (looker < last AND target not equal list[looker])
   1   increment looker
 3 end loop
 4 set locn to looker
 5 if (target equal list[looker])
   1   set found to true
 6 else
   1   set found to false
 7 end if
 8 return found
end seqSearch
```

Algorithm 13-1 Analysis

We have only one comment. We could have used the location parameter for the search rather than create the `looker` index. However, it is generally not a good idea to use a parameter as a working variable because doing so destroys any initial value that may be needed later in the algorithm.

Variations on Sequential Searches

Three useful variations on the sequential search algorithm are: (1) the sentinel search, (2) the probability search, and (3) the ordered list search. We look at each briefly in the following sections.

Sentinel Search

If you examine the search algorithm carefully, you note that the loop tests two conditions: the end of the list and the target's not being found. Knuth states, "When the inner loop of a program tests two or more conditions, we should try to reduce the testing to just one condition."[2] If we know that the target will be found in the list, we can eliminate the test for the end of the list. The

2. Donald E. Knuth, *The Art of Computer Programming,* vol. 3, *Sorting and Searching,* Second Edition (Reading, MA: Addison-Wesley, 1998), 398.

only way we can ensure that a target is actually in the list is to put it there ourself. A target is put in the list by adding an extra element (sentinel entry) at the end of the array and placing the target in the sentinel. We can then optimize the loop and determine after the loop completes whether we found actual data or the sentinel.[3] The obvious disadvantage is that the rest of the processing must be careful to never look at the sentinel element at the end of the list. The pseudocode for the sentinel search is shown in Algorithm 13-2.

ALGORITHM 13-2 Sentinel Search

```
Algorithm SentinelSearch (list, last, target, locn)
Locate the target in an unordered list of elements.
   Pre    list must contain element at the end for sentinel
          last is index to last data element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn & found
             set true
          if not found--last stored in locn & found false
   Return found true or false
 1 set list[last + 1] to target
 2 set looker to 0
 3 loop (target not equal list[looker])
   1   increment looker
 4 end loop
 5 if (looker <= last)
   1   set found to true
   2   set locn  to looker
 6 else
   1   set found to false
   2   set locn  to last
 7 end if
 8 return found
end SentinelSearch
```

Probability Search

One of the more useful variations of the sequential search is known as the **probability search**. In the probability search, the data in the array are arranged with the most probable search elements at the beginning of the array and the least probable at the end. It is especially useful when relatively few elements are the targets for most of the searches. To ensure that the probability ordering is correct over time, in each search we exchange the located element with the element immediately before it in the array. A typical implementation of the probability search is shown in Algorithm 13-3.

3. It is not always possible to reduce a loop to only one test. We will see many loops that require two or more tests to satisfy the logic.

ALGORITHM 13-3 Probability Search

```
Algorithm ProbabilitySearch (list, last, target, locn)
Locate the target in a list ordered by the probability of each
element being the target--most probable first, least probable
last.
   Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
          locn is address of index in calling algorithm
   Post   if found--matching index stored in locn,
             found true, and element moved up in priority.
          if not found--last stored in locn & found false
   Return found true or false
 1 find target in list
 2 if (target in list)
   1   set found to true
   2   set locn to index of element containing target
   3   if (target after first element)
       1  move element containing target up one location
   4   end if
 3 else
   1   set found to false
 4 end if
 5 return found
end ProbabilitySearch
```

Ordered List Search

Although we generally recommend a binary search when searching a list ordered on the key (target), if the list is small it may be more efficient to use a sequential search. When searching an ordered list sequentially, however, it is not necessary to search to the end of the list to determine that the target is not in the list. We can stop when the target becomes less than or equal to the current element we are testing. In addition, we can incorporate the sentinel concept by bypassing the search loop when the target is greater than the last item. In other words, when the target is less than or equal to the last element, the last element becomes a sentinel, allowing us to eliminate the test for the end of the list.

Although it can be used with array implementations, the ordered list search is more commonly used when searching linked list implementations. The pseudocode for searching an ordered array is found in Algorithm 13-4.

ALGORITHM 13-4 Ordered List Search

```
Algorithm OrderedListSearch (list, last, target, locn)
Locate target in a list ordered on target.
Pre    list must contain at least one element
          last is index to last element in the list
          target contains the data to be located
```

continued

ALGORITHM 13-4 Ordered List Search *(continued)*

```
            locn is address of index in calling algorithm
   Post     if found--matching index stored in locn-found true
            if not found--locn is index of first element >
               target or locn equal last & found is false
   Return found true or false
 1 if (target less than last element in list)
   1   find first element less than or equal to target
   2   set locn to index of element
 2 else
   1   set locn to last
 3 end if
 4 if (target in list)
   1   set found to true
 5 else
   1   set found to false
 6 end if
 7 return found
end OrderedListSearch
```

Binary Search

The sequential search algorithm is very slow. If we have an array of 1000 elements, we must make 1000 comparisons in the worst case. If the array is not sorted, the sequential search is the only solution. However, if the array is sorted, we can use a more efficient algorithm called the **binary search**. Generally speaking, we should use a binary search whenever the list starts to become large. Although the definition of *large* is vague, we suggest that you consider binary searches whenever the list contains more than 16 elements.

The binary search starts by testing the data in the element at the middle of the array to determine if the target is in the first or the second half of the list. If it is in the first half, we do not need to check the second half. If it is in the second half, we do not need to test the first half. In other words, we eliminate half the list from further consideration with just one comparison. We repeat this process, eliminating half of the remaining list with each test, until we find the target or determine that it is not in the list.

To find the middle of the list, we need three variables: one to identify the beginning of the list, one to identify the middle of the list, and one to identify the end of the list. We analyze two cases here: the target is in the list and the target is not in the list.

Target Found

Figure 13-4 traces the binary search for a target of 22 in a sorted array.

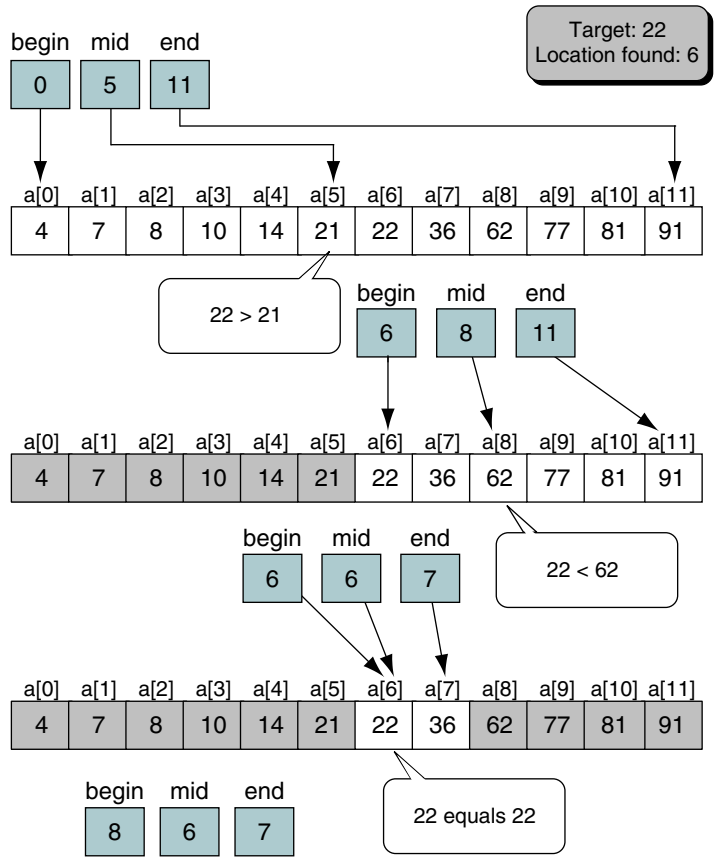


FIGURE 13-4 Successful Binary Search Example

We descriptively call our three indexes `begin`, `mid`, and `end`. Given `begin` as 0 and `end` as 11, we can calculate `mid` as follows:

$$\texttt{mid} = \lfloor (\texttt{begin} + \texttt{end}) \ / \ 2 \rfloor = \lfloor (0 + 11) \ / \ 2 \rfloor = 5$$

At index location 5, we discover that the target is greater than the list value (22 > 21). We can therefore eliminate the array locations 0 through 5. (Note that `mid` is automatically eliminated.) To narrow our search, we set `mid + 1` to `begin` and repeat the search.

The next loop calculates `mid` with the new value for `begin` (6) and determines that the midpoint is now 8.

```
mid = ⌊(6 + 11) / 2⌋ = ⌊17 / 2⌋ = 8
```

When we test the target to the value at `mid` a second time, we discover that the target is less than the list value (22 < 62). This time we adjust the end of the list by setting `end` to `mid - 1` and recalculating `mid`.

```
mid = ⌊(6 + 7) / 2⌋ = ⌊13 / 2⌋ = 6
```

This step effectively eliminates elements 8 through 11 from consideration. We have now arrived at index location 6, whose value matches our target. To stop the search in the algorithm (see page 607), we force `begin` to be greater than `end`.

Target Not Found

A more interesting case occurs when the target is not in the list. We must construct our search algorithm so that it stops when we have checked all possible locations. We do this in the binary search by testing for the `begin` and `end` indexes crossing; that is, we are done when `begin` becomes greater than `end`. We now see that two conditions terminate the binary search algorithm: the target is found or it is not found. To terminate the loop when it is found, we force `begin` to be greater than `end`. When the target is not in the list, `begin` becomes larger than `end` automatically.

Let's demonstrate the target's not being found with an example. In Figure 13-5 we search for a target of 11, which doesn't exist in the array.

In this example the loop continues to narrow the range as we saw in the successful search until we are examining the data at index locations 3 and 4. These settings of `begin` and `end` set the `mid` index to 3.

```
mid = ⌊(3 + 4) / 2⌋ = ⌊7 / 2⌋ = 3
```

The test at index location 3 indicates that the target is greater than the list value, so we set `begin` to `mid + 1`, or 4. We now test the data at location 4 and discover that 11 < 14.

```
mid = ⌊(4 + 4) / 2⌋ = ⌊8 / 2⌋ = 4
```

At this point we have discovered that the target should be between two adjacent values; in other words, it is not in the list. We see this algorithmically because `end` is set to `mid - 1`, which makes `begin` greater than `end`, the signal that the value we are looking for is not in the list.

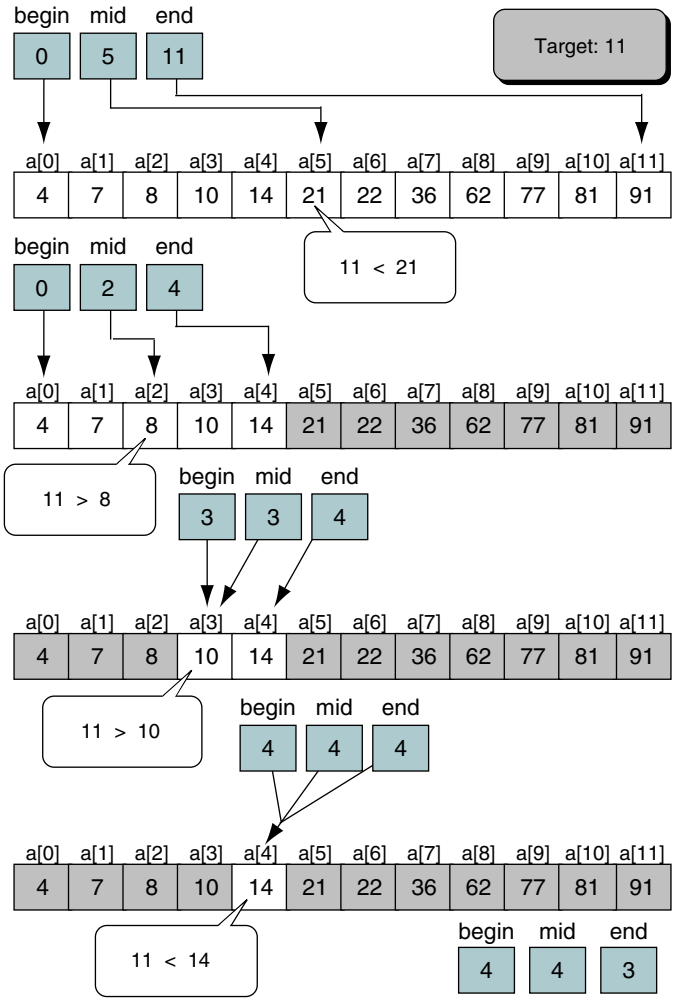


FIGURE 13-5 Unsuccessful Binary Search Example

Binary Search Algorithm

Algorithm 13-5 contains the implementation of the binary search algorithm we have been describing. It is constructed along the same design we saw for the sequential search. The first three parameters describe the list and the target we are looking for, and the last one contains the address into which we place the located index. One point is worth noting: when we terminate the loop with a not-found condition, the index returned is unpredictable—it may indicate that the node is greater than or less than the value in the target.

ALGORITHM 13-5 Binary Search

```
Algorithm binarySearch (list, last, target, locn)
Search an ordered list using Binary Search
   Pre    list is ordered; it must have at least 1 value
          last is index to the largest element in the list
          target is the value of element being sought
          locn is address of index in calling algorithm
   Post   FOUND: locn assigned index to target element
                 found set true
          NOT FOUND: locn = element below or above target
                     found set false
   Return found true or false
1  set begin to 0
2  set end to last
3  loop (begin <= end)
   1   set mid to (begin + end) / 2
   2   if (target > list[mid])
          Look in upper half
       1   set begin to (mid + 1)
   3   else if (target < list[mid])
          Look in lower half
       1   set end to mid - 1
   4   else
          Found: force exit
       1   set begin to (end + 1)
   5   end if
4  end loop
5  set locn to mid
6  if (target equal list [mid])
   1   set found to true
7  else
   1   set found to false
8  end if
9  return found
end binarySearch
```

Analyzing Search Algorithms

Of the search algorithms discussed, which is the best? An application often determines which algorithm should be used, but we analyze the algorithms to determine which is most efficient.

Sequential Search

The basic loop for the sequential search is shown below.

```
2 loop (looker < last AND target not equal list[looker])
      1 increment looker
```

This is a classic example of a linear algorithm. In fact, in some of the literature this search is known as a **linear search**. Because the algorithm is linear, its efficiency is $O(n)$.

> The efficiency of the sequential search is $O(n)$.

The search efficiency for the sentinel search is basically the same as for the sequential search. Although the sentinel search saves a few instructions in the loop, its design is identical. Therefore, it is also an $O(n)$ search. Likewise, the efficiency of the ordered list search is also $O(n)$. If we know the probability of a search's being successful, we can construct a more accurate formula for searching an ordered list. This improved accuracy, however, turns out to be a coefficient in the formula, which, as you recall, is dropped when using big-O analysis.

It is not possible to generalize the efficiency of the probability search without knowing the probability of each element in the list. On the other hand, if the probability of the first few elements of a list totals more than 90%, it can be a very efficient search, even considering the additional overhead required to maintain the list. In general, however, we recommend the binary search, especially for large lists.

Binary Search

The binary search locates an item by repeatedly dividing the list in half. Its loop is:

```
3  loop (begin <= end)
   1  set mid to (begin + end) / 22  if (target > list[mid])
      1  set begin to (mid + 1)
   3  else if (target < list[mid])
      1  set end to mid - 1
   4  else
      1  set begin to (end + 1)
   5  end if
4  end loop
```

This loop obviously divides, and it is therefore a logarithmic loop. The efficiency is thus $O(\log n)$, which you should recognize as one of the most efficient of all the measures.

> The efficiency of the binary search is $O(\log n)$.

Comparing the two searches, we see that, disregarding the time required to order the list, the binary search is obviously more efficient for searching a list of any significant size (see Table 13-1). For this reason the binary search is recommended for all but the smallest of lists (i.e., lists with fewer than 16 elements).

List size	Iterations	
	Binary	Sequential
16	4	16
50	6	50
256	8	256
1000	10	1000
10,000	14	10,000
100,000	17	100,000
1,000,000	20	1,000,000

TABLE 13-1 Comparison of Binary and Sequential Searches

13.2 Search Implementations

In this section we develop the C code for the sequential and binary searches. As we saw with sorts, we can write a generic search algorithm for any type of data. To allow us to concentrate on the search algorithms, we develop them using only integer data.

Sequential Search in C

The sequential search in Program 13-1 parallels the pseudocode implementation in Algorithm 13-1.

PROGRAM 13-1 Sequential Search

```
 1  /* Locate target in an unordered list of size elements.
 2        Pre  list must contain at least one item
 3             last is index to last element in list
 4             target contains the data to be located
 5             locn is address of index in calling function
 6        Post FOUND: matching index stored in locn address
 7                    return true (found)
 8             NOT FOUND: last stored in locn address
 9                        return false (not found)
10  */
11  bool seqSearch (int list[ ], int  last,
12                  int target,  int* locn)
13  {
14  // Local Definitions
15     int looker;
```

continued

PROGRAM 13-1 Sequential Search *(continued)*

```
16
17  // Statements
18     looker = 0;
19     while (looker < last && target != list[looker])
20        looker++;
21
22     *locn = looker;
23     return ( target == list[looker] );
24  }  // seqSearch
```

Program 13-1 Analysis Although this program is simple, it merits some discussion. First, why did we use a *while* statement rather than a *for* loop? Even though we know the limits of the array, it is still an event-controlled loop. We search until we find what we are looking for or reach the end of the list. Finding something is an event, so we use an event loop.

Next, note that there are two tests in the limit expression of the loop. We have coded the test for the end of the array first. In this case it doesn't make any difference which test is first from an execution standpoint, but in other search loops it might. Therefore, you should develop the habit of coding the limit test first because it doesn't use an indexed value and is therefore safer.

The call-by-address use for **`locn`** also merits discussion. Because we need to pass the found location back to the variable in the calling program, we need to pass its address to the search.

Notice how succinct this function is. In fact, there are more lines of documentation than there are lines of code. The entire search is contained in one *while* statement. With this short code, you might be tempted to ask, "Why write the function at all? Why not just put the one line of code wherever it is needed?" The answer lies in the structured programming concepts that each function should do only one thing and in the concept of reusability. By isolating the search process in its own function, we separate it from the process that needs the search. This is a better approach to structured programming and also makes the code reusable in other parts of the program and portable to other programs that require searches.

Binary Search In C

The C implementation of the binary search algorithm is shown in Program 13-2.

PROGRAM 13-2 Binary Search

```
1  /* Search an ordered list using Binary Search.
2       Pre  list must contain at least one element
3            last is index to largest element in list
4            target is value of element being sought
5            locn is address of index in calling function
6       Post FOUND: locn = index to target element
7                    return true (found)
8            NOT FOUND: locn = index below/above target
9                       return false (not found)
```

continued

PROGRAM 13-2 Binary Search *(continued)*

```
10  */
11  bool binarySearch   (int list[ ], int  last,
12                       int target,  int* locn )
13  {
14  // Local Definitions
15     int begin;
16     int mid;
17     int end;
18
19  // Statements
20     begin = 0;
21     end  = last;
22     while (begin <= end)
23        {
24        mid = ( begin + end ) / 2;
25        if ( target > list[ mid ] )
26           // look in upper half
27           begin = mid + 1;
28           else if ( target < list[ mid ] )
29              // look in lower half
30              end = mid - 1;
31        else
32              // found: force exit
33              begin = end + 1;
34        } // end while
35     *locn = mid;
36     return (target == list [mid]);
37  }  // binarySearch
```

13.3 Hashed List Searches

The search techniques discussed in Section 13.1, "List Searches," require several tests before we can find the data. In an ideal search, we would know exactly where the data are and go directly there. This is the goal of a hashed search: to find the data with only one test. For our discussion we use an array of data. The general concept is easily extended to other structures, such as files stored on a disk. These structures are beyond the scope of this text, however, and their discussion is left to other books.

Basic Concepts

In a **hashed search**, the key, through an algorithmic function, determines the location of the data. Because we are searching an array, we use a hashing algorithm to transform the key into the index that contains the data we need to locate. Another way to describe hashing is as a key-to-address transformation in which the keys map to addresses in a list. This mapping transformation is

Appendix E
Mathematical Series and Recursive Relations

To understand how the efficiency of an algorithm is computed, you need to have a basic understanding of how a mathematical series works. Because so much of algorithmic design involves recursion, the effect of recursion on a series must also be understood.

This appendix explores four basic concepts behind mathematical series and recursive relations: arithmetic series, geometric series, harmonic series, and recursive relations.

E.1 Arithmetic Series

In mathematics an arithmetic series is defined as shown in Figure E-1.

$$S_a = \sum_{i=1}^{n} i \,.\, x = x + 2x + \ldots + (n-1)x + nx$$

FIGURE E-1 Arithmetic Series

We present an informal solution to this series by writing it twice, first in its normal order and then in the reverse order of terms. To get equal terms, we add them as shown in Figure E-2.

$$
\begin{array}{rcccccccccc}
S_a = & x & + & 2x & + & \ldots & + & (n-1)x & + & nx \\
S_a = & nx & + & (n-1)x & + & \ldots & + & 2x & + & x \\
\hline
2S_a = & (n+1)x & + & (n+1)x & + & \ldots & + & (n+1)x & + & (n+1)x \\
 & \multicolumn{9}{c}{\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{n \text{ terms}}}
\end{array}
$$

Result: $S_a = \dfrac{n(n+1)x}{2}$

FIGURE E-2 Arithmetic Series Solution

Example Find the sum of the following series:

```
1 + 2 + 3 + 4 + ... + 100
```

This is the arithmetic series with `x = 1` and `n = 100`. Although we could add all of the numbers, it is much easier to apply the series formula.

```
Sa = 100(101) / 2 = 5050
```

Example In this example we use a series where `x` is not `1`.

```
5 + 10 + 15 + 20 + ... + 50
```

In this series, we see that each number increases by 5, which means `x = 5`. We use the general formula

```
Sa = xn (n + 1) / 2 = 5 (10)(11) / 2 = 275
```

Example For our final example, we find the sum of a series where `x` is not `1` and `n` does not start with `1`. Find the following sum:

```
100 + 105 + 110 + 115 + ... + 200
```

To be able to use our formula, we need to write this series as the combination of two series.

```
(5 + 10 + 15 + ... + 200) - (5 + 10 + 15 + ... + 95)
```

The first series (from 5 to 200) is an arithmetic series with `x = 5` and `n = 40`. The second series is an arithmetic series with `x = 5` and `n = 19`. Using our formula, we see that

```
Sa = ( 5 (40)(41) / 2) - 5(19)(20) / 2 = 3150
```

E.2 Geometric Series

In mathematics a geometric series is defined as shown in Figure E-3.

$$S_g = \sum_{i=0}^{n} x^i = 1 + x + x^2 + \ldots + x^{(n-1)} + x^n$$

FIGURE E-3 Geometric Series

Again, we present an informal solution to this series as shown in Figure E-4.

Given

$$S_g = 1 + x + x^2 + \ldots + x^{(n-1)} + x^n$$

We multiply both sides by x

$$x \cdot S_g = x + x^2 + x^3 + \ldots + x^n + x^{(n+1)}$$

We add and subtract 1 from right side

$$x \cdot S_g = \underbrace{1 + x + x^2 + x^3 + \ldots + x^n}_{S_g} + x^{(n+1)} - 1$$

After substituting part of the right side with S_g

$$x \cdot S_g = S_g + x^{(n+1)} - 1$$

After reordering and factoring

$$S_g = \frac{x^{(n+1)} - 1}{x - 1}$$

FIGURE E-4 Geometric Series Solution

Example Find the sum of the following series:

```
1 + 2 + 4 + 8 + ... + 1024
```

We recognize that this is a geometric series in which x is 2 (the series is powers of 2) and n is 10 (2^{10} is 1024). Using the geometric formula, we get

$$S_g = (x^{n+1} - 1) / (x - 1) = (2^{11} - 1) / (2 - 1) = 2047$$

Example Find the sum of the following series:

$$1 + 1/2 + 1/4 + 1/8 + \ldots$$

This is a case of the geometric formula in which x is 1/2. One property of the geometric series is that it converges if x is less than 1 and n approaches infinity. In this case x^{n+1} becomes 0 because a number less than 1 to the power infinity is 0. This is shown in this example in which x is less than 1 and n is infinity.

$$\begin{aligned} S_g &= (x^{n+1} - 1) / (x - 1) \\ &= (-1) / (x - 1) \\ &= 1 / (1 - x) \\ &= 1 / (1 - 1/2) \\ &= 2 \end{aligned}$$

E.3 Harmonic Series

Figure E-5 shows a harmonic series. This series does not converge, which means that the value of H_n does not get close to 0 when n gets close to infinity. Although the exact calculation of this series is beyond the scope of this text, we provide the approximation shown in the figure.

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

Result: $H_n = \ln n$

FIGURE E-5 Harmonic Series

Example Find the sum of the following series.

$$1 + 1/2 + 1/3 + \ldots + 1/100$$

The answer is ln 100 or 4.60.

E.4 Recursive Relations

To find the complexity of recursive algorithms, we need to solve a recursive relation. A recursive relation is one in which the value of a function is defined in terms of itself. For example,

```
f(n) = f(n - 1) + 1
```

is a recursive relation because the value of `f(n)` is defined in terms of `f(n - 1)`. Although there is not a general formula for solving these types of relations, the general approach requires that we develop the relations for `n`, `n - 1`, `n - 2`, `n - 3`, and so on. In this process we try to eliminate terms that involve `f(i)` so that on the right side we have only `f(n)` and on the left side only constants. Note that for a relation to be solvable, some initial values of `f` must be provided. The most common given is the value of `f(0)` or `f(1)`. We develop the concept heuristically through examples.

Example Figure E-6 shows the recursive solution for a recursive relationship.

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ f(n-1) + 1 & \text{Otherwise} \end{cases}$$

We substitute n with n – 1 repeatedly

$$\begin{aligned} f(n) &= f(n-1) + 1 \\ f(n-1) &= f(n-2) + 1 \\ f(n-2) &= f(n-3) + 1 \\ &\dots \\ f(1) &= f(0) + 1 \end{aligned}$$

When we add the equations, some terms are canceled

$$f(n) = f(0) + \underbrace{1 + 1 + \dots + 1}_{n}$$

The base case, f(0) = 0, so we have

$$f(n) = n$$

FIGURE E-6 Solution for First Recursive Relation

Example Figure E-7 shows another recursive relation and its solution.

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ f(n-1) + n & \text{Otherwise} \end{cases}$$

We substitute n with n – 1 repeatedly

$$\begin{aligned} f(n) &= f(n-1) + n \\ f(n-1) &= f(n-2) + n - 1 \\ f(n-2) &= f(n-3) + n - 2 \\ &\dots \\ f(1) &= f(0) + 1 \end{aligned}$$

When we add the equations, some terms are canceled

$$f(n) = \underbrace{f(0)}_{0} + \underbrace{1 + 2 + \dots + n}_{\text{Arithmetic series}}$$

Result: $\boxed{f(n) = \dfrac{n(n-1)}{2}}$

FIGURE E-7 Solution for Second Recursive Relation

Example Figure E-8 shows a third recursive relation.

$$f(n) = \begin{cases} 0 & \text{if } n = 1 \\ f(n/2) + 1 & \text{Otherwise} \end{cases}$$

Where n is 2^k

We substitute n with 2^k and repeatedly replace k by k –1

$$\begin{aligned} f(2^k) &= f(2^{k-1}) + 1 \\ f(2^{k-1}) &= f(2^{k-2}) + 1 \\ &\dots \\ f(2^1) &= f(2^0) + 1 \end{aligned}$$

If we add right sides and left sides, some terms are canceled

$$f(2^k) = \underbrace{f(1)}_{0} + \underbrace{1 + 1 + \dots + 1}_{k \text{ times}}$$

$$f(n) = k = \log_2 n$$

The value of k is $\log_2 n$

$$\boxed{f(n) = \log_2 n}$$

FIGURE E-8 Recursive Solution for an n log n Series

Appendix F
Array Implementations of Stacks and Queues

In Chapter 3 we developed a stack ADT using dynamic memory and a linked list implementation. In Chapter 4 we similarly developed a queue ADT. There are times, however, when we either desire to or need to use an array implementation. We discuss stack and queue array implementations in this appendix.

F.1 Stack ADT

If a stack's maximum size can be calculated before the program is written, an array implementation of a stack is more efficient than the dynamic implementation using a linked list. In addition, an array stack is a more easily understood and natural picture of a stack.

When implementing a stack in an array, the base is found at the first stack element, index 0 in C. The top then moves up and down the array as data are inserted and deleted. To push an element into the stack, we add 1 to top and use it as the array index for the new data. To pop an element from the stack, we copy the data at index location top and then subtract 1 from top. One additional metadata element is required: the maximum number of elements in the stack. The structure for the array implementation with a maximum size of 5 elements is shown in Figure F-1.

In this section we redesign the eight basic stack algorithms, using an array rather than a linked list implementation.

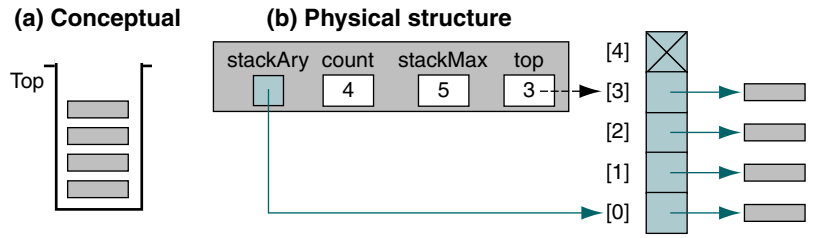


FIGURE F-1 Stack Array Implementation

Array Data Structure

There are three differences in the data structure for an array implementation: first, the stack top is an index rather than a pointer; second, we need to store the maximum number of elements allowed in the stack; and, third, we don't need next fields. Because the stack is a LIFO structure, each element has a physical adjacency to its predecessor.

Although the data structure is significantly different, the array implementation of a stack requires the same basic algorithms used in the linked list implementation. The ADT declaration is shown in Program F-1. The algorithms are developed in the sections that follow.

PROGRAM F-1 Stack Array Definition

```
1  // Stack Definitions for Array Implementation
2     typedef struct
3       {
4        void**  stackAry;
5        int     count;
6        int     stackMax;
7        int     top;
8       } STACK;
9
10 // Prototype Declarations
11    STACK* createStack  (int maxSize);
12    STACK* destroyStack (STACK* queue);
13
14    bool  pushStack  (STACK* stack, void* itemPtr);
15    void* popStack   (STACK* stack);
16    void* stackTop   (STACK* stack);
17    int   stackCount (STACK* stack);
18    bool  emptyStack (STACK* stack);
19    bool  fullStack  (STACK* stack);
20 // End of Stack ADT Definitions
```

Create Stack

The ATD implementation of create stack is shown in Program F-2. The application program specifies the stack maximum size when the stack is created.

PROGRAM F-2 Create Stack

```
 1  /* ================== createStack =================
 2     This algorithm creates an empty stack by allocating
 3     the head structure and the array from dynamic memory.
 4        Pre    maxSize is max number of elements
 5        Post   returns pointer to stack head structure
 6             -or- NULL if overflow
 7  */
 8  STACK* createStack (int maxSize)
 9  {
10  // Local Definitions
11     STACK* stack;
12
13  // Statements
14     stack = (STACK*) malloc( sizeof (STACK));
15     if (!stack)
16        return NULL;
17
18     // Head allocated. Initialize & allocate stack.
19     stack->count    =  0;
20     stack->top      = -1;
21     stack->stackMax = maxSize;
22     stack->stackAry =  (void**)calloc(stack->stackMax,
23                                sizeof(void*));
24     if (!stack->stackAry)
25        {
26         free (stack);
27         return NULL;
28        } // if
29     return stack;
30  }  // createStack
```

Program F-2 Analysis

There are two possible errors in the create stack function; both involve memory allocation. In statement 14 we allocate memory for the stack head structure. If this allocation fails, we return a null pointer. Then, in statement 22, we use *calloc* to allocate the array of *void* pointers and store its address in the stack head structure. Again, if this allocation fails, we return a null pointer. Note that the stack array in the head structure is a pointer to an array of *void* pointers. This means that when we cast the pointer type, it must be cast as a pointer to a pointer to *void*.

Because the newly created stack is by definition empty, we set the stack top index to −1. We must use −1 as the value for a null stack because a stack with only one item in it has a stack top of zero.

Push Stack

The logic for the array implementation of push stack is very simple. We first test for overflow by comparing the count of the elements in the stack with the maximum stack size. If the stack is full, we return false, indicating that the push failed. If there is room, we copy the data to the stack, increase the top index and the stack count, and return true for success. The function code is shown in Program F-3.

PROGRAM F-3 Push Stack

```
/* ================== pushStack =================
   This function pushes an item onto the stack.
      Pre  stack is pointer to stack head structure
           dataInPtr is pointer to be inserted
      Post returns true if success; false if overflow
*/
bool pushStack (STACK* stack, void* dataInPtr)
{
// Statements
   if (stack->count == stack->stackMax)
      return false;

   (stack->count)++;
   (stack->top)++;
   stack->stackAry[stack->top] = dataInPtr;

   return true;
}  // pushStack
```

Pop Stack

The array implementation of pop stack is also quite simple and parallels the linked list implementation. The code is shown in Program F-4.

PROGRAM F-4 Pop Stack

```
/* ================== popStack ==================
   This function pops the item on the top of the stack.
      Pre  stack is pointer to stack head structure
      Post returns pointer to user data if successful
                   NULL if underflow
*/
void* popStack (STACK* stack)
{
// Local Declarations
   void* dataPtrOut;

```

continued

PROGRAM F-4 Pop Stack *(continued)*

```
12  // Statements
13     if (stack->count == 0)
14         dataPtrOut = NULL;
15     else
16        {
17         dataPtrOut = stack->stackAry[stack->top];
18         (stack->count)--;
19         (stack->top)--;
20        } // else
21
22     return dataPtrOut;
23  }  // popStack
```

Program F-4 Analysis

When working with data structures, you should always test what happens when you delete the only item in the structure. For our array stack, we need to ensure that when the only item in the stack is deleted, the stack is properly set to a null status. We have defined a null stack as a stack with a top index of −1. If there is only one element in the stack, the top is 0. Subtracting 1 from 0 gives us −1, our designated flag for a null stack.

Stack Top

Stack top is a simple function. We test for data in the stack and return the data pointer in the top element if the stack is not empty or a null pointer if it is empty. The code is shown in Program F-5.

PROGRAM F-5 Stack Top

```
1   /* ==================== stackTop ====================
2      This function retrieves the data from the top of the
3      stack without changing the stack.
4         Pre  stack is pointer to the stack
5         Post returns data pointer if successful
6                 -or- null pointer if stack empty
7   */
8   void* stackTop (STACK* stack)
9   {
10  // Statements
11     if (stack->count == 0)
12        return NULL;
13     else
14        return stack->stackAry[stack->top];
15  }  // stackTop
```

Empty Stack

Empty stack logic is the same regardless of the implementation structure. It simply tests the stack count in the head structure and returns true if it is 0 or false if it is not. The code is shown in Program F-6.

PROGRAM F-6 Empty Stack

```
/* ================= emptyStack ================
   This function determines if a stack is empty.
      Pre  stack is a pointer to the stack
      Post returns true if empty; false if data in stack
*/
bool emptyStack (STACK* stack)
{
// Statements
   return (stack->count == 0);
}  // emptyStack
```

Full Stack

To determine whether the stack is full, we simply compare the count with the maximum number of elements allocated for the array. If they are equal, the stack is full; if not, the stack has room for more data. The code is shown in Program F-7.

PROGRAM F-7 Full Stack

```
/* ================= fullStack ================
   This function determines if a stack is full.
      Pre  stack is a pointer to a stack head structure
      Post returns true if full; false if empty elements
*/
bool fullStack (STACK* stack)
{
// Statements
   return (stack->top == stack->stackMax);
}  // fullStack
```

Stack Count

Like empty stack, the logic for stack count is identical in all implementations. We simply return the stack count found in the stack head structure. The code is shown in Program F-8.

PROGRAM F-8 Stack Count

```
/* ================== stackCount =================
   Returns number of elements in stack.
      Pre  stack is a pointer to the stack
      Post count returned
*/
int stackCount(STACK* stack)
{
// Statements
   return stack->count;
}  // stackCount
```

Destroy Stack

To destroy a stack implemented using an array, we must first free the array structure and then the stack head structure. We then return a null pointer, which the calling function should store in the application stack pointer. The code is shown in Program F-9.

PROGRAM F-9 Destroy Stack

```
/* ================= destroyStack ================
   This function releases all memory to the heap.
   Pre  stack is pointer to stack head structure
   Post returns null pointer
*/
STACK* destroyStack (STACK* stack)
{
// Statements
   if (stack)
      {
       // Release data memory
       for (int i = 0; i < stack->count; i++)
           free (stack->stackAry[i]);

       // Release stack array
       free (stack->stackAry);

       // Now release memory for stack head
       free (stack);
      } // if stack
   return NULL;
}  // destroyStack
```

Program F-9 Analysis Our only concern is that the stack pointer may be null. If it is, we do nothing except return a null pointer. If the stack pointer is not null, we free the array memory first, then the head structure memory, and return a null pointer.

F.2 Queue ADT

Although the linked list implementation is very popular, it is not the only way to implement a queue. Queues can also be implemented in arrays and, for very large queues, in files. Because a file can be thought of as an array stored on a disk, the array and file implementations have similar solutions. In this section we rewrite the queue ADT using an array.

An enqueue to an empty queue is placed in the first element of the array, which becomes both the front and the rear. Subsequent enqueues are placed at the array location following rear; that is, each enqueue stores the queue data in the next element after the current queue rear. Thus, if the last element in the queue is stored at array location 11, the data for the next enqueue is placed in element 12.

Dequeues take place at the front of the queue. As an element is deleted from the queue, the queue front is advanced to the next location; that is, queue front becomes queue front plus one. Figure F-2(a) shows a conceptual queue; Figure F-2(b) shows a physical queue after it has been in operation for a period of time. At the point shown, the data have migrated from the front of the array to its center.

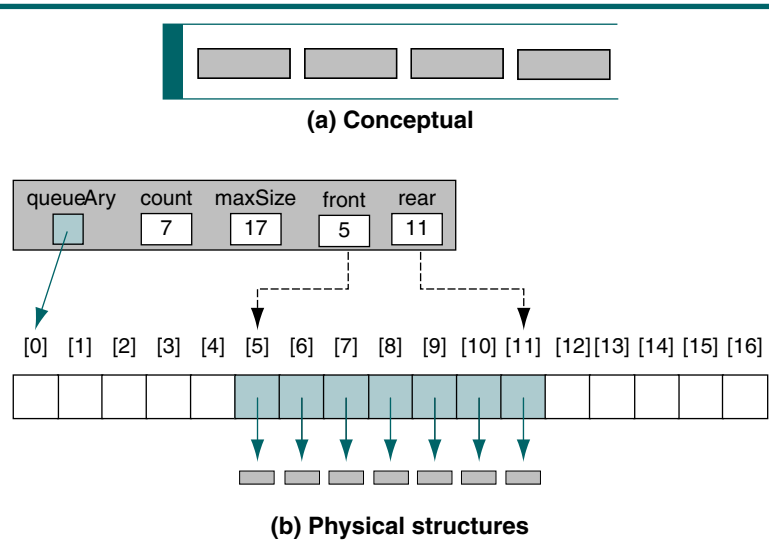


FIGURE F-2 Queue Stored in an Array

When we implement a queue in an array, we use indexes rather than pointers. Thus, the front of the queue in Figure F-2 is 5 and the rear is 11.

The definition of a full queue changes when we implement a queue in an array. A full queue is defined as every element filled. Because arrays have a

finite number of elements, we can determine that the queue is full by testing the queue count against the maximum number of elements.[1]

When data arrive faster than the queue service time, the queue tends to advance to the end of the array. This leads to the situation where the last element in the array is occupied but the queue is not full because there are empty elements at the beginning of the array. This situation is shown in Figure F-3.

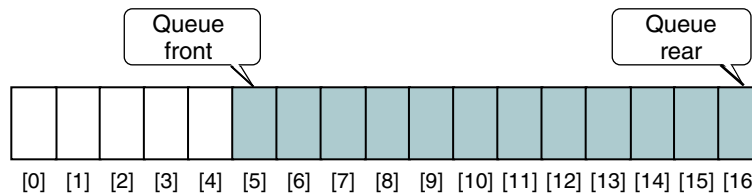


FIGURE F-3 Array Queue with Last Element Filled

When the data are grouped at the end of the array, we need to find a place for the new element when we enqueue data. One solution is to shift all of the elements from the end to the beginning of the array. For example, in Figure F-3 element 5 is shifted to element 0, element 6 to 1, 7 to 2, and so forth until element 16 is shifted to 11.

A more efficient alternative is to use a circular array. In a circular array, the last element is logically followed by the first element. This is done by testing for the last element and, rather than adding one, setting the index to zero. Given our array queue above, the next element after 16 is 0. A circular queue is shown in Figure F-4.

With this understanding of a circular queue, we are ready to rewrite the abstract data type. The only difference in the calling sequence between the two implementations is the addition of a maximum queue size in the create queue function.

Array Queues Implementation

Two changes are needed to the data structure for the queue head. We need to store the address of the queue array, which we allocate from dynamic memory. We also need to store the maximum number of elements in the array. The revised data structure is shown in Program F-10. We also include the prototype declarations for the queue functions. As you study the data structure, compare it with the linked list structure in Program 4-1, "Queue ADT Data Structures."

1. Even though we are implementing the queue in C, we ignore the possibility of dynamically changing the array size with *realloc*.

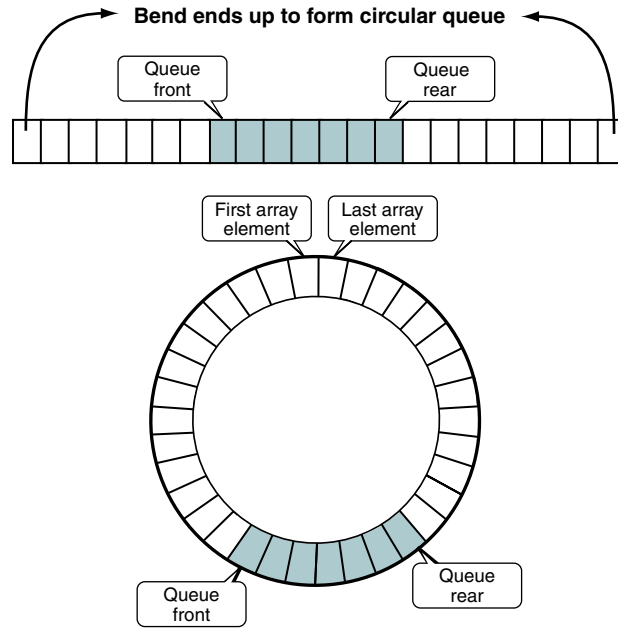


FIGURE F-4 Circular Queue

PROGRAM F-10 Queue Array Definition

```
1  // Queue ADT Defintions
2     typedef struct
3        {
4         void**   queueAry;
5         int      maxSize;
6         int      count;
7         int      front;
8         int      rear;
9        } QUEUE;
10
11 // Prototype Declarations
12    QUEUE* createQueue  (int maxSize);
13    QUEUE* destroyQueue (QUEUE* queue);
14
15    bool  enqueue    (QUEUE* queue, void* itemPtr);
16    void* dequeue    (QUEUE* queue);
17    void* queueFront (QUEUE* queue);
18    void* queueRear  (QUEUE* queue);
19    int   queueCount (QUEUE* queue);
20    bool  emptyQueue (QUEUE* queue);
21    bool  fullQueue  (QUEUE* queue);
22 // End of Queue ADT Definitions
```

Create Queue

The array implementation of create queue allocates memory for a queue head structure and the array itself in dynamic memory and returns the address of the head structure to the caller. If there is not enough memory for the queue structure, it returns a null pointer. The code for create queue is shown in Program F-11.

PROGRAM F-11 Create Queue

```
 1  /* =============== createQueue ================
 2     Allocates memory for a queue head node from dynamic
 3     memory and returns its address to the caller.
 4        Pre    nothing
 5        Post   head has been allocated and initialized
 6        Return head's address if successful;
 7               null if overflow
 8  */
 9  QUEUE* createQueue (int maxSize)
10  {
11  // Local Definitions
12     QUEUE* queue;
13
14  // Statements
15     queue = (QUEUE*) malloc (sizeof (QUEUE));
16     if (queue)
17        {
18         // head structure created. Now allocate queue
19         queue->queueAry =
20              (void**)calloc(maxSize, sizeof(void*));
21         queue->count   =  0;
22         queue->front   = -1;
23         queue->rear    = -1;
24         queue->maxSize = maxSize;
25        } // if
26
27     return queue;
28  }  // createQueue
```

Program F-11 Analysis

The most difficult code in create queue is found in statement 19. We must cast the array pointer as a pointer to a *void* pointer. This is because we cannot store the data directly in the ADT. The calling function must pass a pointer to data in dynamic memory. We can then store it in the queue array as a *void* pointer. Because the array is an array of *void* pointers, its name is a pointer constant to a *void* pointer. This means that the pointer returned by *calloc* must be a pointer to a *void* pointer.

A second point worth noting is that we set the front and rear indexes to −1. We have chosen −1 as the index for a null queue to make the enqueue logic as simple as possible. Whenever we insert an element into the queue, we add one to the rear index. When the queue is null, this automatically sets rear to zero when the queue is null. The initial value for the front queue could be anything, but we make it −1 for consistency.

Enqueue

The parameter list for the array implementation of enqueue is identical to the linked list implementation. The logic, however, differs considerably. First, there is no memory to be allocated. We simply need to determine in which array element to store the data pointer.

The test for a full queue has also been changed. To determine if the queue is full, we compare the queue count to the maximum queue size. If they are equal, we have a full queue.

To complete the insert, we add one to the rear index. If rear is at the last element, we need to wrap around to the first element of the queue. The code is shown in Program F-12.

PROGRAM F-12 Enqueue

```
1  /* ================ enqueue ================
2     This algorithm inserts data into a queue.
3        Pre    queue has been created
4        Post   data have been inserted
5        Return true if successful; false if overflow
6  */
7  bool enqueue (QUEUE* queue, void* itemPtr)
8  {
9  // Statements
10    if (queue->count == queue->maxSize)
11       return false;
12
13    (queue->rear)++;
14    if (queue->rear  == queue->maxSize)
15        // Queue wraps to element 0
16        queue->rear  =  0;
17    queue->queueAry[queue->rear] = itemPtr;
18
19    if (queue->count == 0)
20       {
21        // Inserting into null queue
22        queue->front  = 0;
23        queue->count  = 1;
24       } // if
25    else
26       (queue->count)++;
27    return true;
28 }  // enqueue
```

Program F-12 Analysis

You need to study two points in this function carefully. First, note how the wraparound logic in statements 14 to 16 is handled. At this point in the function, we know that the queue is not full. If the rear index is equal to the maximum queue size, however, we need to wrap around to the first element. (Remember, C uses zero indexing, so when the new rear is equal to the count, the last element has been filled.)

Second, if we are inserting into a null queue, identified by a count of zero, we must set the front index to 0. The rear index is set to 0 automatically when we add one to the null rear pointer, which is set to −1 whenever the queue is empty.

Dequeue

Dequeue deletes data at the front of the queue and returns it to the calling function. Its code is shown in Program F-13.

PROGRAM F-13 Dequeue

```
1  /* ================== dequeue ==================
2     This algorithm deletes a node from the linked list.
3        Pre  queue has been created
4        Post returns pointer to user data if successful
5                     NULL if underflow
6  */
7  void* dequeue (QUEUE* queue)
8  {
9  // Local Declarations
10    void* dataPtrOut;
11
12 // Statements
13    if (!queue->count)
14       return NULL;
15
16    dataPtrOut = queue->queueAry[queue->front];
17    (queue->front)++;
18    if (queue->front == queue->maxSize)
19       // queue front has wrapped to element 0
20       queue->front =  0;
21    if (queue->count == 1)
22       // Deleted only item in queue
23       queue->rear = queue->front = -1;
24    (queue->count)--;
25
26    return dataPtrOut;
27 }  // dequeue
```

Program F-13 Analysis

Take note of two pieces of code in this function. First, once again we must wrap around from the end of the queue. For dequeue we test for a wrap after we have deleted the data from the queue. If front was in the last element of the array, we must set the new front to the first element, 0.

The second point is that we need to reset both front and rear to −1 when we delete the last item in the queue. This is done in statements 21 to 23.

Queue Front

The queue front logic parallels dequeue except that the status of the queue is not changed. Its logic is shown in Program F-14.

PROGRAM F-14 Queue Front

```
1  /* ================== queueFront =================
2     Retrieve the data at the front of the queue
3     without changing the queue contents.
4        Pre  queue is a pointer to initialized queue
5        Post returns pointer to user data if successful;
6                     NULL if underflow
7  */
8  void* queueFront (QUEUE* queue)
9  {
10 // Statements
11    if (!queue->count)
12        return NULL;
13    return queue->queueAry[queue->front];
14 }  // queueFront
```

Queue Rear

Queue rear returns a pointer to the data at the rear of the queue without changing the contents of the queue. It's logic is shown in Program F-15.

PROGRAM F-15 Queue Rear

```
1  /* ================== queueRear =================
2     Retrieves the data at the rear of the queue
3     without changing the queue contents.
4        Pre  queue is pointer to initialized queue
5        Post returns pointer to user data if successful;
6                     NULL if underflow
7  */
8  void* queueRear (QUEUE* queue)
9  {
10 // Statements
11    if (!queue->count)
12        return NULL;
13    return queue->queueAry[queue->rear];
14 }  // queueRear
```

Full Queue

The full queue logic is much simpler in the array implementation than in the linked list. We simply test the count to the maximum queue size. If they are equal, the queue if full. The code is shown in Program F-16.

Empty Queue

The empty queue logic is much simpler in the array implementation than in the linked list. We simply test the count to the maximum queue size. If they are equal, the queue is full. The code is shown in Program F-17.

PROGRAM F-16 Full Queue

```
/* ================== fullQueue =================
   Checks to see if a queue is full. The queue is full
   if the array is full.
      Pre    queue is pointer to a queue head node
      Return true if full; false if room for more
*/
bool fullQueue (QUEUE* queue)
{
// Statements
   return (queue->count == queue->maxSize);
}  // fullQueue
```

PROGRAM F-17 Empty Queue

```
/* ==================== emptyQueue ==================
   Checks to see if a queue is empty.
      Pre    queue is pointer to a queue head node
      Return true if empty; false if data in queue
*/
bool emptyQueue (QUEUE* queue)
{
// Statements
   return (queue->count == 0);
}  // emptyQueue
```

Queue Count

The queue count returns the number of elements in the queue. The code is shown in Program F-18.

PROGRAM F-18 Queue Count

```
/* ==================== queueCount ==================
   Returns the number of elements in the queue.
      Pre    queue is pointer to a queue head node
      Return queue count
*/
int queueCount (QUEUE* queue)
{
// Statements
   return queue->count;
}  // emptyQueue
```

Destroy Queue

The last function for the array implementation is destroy queue. The queue count and empty queue functions are identical for the linked list implementation and the array implementation, so we won't repeat them.

To destroy the queue, we simply free the array and head structures and return a null pointer. The code is shown in Program F-19.

PROGRAM F-19 Destroy Queue

```
/* ================= destroyQueue ================
   This function deletes the queue and array memory.
      Pre    queue is a valid queue
      Post   all data have been deleted and recycled
      Return null pointer
*/
QUEUE* destroyQueue (QUEUE* queue)
{
// Local Definitions
   int locn = queue->front;

// Statements
   if (queue)
      {
       // Free data memory
       while (queue->count != 0)
          {
           free (queue->queueAry[locn++]);
           queue->count--;
           if (locn == queue->maxSize)
               locn = 0;
          } // while
       free (queue->queueAry);
       free (queue);
      } // if
   return NULL;
}  // destroyQueue
```